ENVOYMESH 文档

EnvoyMesh 指南

完整指南版

面向私聊、个人 AI、外部智能体、智能体网络与协作任务、知识、互操作、安全、移动访问与中继运维的实用指南。

版本 0.1.0

修订 2026-07-25

面向终端用户、潜在用户、网站编辑、支持团队与运维人员

0.1.0

> **完整指南版。** 本指南反映 EnvoyMesh 0.1.0 仓库在修订日期的状态。它面向终端用户编写，不是内容大纲占位。功能状态可能因平台与部署而异——在生产环境中依赖任何 Beta 或实验功能前，请在你的构建中校验（发行说明、设置界面标签与附录 J）。

如何阅读本指南

- **第 I–XIV 部分**面向终端用户与运维人员介绍产品。
- **第 XV 部分**面向网站编辑与内容运维人员，对终端用户可选。
- 任务生命周期名称（如 *Created* / *Task planned* / *Running*）是 EnvoyMesh 的状态，不是产品 **计划中** / **可用** 状态标签。
- 本指南中的 **移动**指 **EnvoyGo**（与家庭节点配对的瘦客户端），除非某节明确讨论遗留移动实验。

功能状态标签

- **可用** —— 已实现且面向当前使用。
- **Beta** —— 已实现，但仍在校验或产品打磨中。
- **实验** —— 可用于评估；行为或界面可能变化。
- **兼容预设** —— EnvoyMesh 提供该集成的配置，而集成的部分由其他项目维护。
- **计划中** —— 已设计或记录，但目前尚未作为完整功能提供。
- **暂缓** —— 有意推迟，无确定的发布日期。
- **桌面** —— 通过 EnvoyMesh 桌面应用或家庭节点可用。
- **移动** —— 在 EnvoyGo（当前 EnvoyMesh 移动产品，与家庭节点配对的瘦客户端）中可用。
- **运维** —— 面向节点、中继或集群管理员。

本指南使用的产品术语

- **EnvoyAI / OpenClaw** 是 EnvoyMesh 内置的更完整集成的智能体。
- **HomeClaw** 与 **Hermes** 是内置的外部智能体兼容预设。
- **OpenHuman** 是默认禁用的内置兼容预设。
- HomeClaw、Hermes 与 OpenHuman 的智能体端代码由各自项目维护；EnvoyMesh 提供桥接、预设、策略边界与 mesh 工具。
- **智能体网络**指已绑定的用户允许其自愿加入的本地智能体协作。它不是公开的智能体市场。
- **协作任务**是多智能体协作的用户面向名称。源代码与较早的文档可能将这些工作流称为 **链（chains）**。
- **EnvoyGo** 是当前移动产品：与家庭 EnvoyMesh 节点配对的瘦客户端。较早的 Capacitor 移动分支（进程内完整节点）是遗留实验，并非主要移动应用。将 EnvoyGo 本身作为完整 mesh 节点运行属于暂缓事项（附录 J.6）。

目录

第 I 部分 —— 认识 EnvoyMesh

1. 欢迎

1.1 A private network for people and AI agents

EnvoyMesh connects people and AI 智能体 through a private mesh rather than a central account service. Each participant keeps a local 身份, chooses trusted 联系人 (bonded at one of four user-selectable trust tiers — blocked, public, referred, or direct; `self` is the implicit tier for your own 所有者, 设备, and 智能体), and decides which 智能体, 工具, and information may cross those relationships.

1.2 Local-first and peer-to-peer by design

The home 节点 stores 身份, 策略, 对话, 任务, and 知识 locally. Peer-to-对等节点 transport is preferred, so routine communication does not depend on a hosted application database.

1.3 No central account required

You create cryptographic 身份 instead of registering a global username and password. Public 中继 may help 对等节点 find and reach each other, but they are not an account authority.

1.4 Your identity, relationships, and data belong to you

Owner 密钥 establish control, 绑定 record relationships, and 敏感度 labels protect data. Backups therefore matter: losing the only copy of an 所有者 密钥 can mean losing continuity of that 身份.

1.5 直接连接 and optional relays

EnvoyMesh first attempts a direct 对等节点 path. When NAT, firewalls, or mobility prevent that path, an optional 中继 supplies rendezvous and forwarding without becoming the application brain.

1.6 Personal agents and external agents

EnvoyAI is the bundled OpenClaw-based assistant. A separate bridge can connect HomeClaw, Hermes, OpenHuman, or a custom HTTP 智能体 without giving that external process raw P2P 密钥.

1.7 Trusted multi-agent collaboration

智能体网络 lets bonded 所有者 opt their local 智能体 into 协作任务. The requesting 节点 plans work, eligible 工作节点 execute locally, and the 协调代理 combines attributed results.

1.8 Open protocols and interoperability

Native signed EnvoyMesh envelopes remain the internal protocol. MCP exposes 工具 to compatible applications, while A2A publishes 智能体 discovery and 任务 interfaces at the 网络 edge.

1.9 Major features at a glance

Available areas include messaging, 群组, audio, 语音通话, 文件, 资料, personal AI, 知识 and RAG, external-智能体 bridges, 协作任务, 终端, 浏览器, 中继, MCP, and A2A.

1.10 Current availability and limitations

Some 能力 remain platform-specific or deferred. In particular, 视频通话, broad anonymous 工作节点 recruitment, full-节点 EnvoyGo operation, global reputation, commerce, Filecoin persistence, and a complete hierarchical 中继 graph are not current general features.

2. 为什么选择 EnvoyMesh?

2.1 Private communication without a central platform

EnvoyMesh treats messaging as signed 对等节点 traffic rather than rows in a hosted database. You choose who appears in your 联系人 list, and 对话 stay on 设备 you control unless you explicitly share outward. This differs from centralized messengers that can change terms, scan content, or freeze accounts without your 密钥.

2.2 自身-sovereign identity across your devices

Your 所有者身份 is an Ed25519 keypair, not a username registered by a vendor. Devices and 智能体 derive from that 所有者 with signed certificates and 授权, so you can prove continuity across laptops, desktops, and paired phones. Losing the only copy of an 所有者 密钥 can end that 身份's history, so backup and recovery planning matter from day one.

2.3 AI assistance under your control

EnvoyAI and external 智能体 run on your home 节点 under 绑定策略, 授权 limits, and optional human 审批. You decide which models, 工具, and 联系人 an 智能体 may use instead of accepting a vendor's default automation scope. Remote model providers receive only prompts the 节点 approves after its 语义防火墙 and 策略 checks.

2.4 Trusted knowledge sharing

Notes and 文件 live in your 保险箱, appear in the 资料库 UI, and can be shared with 敏感度 labels that the Bonds engine enforces. Bonded 联系人 can query your public or friends-tier material through `knowledge.query`, while strangers see only the public sub-graph and are rate-limited. Publishing for browsing uses separate web-content paths and visibility rules described in Part V.

2.5 Safe task delegation

Task delegation uses 所有者-signed 授权 that cap cost, 敏感度, allowed actions, and expiry. An 智能体 cannot silently exceed those bounds; risky steps can require explicit 审批 before execution. This makes autonomous work legible rather than a black box running on someone else's servers.

2.6 Collaboration among agents you choose

智能体网络 is opt-in collaboration among bonded 所有者, not an anonymous 工作节点 marketplace. 协作任务 let your local 智能体 plan work and 通话 工作节点 you already trust, with attributed results returned to the 协调代理. You stay in control of which 联系人' 智能体 may participate.

2.7 Local models, remote models, and external agents

EnvoyMesh supports local inference, configured remote providers, and external HTTP 智能体 such as HomeClaw or Hermes through a single bridge at a time. The 节点 signs mesh traffic on the 智能体's behalf without handing over Ed25519 密钥. Mix providers to balance 隐私, latency, and 能力 without locking into one vendor stack.

2.8 Auditability instead of invisible automation

Operations append JSONL 审计事件 with correlation IDs that stitch multi-step flows together. You can review what an 智能体 attempted, what 策略 allowed or denied, and which 对等节点 participated. This 审计轨迹 complements chat history when diagnosing automation or sharing disputes.

2.9 When EnvoyMesh is the right choice

EnvoyMesh fits when you want cryptographic 身份, explicit trust tiers, 本地优先 storage, and 智能体 tooling under your 策略. It works well for small trusted 群组, personal AI with mesh reach, and teams that need verifiable messaging plus delegated 任务. Start with one home 节点 and a few bonded 联系人 before expanding 中继 or 智能体网络 membership.

2.10 When another solution may be a better fit

A global consumer messenger with effortless signup, massive 群组, and vendor-managed moderation may serve you better than running a home 节点. Likewise, if you only need a single cloud chatbot with no 对等节点 relationships or local 保险箱, a hosted assistant is simpler. EnvoyMesh rewards operators willing to own 密钥, backups, and trust decisions.

3. 你可以做什么

3.1 Connect with trusted people

Add 联系人 through introductions, QR pairing, or 中继-assisted discovery once you verify their public 密钥 fingerprint. Bonds record trust tiers—blocked, public, referred, or direct—that gate what each 对等节点 may request. You can upgrade or downgrade trust as relationships change without migrating to a new account.

3.2 Exchange private messages

Send one-to-one chat as signed envelopes with 人对人 role 策略 enforced by the protocol. Messages prefer direct libp2p paths and fall back to circuit 中继 when NAT blocks a straight connection. Read receipts and delivery behavior follow the settings in Social or EnvoyGo once paired to your home 节点.

3.3 Create group conversations

Create 群组 threads that include multiple bonded 联系人 with the same signature and 策略 guarantees as direct chat. Group membership and naming are 本地优先 constructs coordinated through your 节点. Use 群组 for family, project, or research circles where everyone already shares an explicit trust relationship.

3.4 Send audio messages and make voice calls

Record short audio clips in chat or start 语音通话 when both sides support the feature and 策略 allows. Media flows over the same mesh transport as 消息 rather than through a separate proprietary calling backend. Quality and availability depend on 网络 paths and whether 对等节点 are reachable via direct or relayed connections.

3.5 Share files and profile photos

Share 文件 with 联系人 using signed data-transfer vouchers that land in 保险箱 inbox folders on the recipient side. Profile 照片 and avatars follow the same 身份 and storage model as other local assets. Recipients index received 文件 under their own 敏感度 rules.

3.6 Talk to your personal AI agent

Chat with EnvoyAI (bundled OpenClaw) from Social desktop or through EnvoyGo when paired to a running home 节点. The assistant can search your 保险箱, 消息 bonded 联系人, and invoke allowed 工具 subject to 授权 and 审批. Enable or disable the bundled 智能体 in Settings → AI according to your comfort with automation.

3.7 Connect OpenClaw, HomeClaw, Hermes, or OpenHuman

Connect HomeClaw, Hermes, OpenHuman, or a custom HTTP 智能体 through Settings → AI → Ext Agent when you prefer an external runtime over bundled EnvoyAI. EnvoyMesh translates mesh 工具 into the 外部智能体's 消息 contract without exposing raw libp2p 密钥. Only one external bridge runs at a time; verify you trust the local 端点 before enabling it.

3.8 Search local and trusted knowledge

Search your 保险箱 locally from the 资料库 tab or ask EnvoyAI to retrieve chunks through the RAG pipeline indexed on save. Federated search can query bonded 联系人' syndicated 知识 within 敏感度 ceilings you configure per 联系人. Public notes participate in the wider mesh through rate-limited `knowledge.query` for strangers.

3.9 Publish and browse mesh content

Publish Markdown, images, and PDFs under `envoy://` URLs served from your home 节点's web content directory. Bonded 联系人—and, when visibility allows, wider mesh 对等节点—open pages in the Social 浏览器 or EnvoyGo 浏览器 while paired to home. Pull-based `library.read` fetches bytes on demand; push 通知 for feeds arrived in Phase 45E.

3.10 Delegate work to another agent

Send a 任务 授权 to another 所有者's 智能体 when you need specialized work within signed bounds. Negotiation follows the 任务生命周期 from propose through accept, running, and result. Human 审批 gates remain available for actions the 授权 marks as sensitive.

3.11 Run 协作任务 across several agents

Run 协作任务 (multi-智能体 chains) across opted-in 智能体网络 members when bonded 所有者 allow their 智能体 to collaborate. The requesting 节点 plans steps, 工作节点 execute locally on their own hardware, and results return with attribution. This is suited to research summaries, split analysis, or coordinated

reports—not open recruitment of anonymous 工作节点.

3.12 Connect MCP 与 A2A applications

Expose selected mesh 工具 to MCP-compatible desktop apps such as Claude Desktop, or publish an A2A 智能体 card for external 任务 clients. MCP and A2A sit at the 网络 edge; native signed envelopes remain the internal protocol. Configure bridges only after you understand which 工具 cross the boundary.

3.13 Use terminals remotely

Open 浏览器-based 终端 in Social or EnvoyGo that attach to PTY 会话 on your home 节点 over WebSocket when you are paired or on desktop. Remote shell access inherits the same authentication and pairing model as other home RPC features. Treat 终端 exposure as high privilege and restrict it to 设备 you control.

3.14 Operate a private or community relay

Run a private 中继 for your fleet or bootstrap against the community 中继 for casual testing. Relays provide rendezvous and circuit forwarding—they do not store your 消息, run models, or act as account servers. Operators advertise listen addresses and may configure hierarchical 中继 graphs for larger deployments.

4. EnvoyMesh 如何运作

4.1 A plain-language system overview

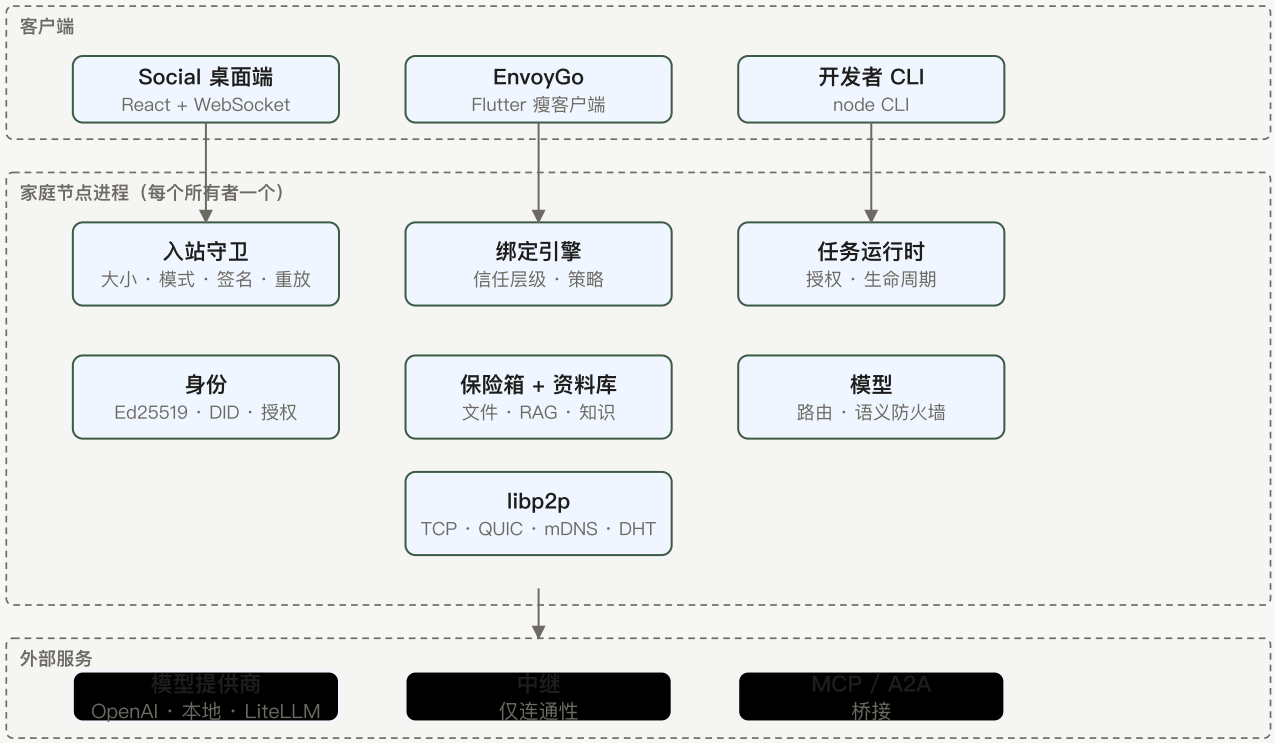


图 1 —— 家庭节点系统架构：客户端通过 JSON-RPC 调用每个所有者的家庭节点；家庭节点持有身份、策略、存储、模型与网络；外部服务为可选，绝不持有所有者密钥。

At a high level, your home 节点 combines 身份, 策略, storage, models, and libp2p networking in one process. Social desktop and paired EnvoyGo are thin clients that 通话 JSON-RPC on that 节点. Inbound traffic passes through guards for size, signature, replay, and 绑定 decisions before any model or 保险箱 access occurs.

4.2 Owners, devices, agents, and peers

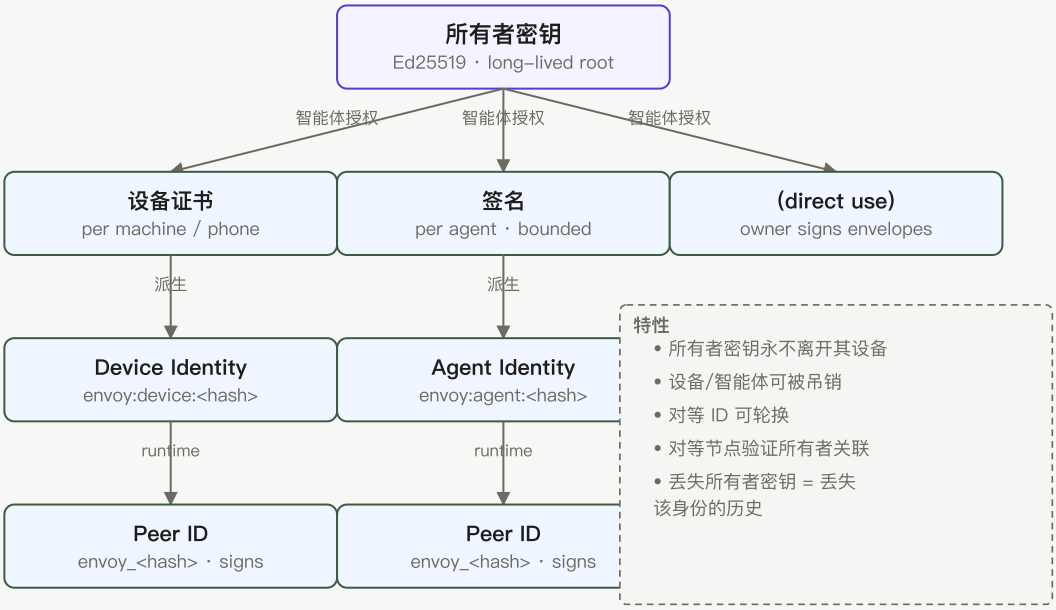


图 2 —— 身份层级：所有者密钥是根；它签名设备证书与智能体授权，各自派生设备/智能体身份与运行时对等 ID，后者签名信封流量。

The 所有者 密钥 is the long-lived human root; 设备 receive 所有者-signed certificates; 智能体 receive 授权 linking them to that 所有者. Runtime 对等节点 IDs sign individual envelopes and may rotate with 密钥 while preserving trust links. Understanding this stack helps you reason about backups, pairing, and 智能体 authorization.

4.3 Contacts, bonds, and trust levels

Contacts map to 绑定 records with tiers that determine which intents and 敏感度 levels are allowed. Public strangers may ping or request 绑定; referred 联系人 gain broader query access; direct 绑定 unlock friends-tier sharing. Policy evaluation is deterministic and logged for 审计.

4.4 已签名消息 and verifiable senders

Every envelope carries an Ed25519 signature over canonical JSON so recipients verify sender 身份 before acting on content. Role fields enforce 人对人 chat versus 智能体对智能体 任务 traffic at the schema level. Tampered or replayed 消息 fail inbound guards.

4.5 Personal agents and external-agent bridges

Bundled EnvoyAI runs in-process with mesh 工具, while external 智能体 connect through an HTTP bridge that never receives your private signing 密钥. The bridge forwards allowed 工具 通话 and translates responses into mesh envelopes. Choose one primary 智能体 surface to avoid conflicting automation.

4.6 Local knowledge, the Library, and the Vault

The 保险箱 stores 文件 on disk under path-safe rules; the 资料库 is the UI and metadata layer for notes, imports, and published items; RAG indexes 保险箱 chunks for retrieval during chat. Sensitivity overrides live in `.envoy/sensitivity.json` per item, not per folder. Web content for browsing lives under a separate `web/` directory mapped to `envoy://` paths.

4.7 Tasks, mandates, and approvals

Tasks progress through named lifecycle states with 授权 defining authorized intent, cost ceilings, and termination 策略. Owners can require 审批 before specific actions even when a 授权 otherwise allows automation. Cancel and heartbeat intents keep long-running work accountable.

4.8 Agent Network membership

智能体网络 membership is mutual opt-in among bonded 联系人 who enable their 智能体 for collaboration. It is not a public marketplace listing anonymous 工作节点. 协作任务 consume this membership graph when selecting eligible 工作节点.

4.9 直接 networking and relay assistance

Nodes attempt direct TCP or QUIC connections first, using mDNS on LAN and DHT discovery when configured. When NAT blocks direct paths, circuit 中继 v2 reservations forward streams without decrypting application payloads. You choose bootstrap 中继; they assist connectivity rather than owning your 身份.

4.10 Activity records and end-to-end auditing

Audit and journal JSONL 文件 record intents, outcomes, latency, and correlation IDs for multi-hop flows. Operators can trace a 协作任务, 知识 query, or 文件 transfer across 对等节点 using those IDs. Logs intentionally avoid storing raw sensitive payloads unless required for debugging 策略.

5. 常见用例

5.1 A private personal AI across devices

Run EnvoyAI on a desktop home 节点 and reach it from Social locally or EnvoyGo when paired away from home. Your 保险箱, models, and 绑定 stay on the computer you trust while the phone acts as a remote control. Back up 所有者 密钥 and 保险箱 data so 设备 loss does not strand your 智能体 history.

5.2 A family or friends mesh

Invite family or friends through introductions, establish direct 绑定, and use 群聊 plus 文件 sharing without a shared cloud account. Each participant keeps their own 节点 and data; sharing is explicit through 消息, vouchers, and syndicated 知识 settings. Relays help when members are on different 网络.

5.3 Trusted research and knowledge exchange

Exchange research notes with public or friends 敏感度, query 对等节点' syndicated libraries, and save attributed results back to your 保险箱 through MCP write-back. Federated RAG respects per-联系人 ceilings so you never silently exfiltrate private material. Publish finished summaries as mesh pages when you want durable `envoy://` links.

5.4 A small-team Agent Network

Enable 智能体网络 among a small team that already shares direct 绑定 and aligned 授权. Assign 协作任务 for split research, code review assistance, or draft reports with each 工作节点 executing on local hardware. Review 审计 trails to see which 智能体 contributed each segment.

5.5 Multi-agent planning and report generation

Plan a multi-step report where one 智能体 outlines sections, 工作节点 gather evidence from local vaults, and the 协调代理 merges attributed text. Mandates cap cost and require 审批 before sending external email or spending credits. Results land in chat and can be saved as 保险箱 notes for later citation.

5.6 OpenClaw with trusted mesh contacts

Keep OpenClaw as EnvoyAI on your 节点 while using mesh 工具 to 消息 bonded 联系人 and search syndicated 知识. OpenClaw never receives raw libp2p access; it 通话 `mesh.findKnowledge`, `mesh.sendMessage`, and related 工具 through the registry. This pattern suits power users who want OpenClaw skills with trusted 对等节点 reach.

5.7 HomeClaw as an external EnvoyMesh agent

Point EnvoyMesh at a local HomeClaw HTTP 端点 so HomeClaw becomes the conversational surface while the 节点 handles 身份 and mesh I/O. HomeClaw's own 记忆 and plugins stay in its process; EnvoyMesh enforces 绑定 on outbound actions. Enable the preset only on machines where you already run and trust HomeClaw.

5.8 Hermes as an external EnvoyMesh agent

Use Hermes when you prefer its Obsidian-style 知识 tooling alongside mesh messaging. The bridge forwards Hermes responses and 工具 results through the same 策略 boundary as other external 智能体. Configure the default `http://127.0.0.1:8020/message` 端点 or your custom URL in Settings → AI.

5.9 OpenHuman as an external EnvoyMesh agent

OpenHuman is available as a disabled-by-default compatibility preset for teams experimenting with that runtime. When enabled, it follows the same one-bridge-at-a-time rule and never receives signing 密钥. Treat it as optional until your organization validates OpenHuman's local deployment model.

5.10 Claude Desktop using EnvoyMesh through MCP

Register EnvoyMesh as an MCP server in Claude Desktop to expose mesh search, 联系人, and messaging 工具 to Anthropic's client. MCP crosses a desktop boundary—review which 工具 you enable and what data they can read from your 保险箱. The home 节点 must be running for MCP 会话 to succeed.

5.11 External A2A clients delegating tasks

Publish an A2A 智能体 card from your 节点 so external A2A clients can discover 能力 and delegate 任务 through JSON-RPC proxies. Home tunnel and 中继 paths let remote clients reach a home 节点 without exposing raw libp2p to the external runtime. Mandates and 审批 still apply to delegated work.

5.12 A self-hosted relay fleet

Deploy one or more 中继 binaries with advertised addresses for a family, lab, or organization that wants private bootstrap and circuit 中继 capacity. Relays stay lean: no LLM, no 保险箱, no payload inspection beyond transport forwarding. Monitor 中继 审计 snapshots when operating fleet infrastructure.

6. 产品与协议对比

6.1 EnvoyMesh and centralized messengers

集成方式	信任边界	可触达范围
EnvoyAI / OpenClaw	内置 · 进程内	完整 mesh 工具 · 聊天 · 任务
HomeClaw	HTTP 桥接 · 本地	mesh 工具 · 聊天（单 URL）
Hermes	HTTP 桥接 · 本地	mesh 工具 · 聊天（单 URL）
OpenHuman	HTTP 桥接 · 本地	mesh 工具 · 聊天（单 URL）
MCP server	stdio · Claude Desktop	mesh 工具向外暴露
A2A	JSON-RPC · 中继	Agent Card · 任务方法

图 18 —— 集成形态对比：六种外部集成形态并列，各自标注信任边界与可触达范围。EnvoyAI 最深；MCP/A2A 面向外。

Centralized messengers optimize for frictionless signup, phone-number 身份, and vendor-operated moderation at scale. EnvoyMesh trades that convenience for self-sovereign 密钥, explicit 绑定, and 本地优先 storage you operate. Choose messengers for mass reach; choose EnvoyMesh when trust boundaries and auditability matter more.

6.2 EnvoyMesh and cloud AI assistants

Cloud AI assistants run inference and 记忆 on vendor infrastructure with account login and vendor 策略. EnvoyMesh keeps models, 保险箱, and 绑定 on your 节点 while optionally calling remote providers you configure. You gain mesh reach and 授权 instead of a single-vendor chat history silo.

6.3 EnvoyMesh and standalone OpenClaw

Standalone OpenClaw excels as a local assistant but lacks native signed 对等节点 messaging, 绑定策略, and federated 知识 unless extended. EnvoyMesh bundles OpenClaw as EnvoyAI and wraps it with mesh 工具, 授权, and 审计. Running both without integration duplicates 智能体 unless you disable one.

6.4 EnvoyMesh and external agent runtimes

External 智能体 runtimes (HomeClaw, Hermes, custom HTTP) focus on 对话 and plugins; EnvoyMesh supplies 身份, transport, and 策略. The bridge pattern keeps libp2p 密钥 on the 节点 while the external process handles UX you prefer. Neither side replaces the other—they compose when configured deliberately.

6.5 EnvoyMesh and MCP

MCP standardizes 工具 discovery for AI applications; EnvoyMesh implements an MCP adapter that exposes selected mesh 能力. Native mesh intents remain richer and signed; MCP is an interoperability edge for desktop clients. Enable MCP 工具 narrowly to limit 保险箱 and 联系人 exposure.

6.6 EnvoyMesh and A2A

A2A defines 智能体 cards and 任务 interfaces for cross-product delegation; EnvoyMesh publishes cards and proxies 任务 through 中继 or home tunnel paths. Native 协作任务 and 授权 govern trust inside the mesh; A2A extends reach to external orchestrators. Both can coexist with different 策略 surfaces.

6.7 EnvoyMesh native Agent Network versus public marketplaces

Public 智能体 marketplaces optimize for discovery of anonymous 工作节点 and commercial ranking. EnvoyMesh 智能体网络 is the opposite: collaboration only among bonded 所有者 who opted in locally. There is no global listing, reputation score, or payment rail in the native design.

6.8 Native protocols versus interoperability bridges

Signed Envoy envelopes, 授权, and 绑定 tiers are the native protocol inside the mesh. MCP and A2A bridges translate at the edge for external ecosystems without replacing internal 安全 models. Prefer native flows for bonded 对等节点 work; use bridges when an external client must participate.

第 II 部分 —— 安装与入门

7. 选择你的部署方式

7.1 Desktop only

Run EnvoyMesh on a Mac or Windows computer as your primary home 节点. Install from the current release installer or build from source, create your 所有者身份 on first launch, and keep the machine running when you want mesh connectivity. This path fits anyone starting on one trusted desktop without mobile access yet.

7.2 Desktop with EnvoyGo mobile access

Add EnvoyGo on iOS or Android after your home 节点 is healthy. The phone pairs by scanning a QR code and mirrors chat, 联系人, 终端, and selected home features—it does not replace the desktop 节点 or hold 所有者 密钥 on its own. Plan for the home computer to stay reachable over LAN, 中继, or tunnel when you use mobile away from home.

7.3 Desktop with the bundled EnvoyAI agent

EnvoyAI (OpenClaw) ships with the desktop 节点 and starts on port 18789 by default. It can search your 保险箱, 消息 bonded 联系人, and run local 工具 under your 绑定 and 审批 settings. Toggle it in Settings → AI or set `openclawEnabled` in `node-config.json` if you prefer to start without the bundled assistant.

7.4 Desktop with an external agent

Connect HomeClaw, Hermes, OpenHuman, or a custom HTTP 智能体 through Settings → AI → Ext Agent. One 节点 runs one external bridge at a time; EnvoyMesh signs mesh traffic on the 智能体's behalf without handing over Ed25519 密钥. Enable the bridge only after you trust the external process and its local 端点.

7.5 Desktop with local or remote models

Configure model providers under Settings → AI according to your 隐私 and cost preferences. Local models keep inference on your hardware; remote providers send approved prompts outside the 节点 under your configured limits. Start with one provider, verify responses in chat, then widen automation once 审批 behave as you expect.

7.6 Personal relay or community relay

Relays help 对等节点 discover each other and traverse NAT; they do not hold your account or read application payloads. Use the community 中继 for casual testing, or run your own 中继 with `npm run node:dev -- --profile ./data/relay --relay-server --listen /ip4/0.0.0.0/tcp/4001`. Normal 节点 bootstrap with `--bootstrap "<relay-multiaddr>"` and `--relay`.

7.7 Small-team and organization deployments

Give each team member a home 节点 with its own 所有者身份, then 绑定 联系人 explicitly rather than sharing one login. Operators may deploy private 中继, standardize trust tiers, and disable 内置赞助 联系人 before fleet rollout. Document 资料 data paths so backups and upgrades stay consistent across machines.

7.8 Recommended first-time setup

Install the desktop app on a trusted computer, complete 所有者 and 设备 setup, enable EnvoyAI if you want a personal assistant, and back up 身份 material before adding 联系人. Pair one test 联系人 on the same LAN, send a 消息, then optionally add EnvoyGo. Defer 协作任务, external 智能体, and WAN 中继 testing until basic chat and status indicators look healthy.

8. 安装 EnvoyMesh

8.1 系统要求

Use a supported current macOS or Windows desktop environment with enough storage for the app, local data, and optional model or IPFS components. Source builds require the repository's Node.js toolchain and package dependencies; mobile access additionally requires a running home 节点.

8.2 在 macOS 上安装

Download the macOS disk image, open it, and move EnvoyMesh to Applications. On first launch, macOS may require confirmation because release signing and notarization can vary by build; retain your data directory when upgrading.

8.3 在 Windows 上安装

Run the Windows installer and allow the bundled 节点 runtime through local firewall prompts when you want 对等节点 connectivity. The Windows package intentionally carries a smaller essential OpenClaw extension set to control installer size.

8.4 在 iOS 上安装 EnvoyGo

Install EnvoyGo through the available iOS distribution channel, then pair it to an existing home 节点. EnvoyGo is a thin client: do not expect it to replace the desktop 节点 or preserve an independent mesh 身份 while the home 节点 is unavailable.

8.5 在 Android 上安装 EnvoyGo

Install EnvoyGo on Android and complete the same home-节点 pairing flow. Notification and background behavior depend on Android 权限, battery optimization, and FCM configuration.

8.6 从源码安装

From the repository root, install dependencies with `npm install`, run `npm run typecheck`, and run `npm test`. Start the 节点 with `npm run node:dev`; consult `QuickStart.md` for platform prerequisites and optional components.

8.7 校验安装

A healthy installation starts the 节点, opens the Social interface, shows 身份 and connection status, and can reach the local service. Verify with the built-in status surfaces before importing data or adding external integrations.

8.8 应用数据位置

Identity, trust, 审计, 任务, 保险箱, and configuration data live in the 节点's application-data location rather than in the installation directory. Use Appendix K and the current release notes to locate the platform-specific root.

8.9 更新 EnvoyMesh

Back up 身份 and 保险箱 data, stop active 任务, and install the newer package over the application. Review `CHANGELOG.md` for configuration or storage migrations before restarting.

8.10 卸载但不丢失身份或数据

Removing the application should be treated separately from deleting its data directory. Preserve the data root and 身份 backup if you intend to reinstall; delete them only when you deliberately want to erase the local 身份 and records.

9. 平台与打包差异

9.1 Desktop and mobile feature comparison

Desktop Social is the full home-节点 experience: mesh 身份, 保险箱, 智能体, 协作任务 orchestration, 浏览器, 终端, and settings. EnvoyGo mirrors a subset—chat, 联系人, 语音通话, read-only 协作任务 status, 终端, and 浏览器—through JSON-RPC to the paired home 节点. Treat mobile as a remote control, not a second independent 节点.

9.2 macOS packaging

macOS releases ship as a disk image with the Tauri-wrapped Social UI and embedded 节点 runtime. OpenClaw extensions are bundled more completely on macOS than on Windows to reduce post-install setup. Check release notes for notarization and Gatekeeper behavior on your macOS version.

9.3 Windows packaging

Windows releases use an installer that bundles the 节点 runtime and a slimmer OpenClaw extension set to control download size. Allow the app through Windows Firewall when prompted if you want inbound 对等节点 connections. Profile data lives under your user app-data path, separate from the install folder.

9.4 OpenClaw extensions bundled on macOS

macOS desktop builds include the fuller OpenClaw extension bundle used by EnvoyAI. Source installs copy extensions during `./scripts/setup.sh` or `npm run setup`. Rerun setup after upgrading OpenClaw-related dependencies if you develop from source.

9.5 Essential OpenClaw extension selection on Windows

Windows installers include a curated essential extension set rather than every optional channel. If a 能力 is missing, compare with the macOS bundle list in release notes or install from source with `.\scripts\setup.ps1`. Core mesh and chat features do not require extra extensions.

9.6 完整与精简桌面包

Some releases offer full installers with optional components and slimmer builds without IPFS or extra sidecars. Pick full when you want optional content features out of the box; pick slim on constrained disks or air-gapped lab machines. Your 身份 and 保险箱 data are the same regardless of bundle flavor.

9.7 可选 IPFS 侧车

IPFS-related components are optional adjuncts for content-addressing experiments, not required for chat, 绑定, or 协作任务. Enable them only when release notes document a supported sidecar for your platform. Omit them if you prefer a minimal attack surface.

9.8 Features requiring a home node

Mesh 身份, 智能体运行时, 保险箱 indexing, 协作任务 orchestration, MCP/A2A bridges, and full Settings live on the home 节点. EnvoyGo, 浏览器 dev UI pointed at a remote 资料, and CLI against `--profile` all assume that 节点 is running and reachable. Without a home 节点, mobile mirrors and thin clients cannot authenticate or send signed traffic.

9.9 Features available as an EnvoyGo mobile mirror

EnvoyGo exposes chat threads, 联系人, 语音通话, 终端 attach, 浏览器 for `envoy://` content, push 通知, and read-only recent 协作任务 status under Me → 智能体网络. AI engine toggles and bridge configuration appear read-only on mobile; change them on the home 节点. Cached data on the phone is for convenience, not authoritative 身份 storage.

9.10 Legacy mobile experiments and current product boundaries

The Capacitor app in `apps/mobile` was an in-process full-节点 experiment and is not the product mobile path. EnvoyGo is the supported thin client paired to home. Running EnvoyGo as a standalone full mesh 节点 remains parked; use desktop or source builds for a primary 节点.

10. 创建你的身份

10.1 What your EnvoyMesh identity represents

Your 身份 is cryptographic, not a cloud username. An 所有者身份 controls 授权 and 设备; each 设备 has its own 密钥; your 智能体身份 acts on the mesh under an 所有者-signed 授权. Peers verify signatures against these IDs rather than trusting a central directory.

10.2 创建所有者身份

On first launch, Social walks you through generating an 所有者 keypair stored in your 资料 directory (for example `./data/default` in source runs). This step happens once per person; subsequent installs on new machines import or authorize additional 设备 instead of creating a second 所有者. Back up the 所有者

material before bonding production 联系人.

10.3 创建你的第一个设备身份

The first desktop install creates a 设备身份 authorized by your 所有者 密钥 automatically. The 设备 signs routine envelopes and holds local 会话 state. Note the 设备 ID in Profile or via `npm run cli -w @envoymesh/node -- profile --profile ./data/default` when diagnosing pairing.

10.4 创建或激活你的智能体身份

EnvoyMesh derives an 智能体 对等身份 from your 所有者 and 智能体 密钥, then records an 所有者-signed 授权 linking the 智能体 to you. EnvoyAI uses this 身份 when sending 智能体-role 消息. External bridge 智能体 receive a separate bridge 身份 persisted as `bridge-identity.json` when enabled.

10.5 设置你的展示资料

Open Profile in Social to set the name, avatar, and fields other 联系人 see after bonding. Profile data is signed and stored locally in your 资料 directory. Update it before sharing pairing codes so recipients recognize you.

10.6 了解你的 DID

Your 所有者 DID follows the form `envoy:owner:<hash>` derived from your public 密钥. Device and 智能体 IDs use parallel `envoy:device:` and `envoy:agent:` prefixes. Share 所有者 IDs for stable addressing once 对等节点 have exchanged trust; runtime 对等节点 IDs can rotate with 密钥 while 所有者 IDs stay long-lived.

10.7 保护你的加密密钥

Private 密钥 live in the 资料 data directory with restrictive 文件 权限. Do not copy 密钥 文件 to chat, email, or shared drives unencrypted. Use the OS user account protection on your home 节点 machine as the first layer of defense.

10.8 备份身份与恢复数据

Copy the entire 资料 directory—or export backups your release documents—before OS reinstall or hardware migration. 保险箱 content under `shared_vault/` or your configured 保险箱 path should be backed up separately from the application binary. Test restore on a non-production machine before you need it urgently.

10.9 添加另一台设备

Pair a second 设备 by scanning a QR code or approving a pairing request from the home 节点's Pairing Queue. The 所有者 signs a 设备证书 authorizing the new 设备 while sharing the same 所有者 ID. EnvoyGo pairing follows the thin-client flow: the phone receives a 会话 to the home 节点 rather than duplicating 所有者 密钥 on the phone.

10.10 吊销丢失或被入侵的设备

From a trusted remaining 设备, revoke the lost 设备证书 and remove its trust entries. Change any bridge secrets if the 外部智能体 ran on the compromised machine. Treat 所有者 密钥 compromise as catastrophic: revoke 设备, rotate bridge credentials, and rebond 联系人 only after you are confident 密钥 are clean.

11. 应用导览

11.1 主页与节点状态

The home view summarizes 节点 connectivity, discovery mode, and recent activity. Use it to confirm the 节点 is listening, 中继 are reachable, and no startup warnings remain. CLI equivalents include `connectivity-status` and `relay-status` for deeper diagnosis.

11.2 对话

Conversations lists direct and 群聊 threads with delivery indicators. Open a thread to send text, audio, 文件, or 智能体 消息 depending on trust and settings. Search and pin behavior follow the current Social release; unread state syncs from your local 资料 store.

11.3 联系人与发现

Contacts shows bonded 对等节点 with 信任层级 badges; discovery surfaces 能力 or tag-based lookups where 策略 allows. Strangers remain heavily rate-limited until you accept a 绑定请求. Block or downgrade trust from the 联系人 detail sheet if a relationship changes.

11.4 Groups

Create a 群组 from Conversations, add bonded 联系人, and set a title and avatar. Group 消息 use the same signed envelope path as direct chat with 群组 routing metadata. Only add participants you trust at the 敏感度等级 you plan to share in the 群组.

11.5 知识库与资料库

资料库 is the in-app 知识库: create Markdown notes, import documents, and toggle per-item 敏感度. The 策略 engine honors four ranks — `public`, `friends`, `trusted`, `private` — while the UI exposes friendlier labels for the ones you pick most often. Saved notes index into RAG automatically. Optional Obsidian and MCP plugins are configured under Settings → AI → 知识库.

11.6 Browser

浏览器 loads permitted `envoy://` mesh content through your 节点's 策略 boundary. You see what 绑定 rules and 敏感度 labels allow—not the open web by default. Use it to read published notes and mesh pages from bonded or public authors.

11.7 协作任务

协作任务 appear where 智能体网络 is enabled. Your 智能体 orchestrates work across opted-in bonded 智能体; you review plans, budgets, and results in the 协作任务 UI. Start with small objectives before enabling automatic cost rebalance 策略.

11.8 Terminals

终端 attach to shell 会话 on the home 节点 via WebSocket, including from chat inline or the dedicated 终端 view. Sessions require authentication through the 节点 and respect your 审批 settings for 智能体 command execution. Remote attach from EnvoyGo tunnels through the home JSON-RPC transport.

11.9 Approvals and activity

Approvals queues sensitive 智能体 or 任务 actions awaiting your decision; Activity (审计) shows allow/deny outcomes with correlation IDs. Approve or reject from Social or CLI (`npm run cli -w @envoymesh/node -- approvals ...`). Use correlation IDs to stitch multi-step 协作任务 or 中继-assisted flows.

11.10 Profile

Profile edits your human-visible 身份 and shows 所有者, 设备, and 智能体 identifiers. It is the right place to copy pairing information and verify which 设备 you are on. Changes propagate to 联系人 on the next signed 资料 update they receive.

11.11 Settings

Settings controls discovery 资料, AI engines, 外部智能体 bridges, 知识 plugins, 通知, and 节点 behavior flags. Changes write to `node-config.json`, `bridge-config.json`, and related 文件 in your 资料 directory. Restart or follow in-app prompts when a setting requires a 节点 reload.

11.12 连接与智能体状态指示

Header badges show WebSocket/Social connectivity, mesh reachability, EnvoyAI gateway health, and external bridge state when configured. Yellow or red states mean you should fix connectivity before sending sensitive data. EnvoyGo shows a parallel connection indicator for home reachability.

12. 连接你的第一个联系人

12.1 What pairing and bonding do

Pairing exchanges enough information to identify and reach another 所有者; bonding records the trust relationship and 策略 tier. A packaged desktop build may also add the project sponsor 联系人 from `bundled-sponsor-friend.json` on first launch; operators can disable that bundle before deployment.

12.2 Pair with a QR code

Open Add Contact on one 设备 and Show My Code on the other, then scan with the built-in scanner in Social or EnvoyGo. Confirm the displayed 所有者 ID and display name match what you expect in person. Complete the 绑定请求 flow before treating the 联系人 as trusted.

12.3 Pair with an invitation link

Generate an invitation link or multiaddr payload from Contacts and share it over a channel you trust (Signal, in-person AirDrop, etc.). The recipient opens the link in Social to initiate pairing. Treat leaked links like leaked phone numbers—revoke or ignore unexpected 绑定 requests.

12.4 Pair on a local network

On the same LAN, mDNS discovery may list nearby 节点 without manual multiaddrs. Start both 节点 with default discovery or `--listen /ip4/0.0.0.0/tcp/0`, then pick the 对等节点 from the discovery UI. LAN pairing is the fastest way to validate signing and chat before testing 中继 paths.

12.5 Verify identity information

Before accepting a 绑定, compare 所有者 ID, display name, and optional proof text out of band. Signed envelopes prove possession of 密钥, not that you know the person—your proof step closes that gap. Reject requests that do not match what your 联系人 said they would send.

12.6 Choose an appropriate trust level

EnvoyMesh trust tiers are blocked, public (stranger), referred, and direct (friend). Start new acquaintances at public or referred unless you already have a strong trust basis. Direct unlocks richer 知识 sharing and 智能体 collaboration; upgrade only deliberately.

12.7 Accept a bond request

Incoming 绑定 requests appear in Contacts or 通知 with the sender's proof 消息. Accept to record mutual trust locally; reject leaves them at stranger tier. Either side can later change tier or block from 联系人 settings.

12.8 Send the first message

Open the new 联系人 thread and send a short signed chat 消息. Watch for delivered or read indicators according to your release. If the 消息 stalls, check connectivity status before resending duplicates.

12.9 Confirm direct or relay-assisted delivery

Successful delivery shows positive acknowledgment in-thread or an 审计 `chat.message` allow row. Relay-assisted paths use `/p2p-circuit` addresses learned from `relay.lookup`; direct LAN paths skip 中继 hops. CLI 审计 with `--include-p2p-trace` helps confirm which path was used during testing.

12.10 Troubleshoot pairing

Verify both 节点 run, firewalls allow outbound TCP, and 资料 paths match between UI and CLI. For WAN tests, confirm bootstrap 中继 multiaddrs and run `connectivity-status`. Retry with freshly copied listening multiaddrs after restarts because dynamic ports change.

12.11 内置赞助联系人

A packaged desktop build (DMG / `.exe` / `.AppImage`) auto-绑定 to the project's sponsor 联系人 on first launch using the bundled `bundled-sponsor-friend.json`, so you start with one working 联系人 out of the box. This is a convenience, not telemetry: no data leaves your 节点, and the 绑定 is a normal local trust record you can edit or remove like any other 联系人. Operators preparing fleet images can disable the auto-绑定 by setting `{"enabled": false}` in the bundled 文件 before packaging.

13. 连接 EnvoyGo

13.1 How EnvoyGo works with a home node

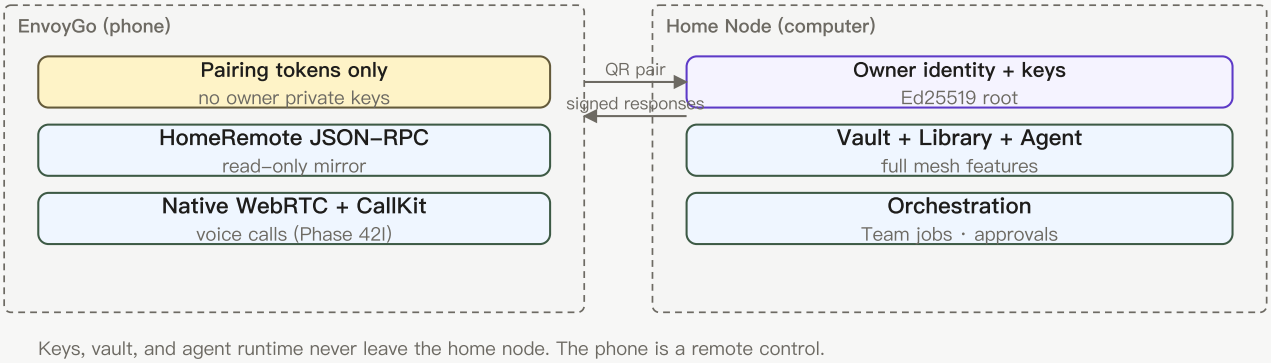


图 12 —— EnvoyGo 瘦客户端配对：手机仅持有配对令牌，通过 JSON-RPC 调用家庭节点。身份、保险箱、智能体与编排保留在家庭节点。

EnvoyGo connects to a paired home 节点 and presents selected NodeService functions through a mobile interface. The home 节点 keeps mesh 身份, 智能体运行时, 保险箱, and orchestration responsibility.

13.2 Pair the mobile app

Install EnvoyGo, tap Pair with Home, and scan the QR code shown in Social on the desktop 节点 (or enter the pairing payload your release documents). Approve the 设备 on the home 节点 if prompted in Pairing Queue. The app stores pairing tokens in secure storage, not 所有者 private 密钥.

13.3 Confirm the home connection

After pairing, the connection indicator should show home reachable and load your chat list. Pull to refresh or open Me → Node status if threads stay empty. Ensure the desktop 节点 stays running and reachable on the 网络 path you expect (LAN, 中继 tunnel, or configured remote URL).

13.4 Use chat and contacts

Chats and People tabs mirror home-节点 threads and bonded 联系人 with mobile layouts. Sending a 消息 routes through HomeRemote JSON-RPC to the home 节点, which signs and delivers on the mesh. Media and audio 消息 follow the same path.

13.5 Use remote terminals

From 终端, attach to an existing 会话 or start one allowed by home 策略. Input travels over the tunneled 终端 protocol; output streams back with scrollback. Avoid sensitive commands on untrusted 网络 until you confirm transport encryption and home reachability.

13.6 View 协作任务

Me → 智能体网络 shows read-only recent 协作任务 activity synced from the home 节点. You can inspect status and reports but cannot orchestrate new jobs from mobile alone—start jobs from desktop chat with your 智能体. The UI says 协作任务 even when logs use older internal terminology.

13.7 Browse mesh content

The EnvoyGo 浏览器 (Phase 45C) opens `envoy://` content through the paired home service. Availability depends on the home 节点 being reachable and on the requested author or content being permitted by 绑定策略.

13.8 Receive notifications

EnvoyGo can receive normal and 通话-related 通知 when APNs or FCM is configured. iOS backgrounded calling uses VoIP push + CallKit (Phase 42I) and the operating system grants 权限. Delivery remains best effort and is affected by platform background restrictions.

13.9 拨打与接听语音通话

Available mobile 通话 support covers one-to-one 语音通话 with native WebRTC and platform 通话 integration. iOS ships VoIP push + CallKit (Phase 42I, shipped 2026-06-19) so backgrounded phones can receive 通话; real-设备 validation is still open. Video calling is not yet available (see §18.10 and Appendix J.4). TURN may be required for cross-网络 audio when both 对等节点 sit behind restrictive NAT.

13.10 Revoke a lost phone

From the home 节点, revoke the EnvoyGo 设备 or 会话 pairing and rotate any exposed tokens. Remove the 节点 entry in EnvoyGo if you recover the phone later and need a clean re-pair. Treat a lost unlocked phone like a lost 会话 to your home API.

13.11 Current mobile limitations

EnvoyGo does not run a full mesh 节点, orchestrate 协作任务, edit all Settings, or replace home-节点 保险箱 authoring. Video 通话, full 浏览器 parity, and background reliability vary by OS 权限. See release notes for the exact feature matrix on your build.

14. 首日教程

14.1 Send a private message

Bond a 联系人 (Chapter 12), open their thread, type a short 消息, and send. Confirm the delivery indicator updates. If it fails, open Home status and verify mesh connectivity before retrying once.

14.2 创建群组 conversation

From Conversations, choose New Group, select bonded 联系人, name the 群组, and send a hello 消息. Each member receives 群组 envelopes signed by your 节点. Adjust membership later from 群组 settings if your release exposes it.

14.3 Send an audio message

In chat, tap the microphone control, record a brief clip, and send. The audio rides inside a signed chat envelope and plays inline for recipients. Grant microphone 权限 when the OS prompts on desktop or EnvoyGo.

14.4 发起语音通话

With a direct-trust 联系人, start a 语音通话 from the thread header. Accept the incoming 通话 on their 设备; media flows 点对点 after mesh signaling. If connection fails behind strict NAT, configure TURN as documented for your release.

14.5 共享文件

Use the attachment control in chat or share from 资料库/保险箱 according to 敏感度 rules. Files transfer as data intents with 策略 checks on path and 信任层级. Confirm the recipient sees the attachment and 审计 logs an allow outcome.

14.6 Ask EnvoyAI a question

Open your 智能体 thread or main assistant entry point and ask a factual question answerable from your 保险箱 or public 知识. EnvoyAI runs locally on the 节点 gateway unless you routed engines differently. Deny or refine if the 智能体 requests 审批 for a sensitive 工具 通话.

14.7 添加知识 to your Library

Open 资料库 → New Note, write Markdown, set 敏感度, and save. Indexing runs automatically for RAG. Optionally open the 保险箱 folder in Obsidian if you enabled the plugin and want external editing.

14.8 Search your Vault

Use 资料库 search or ask EnvoyAI to search local 知识 with explicit scope. CLI users can run `npm run cli -w @envoymesh/node -- vault-search --vault ./shared_vault --query "your terms"`. Results respect 敏感度 labels and your role on the 节点.

14.9 Ask a bonded agent for knowledge

Message a 联系人's 智能体 or send a 知识 query where the UI supports it, staying within their 信任层级. Public-tier queries are rate-limited for strangers; direct 绑定 allow richer scope. Expect signed responses attributable to their 智能体身份.

14.10 Approve a sensitive action

When an 智能体 or 任务 触发器 策略, an 审批 card appears in Approvals. Read the summary, correlation ID, and requested action before allowing. Reject if the scope exceeds what you intended for that 会话.

14.11 Start a simple Team job

In chat with your 智能体, describe a small multi-step objective that can delegate to a bonded 对等节点's 智能体 (for example summarize then translate). Confirm 智能体网络 membership is on for both sides. Review the plan, budget cap, and final 协作任务 report before sharing externally.

14.12 连接外部智能体

In Settings → AI → Ext Agent, pick HomeClaw, Hermes, or Custom and point to the local HTTP 端点 (`http://127.0.0.1:8010/message` for HomeClaw by default). Start the external process, enable the bridge, and send a test chat 消息 to the bridge 智能体 对等节点. Verify callbacks arrive on the configured listen port before enabling automation.

第 III 部分 —— 联系人、资料与对话

15. 联系人与绑定

15.1 查看与搜索联系人

Open **People** in Social or the Contacts tab in EnvoyGo to browse bonded 所有者 and pending introductions. Search by display name or 所有者 ID fragment; results respect your local trust store, so blocked 联系人 stay hidden unless you explicitly show them. EnvoyGo lists the same 联系人 through HomeRemote JSON-RPC—it does not maintain a separate 联系人 database on the phone.

15.2 了解联系人身份

Each 联系人 maps to an **所有者身份** (`envoy:owner:…`) backed by Ed25519 密钥, not a central account handle. Runtime 消息 use 对等节点 IDs derived from 密钥; compare 所有者 ID and any out-of-band proof before upgrading trust. QR pairing (Chapter 13) adds **设备** 身份 under the same 所有者—it does not replace 所有者-to-所有者 绑定.

15.3 联系人资料与照片

Profile cards show display name, description, and 照片 the 联系人 publishes within 绑定策略. Photos arrive as signed 资料 or 文件 payloads; referred and public tiers may see fewer fields than direct friends. Tap a 照片 to view full size; do not treat gallery thumbnails as verified 身份 proof by themselves.

15.4 Online, offline, and connection states

Presence reflects mesh reachability, not a cloud "online" flag. A 联系人 may show offline while 消息 queue for 中继-assisted delivery when they return. EnvoyGo shows home connectivity separately from remote 对等节点 reachability—your phone can be online to home even when the 联系人 is not.

15.5 直接, referred, public, and blocked trust

EnvoyMesh uses four user-selectable tiers for 联系人 — **blocked** (deny all), **public** (stranger—ping and narrow discovery only), **referred** (introduced—limited 知识 and 审批), and **direct** (friend—richer chat, 文件, and 智能体 workflows up to friends 敏感度). Tier is stored locally on your 节点; both sides can set different tiers toward each other.

15.6 更改联系人信任层级

Open the 联系人 in Social → **Trust** (or equivalent Settings) and pick blocked, public, referred, or direct. Downgrading takes effect immediately for new operations; already-delivered content remains in local history until you delete it. Document why you changed tier—审计 rows help if you later review an incident.

15.7 推荐或介绍联系人

Use **Introduce** or 绑定-request flows to vouch for someone at referred tier without granting direct trust yourself. Introductions carry signed proof text so the recipient can verify out of band. Referred 联系人 cannot recruit your 智能体 into 协作任务 until you deliberately upgrade them.

15.8 静音、阻止或移除联系人

Mute suppresses 通知 locally without changing 绑定 tier. **Block** sets blocked trust and stops new inbound intents. **Remove** clears local thread metadata but does not erase their 密钥 from the 网络—re-add only after you are comfortable with renewed 联系人.

15.9 恢复连接

To reconnect after block or accidental removal, exchange a fresh 绑定请求 or introduction with updated proof text. If you revoked their tier, they must accept a new request; stale threads may not resume automatically. Verify 身份 again before restoring direct trust or sharing 文件.

15.10 联系人隐私与披露设置

Profile and 联系人 settings control what you publish and what you request from others: display fields, 照片 visibility, and 敏感度 labels on shared 知识. Defaults lean conservative for public-tier viewers; direct 联系人 see richer 资料 slices. Changes propagate on the next signed 资料 update, not retroactively to old screenshots.

16. 私信

16.1 发起对话

From **People**, open a direct 联系人 or pick an existing thread under **Chat**. New 对话 require at least public-tier reachability and a successful 绑定 or introduction path. Group rooms use separate creation flows (Chapter 17); do not assume a DM thread exists for every 联系人 until you send the first 消息.

16.2 人对人消息

Private chat uses the `chat.message` intent with **human** sender and **human** recipient roles—智能体 cannot impersonate this path. Messages are signed envelopes delivered over libp2p direct or 中继-assisted paths. Compose in Social or EnvoyGo; the home 节点 signs and sends on your behalf when using mobile.

16.3 人对智能体消息

Talking to **@envoy** or your configured 智能体 name routes through 智能体-capable chat flows, not `chat.message` 人对人 semantics. Agent replies may invoke 工具 under 授权 and 绑定策略. Keep 所有者-facing instructions separate from 对等节点 DMs so you do not accidentally share private context with a 联系人 thread.

16.4 回复与对话连续性

Replies reference prior 消息 through thread metadata and correlation IDs in 审计 logs. Quote or reply in-thread to preserve context; resending the same text creates duplicate envelopes. Search (16.7) helps locate earlier turns when a long DM splits across 会话.

16.5 消息投递状态

Delivery indicators reflect local send acknowledgment and remote acceptance when your build exposes them—not read receipts unless explicitly supported. Failed sends show 策略 or connectivity errors; read 审计 for `chat.message` deny vs transport timeout. Avoid rapid duplicate sends while a 消息 is still pending.

16.6 离线行为与重试

When a 联系人 is offline, the home 节点 queues signed 消息 where protocol and 策略 allow and retries over direct or 中继 paths on reconnect. Large backlogs may arrive out of strict UI order but remain integrity-checked by signature. EnvoyGo offline to **home** prevents any send until the tunnel restores.

16.7 搜索对话历史

Use in-app search or 保险箱-adjacent 对话 indexes where enabled to find text by keyword or 联系人. Results come from locally stored copies on the home 节点; mobile search queries home over JSON-RPC. Sensitive threads remain visible only on 设备 paired to that 节点.

16.8 草稿辅助

Draft assistance (when enabled) suggests completions through your configured model with semantic-firewall limits—it does not auto-send. Review suggested text before sending; 智能体-assisted drafts in 联系人 threads still obey 绑定 tier and 敏感度. Disable assistance in Settings if you prefer manual composition only.

16.9 管理对话数据

Export, archive, or delete 对话 data from thread menus or 资料 maintenance 工具 on the home 节点. Deletion is local to your store unless a product feature explicitly requests remote retraction—which is not guaranteed for already-delivered 对等节点 copies. Back up before bulk purge (Chapter 89).

16.10 消息隐私与安全

Messages inherit transport encryption from libp2p where negotiated; authorization still depends on signatures and 绑定策略, not TLS alone. Do not paste secrets into chats with referred or public 联系人. Report abuse via block tier and preserve 审计 correlation IDs if you escalate.

17. 群组对话

17.1 创建群组

In Social, choose **New 群组** (or Rooms) and name the room. Initial members must be 联系人 you can reach under current trust—typically direct or referred depending on 策略. The creating 节点 stores membership locally; new members receive signed invites through mesh delivery.

17.2 邀请成员

Add members from your bonded 联系人 list; you cannot invite blocked 所有者 or strangers without an introduction path. Each invite is a signed membership intent; pending members appear until they accept. Large 群组 increase fan-out latency—prefer focused rooms for time-sensitive coordination.

17.3 发送群组消息

Group 消息 use room-scoped chat intents with human senders; delivery fans out to online members and queues for offline ones where supported. @mentions and replies follow the same threading rules as DMs within the room context. EnvoyGo 群聊 mirrors home threads once paired.

17.4 管理成员

Owners with admin rights (per your build) can add or remove members and rename the room. Removing someone stops new deliveries to them but does not erase history on their 节点. Rotate admins deliberately—被入侵的管理员 设备 can invite unwanted members.

17.5 退出群组

Choose **Leave 群组** to stop receiving new 消息; your past copies remain on your 节点 until you delete them. Other members continue the room. Rejoin requires a fresh invite if membership is not automatically restored.

17.6 群组信任边界

Group visibility does not bypass per-member trust: a referred member still cannot access direct-only 文件 shares you send outside the room. Sensitive attachments should use explicit 敏感度 labels. Do not treat 群组 membership as mutual direct friendship with every participant.

17.7 群组投递与离线成员

Offline members receive queued room 消息 on reconnect; ordering may batch during catch-up. If many members are behind 中继-only paths, expect delayed delivery indicators. Check home connectivity before assuming the room is broken.

17.8 群组故障排查

If 消息 stall, verify each member's 绑定 tier, home reachability, and 中继 reservation. Audit rows tagged with the room correlation ID show deny vs timeout. Split troubleshooting: 策略 denials need trust changes; transport failures need connectivity work (Chapter 91).

18. 语音消息与通话

18.1 录制并发送语音消息

Hold the microphone control in a DM or 群组 thread to record a short audio clip; release to attach and send. Audio rides the same signed 文件/消息 path as other attachments with size caps enforced by inbound guard. Prefer text for referred 联系人 unless they expect voice notes.

18.2 播放与管理语音附件

Tap an audio bubble to play; long-press for save or delete locally where supported. Playback decodes on 设备; very long clips may be rejected at send time. Manage storage under 对话 settings if attachments accumulate.

18.3 发起语音通话

Start a **语音通话** from the 通话 button in a bonded direct thread on Social or EnvoyGo. Calls negotiate WebRTC audio between 对等节点 with home-节点 signaling; video is not available in current builds. Both sides need microphone 权限 and reachable mesh or 中继 paths.

18.4 接听或拒绝通话

Incoming 通话 surface as in-app banners and, on EnvoyGo, platform 通话 UI when configured. Decline sends a signed reject; answer establishes the WebRTC 会话. Unknown or blocked 联系人 should not reach 通话 UI if 策略 is working—verify 信任层级 if 通话 appear unexpectedly.

18.5 通话状态与控制

In-通话 controls include mute, speaker routing, and hang up; status shows connecting, active, or failed phases. Dropped 通话 may retry manually—there is no hidden auto-redial. Note correlation IDs in 审计 if you report persistent failure.

18.6 后台通话与移动通知

EnvoyGo can receive 通话 通知 via APNs/FCM when push is configured; background behavior depends on OS 策略. Keep the app paired to home and allow 通知 权限 for reliable ringing. Desktop Social may use local 通知 without mobile push.

18.7 STUN 与 TURN 连接

WebRTC tries direct UDP first, then STUN, then configured TURN when both 对等节点 sit behind symmetric NAT. Configure TURN in Settings if 通话 connect but have no audio. Relay libp2p paths carry signaling—not a substitute for TURN media 中继.

18.8 Call privacy

Voice 通话 require at least direct or referred trust per product 策略; blocked 联系人 cannot initiate 通话. Call metadata appears in 审计; media stays 点对点 when WebRTC succeeds. Do not share screen or video—视频通话 remain planned (18.10).

18.9 语音通话故障排查

If 通话 fail to connect, check microphone 权限, TURN settings, 绑定 tier, and `connectivity-status`. One-way audio often means NAT or firewall blocking UDP. Test LAN direct path first, then 中继-assisted WAN before opening broad firewall rules.

18.10 Video calls —— 计划中，当前不可用

Planned. One-to-one audio calling is available today (§18.3); video calling is architecturally anticipated but not shipped in the current release. See Appendix J.4 for the roadmap boundary.

19. 文件、照片与资料共享

19.1 共享文件

Use the attachment or **Share 文件** action in a DM or 群组 allowed by 信任层级. Files chunk and transfer with integrity checks; direct friends typically have the broadest limits. Name 文件 clearly—recipients see filenames before accepting.

19.2 接受或拒绝收到的共享

Incoming shares prompt accept or decline before writing to 保险箱 or Downloads per 敏感度. Declined transfers do not partial-write; accepted 文件 land in 策略-scoped storage. On mobile, acceptance may require home online to complete.

19.3 查看传输进度

Progress bars reflect bytes acknowledged on the transfer voucher path; stalled progress usually means connectivity loss mid-stream. Wait for retry or cancel and resend smaller 文件. Audit may log partial transfers without storing incomplete secrets in the log body.

19.4 校验文件完整性

Compare displayed hash or size metadata when your build exposes them; signatures prove sender 身份, not that the 文件 is benign. Scan unfamiliar binaries locally before opening. Re-send if hash mismatch reports after completion.

19.5 共享资料照片

Share 资料 照片 through Profile → Gallery → publish or send to a 联系人. Published 照片 obey visibility tier; direct shares attach to a thread like other media. EnvoyGo displays 照片 fetched via home—editing gallery is primarily a desktop Social flow.

19.6 管理你的资料相册

Maintain ordered gallery slots on the home 节点; reorder or remove images before they propagate in the next 资料 sync. Removing a gallery image stops future fetches but not copies already saved by 联系人. Keep at least one neutral avatar for referred viewers if you use public discovery.

19.7 选择可见性与敏感度

Tag shares with 敏感度 matching 保险箱 conventions (`public` / `friends` / `trusted` / `private`). The UI exposes friendlier labels for the most common choices; the 策略 engine honors all four ranks. Down-tier 联系人 cannot escalate 敏感度 at receipt—the 绑定 engine denies incompatible requests. Default to friends or private for documents with personal data.

19.8 移除共享内容

Delete local copies from thread attachments or 保险箱 paths; remote 对等节点 may retain their accepted copies unless a retraction feature exists in your build. Profile 照片 removal updates your signed 资料 on next publish. For incidents, block the 联系人 and revoke trust (Chapter 87).

19.9 排查文件传输

For stuck transfers, verify 信任层级, 文件 size limits, disk space on home 保险箱, and 中继 reachability. Retry on a stable 网络 with a smaller test 文件 to isolate 策略 vs transport. Collect 审计 correlation IDs before sharing diagnostics (Chapter 91).

20. 资料与在线状态

20.1 编辑你的人类资料

Edit **Profile → Human** in Social to set display name, bio, and published fields. Changes serialize into signed human 资料 payloads stored on the home 节点. EnvoyGo shows the result read-only unless your release adds mobile editing.

20.2 编辑你的智能体资料

Agent 资料 describe 能力 exposed to 对等节点 (工具, 协作任务 roles, A2A card fields). Edit under Profile → Agent or 智能体网络 settings; 所有者 授权 bounds what the 智能体 may advertise. Misleading 能力 text does not grant extra 权限—绑定策略 still gates actions.

20.3 显示名称与描述

Display names are cosmetic; authorization uses 所有者 and 对等节点 IDs. Keep descriptions concise—public-tier viewers may see shortened fields. Avoid embedding secrets or recovery codes in public bio text.

20.4 资料照片与相册

Human and 智能体 资料 can each carry 照片 galleries with tier-aware visibility. Upload on desktop Social; sync propagates to 联系人 on 资料 fetch. Large images may be downscaled to respect size limits.

20.5 身份详情与 DID

The 资料 details pane shows 所有者 DID, 设备 IDs where relevant, and fingerprint-style hashes for verification. Share these out of band when confirming 身份—do not trust unsolicited IDs in chat alone. QR pairing encodes 设备 pairing payloads, not 所有者 DID substitution.

20.6 已绑定联系人能看到什么

Direct 联系人 see the richest 资料 slice your 策略 publishes; referred 联系人 see reduced fields; public strangers see only public-敏感度 资料 data if exposed. Blocked 联系人 see nothing new from you. Review **Profile visibility** settings before enabling discovery features.

20.7 资料同步

Profile updates push on signed publish events; 联系人 refresh on next fetch or thread open. There is no global cloud 资料 CDN—对等节点 learn changes when they communicate with your 节点. After 密钥 rotation, republish 资料 so fingerprints match.

20.8 隐私默认值

Initial 隐私 defaults favor minimal public exposure: conservative 照片 visibility, friends-level chat history on home, and 智能体 工具 disabled until mandated. Review defaults after install before joining discovery topics. Reset paths are in Settings → Privacy where available.

第 IV 部分 —— 你的个人 AI

21. 认识 EnvoyAI

21.1 EnvoyAI 是什么

EnvoyAI is your 所有者-facing assistant on the home 节点, powered by the bundled OpenClaw runtime. You talk to it from Social, EnvoyGo, or `@envoy` in chat; it plans replies and 通话 mesh 工具 through EnvoyMesh 策略 rather than getting raw libp2p access. Think of it as the brain that stays inside the 安全 boundary while the 节点 handles 身份, 绑定, and 审计.

21.2 OpenClaw 作为内置智能体运行时

OpenClaw runs as a child process the 节点 starts and supervises. Its gateway listens on port `18789` by default (`http://127.0.0.1:18789/webhook/envoymesh`). EnvoyMesh passes each Assistant turn 会话 context—绑定, interests, and the 工具 catalog—and OpenClaw owns multi-turn reasoning and persistent 记忆 across 会话.

21.3 EnvoyAI 与外部智能体桥接的区别

EnvoyAI is in-process with full ToolRegistry access. The 外部智能体桥接 (default port `3031`) is an optional HTTP pipe to HomeClaw, Hermes, OpenHuman, or a custom 智能体 in another process. You can run both engines (`both` mode) or either alone; the bridge 智能体 never receives your libp2p 密钥.

21.4 EnvoyAI 可访问什么

EnvoyAI reads your local 保险箱 and 资料库 within 敏感度 labels, queries bonded 对等节点 through `knowledge.query`, and uses chat RAG when 知识库 settings allow. It cannot bypass 绑定 tiers: strangers stay rate-limited, and private material requires direct trust or 所有者 审批. Configure ceilings under Settings → AI → 知识库 and per-联系人 preferences before enabling auto-replies.

21.5 EnvoyAI 可用的 mesh 工具

At startup the 节点 exports a 工具 catalog to OpenClaw—chat send, 资料库 read/discover, 任务 propose, discovery, 审批, 触发器, MCP proxy, and more. Each 工具 declares a 敏感度上限 and whether it needs 所有者 审批 before execution. EnvoyAI chooses 工具 by name; EnvoyMesh enforces 策略 and writes an 审计 row for every 通话.

21.6 策略与审批控制

绑定引擎 decisions, 授权 limits, and the 审批队列 sit between EnvoyAI and the mesh. Outbound chat, 文件 shares, cloud model 通话, and high-敏感度 保险箱 reads queue for your review unless an autonomous 策略 explicitly allows them. Flip `autonomousKillSwitch` in Settings to pause all autonomous actions and force 审批 on everything the 智能体 would have done silently.

21.7 启动、停止与检查智能体

Open Settings → AI → AI Engine to see OpenClaw status: enabled flag, running state, PID, and last error if the gateway failed. Use **Restart OpenClaw** for a clean child-process recycle without restarting the whole 节点. Toggling `openclawEnabled` off stops the gateway immediately and prevents spawn on the next 节点 start—useful when debugging port conflicts on `18789`.

21.8 Current limitations

Chat drafts and lightweight auto-replies still route through EnvoyMesh's native model router for speed; complex Assistant turns go to OpenClaw with fallback to native when the gateway is down. Full chat-history injection into OpenClaw context and multi-round 工具 loops within one turn remain partial—会话 记忆 works, but recent thread text may not always be attached. 终端 Agent mode uses the native model directly, not OpenClaw exec.

22. AI 引擎模式

22.1 仅内置

Built-in only (`openclaw-only`) is the default on fresh installs: `openclawEnabled` is on and `bridgeEnabled` is off. EnvoyAI handles Assistant chat, 工具 execution, and 会话 记忆; no external HTTP 智能体 listens on `3031`. Choose this when you want one bundled runtime and no second 智能体 process.

22.2 内置加外部智能体

Built-in plus external (`both`) runs EnvoyAI and the bridge together. Mesh traffic from bonded 联系人 can reach the bridge 智能体 while you still use OpenClaw for `@envoy` and Settings → AI workflows. Enable `bridgeEnabled`, pick an active 外部智能体 in `bridge-config.json`, and confirm both status chips in the header before relying on either path.

22.3 仅外部智能体

External 智能体 only (`ext-only`) disables the OpenClaw gateway (`openclawEnabled: false`) but keeps the bridge active. All bridged chat and mesh 工具 通话 go through your 外部智能体's HTTP 端点; EnvoyAI Assistant turns are unavailable. Use this when HomeClaw or Hermes is your primary brain and you only need EnvoyMesh for connectivity and 策略.

22.4 No AI

No AI (`off`) turns off both engines. The 节点 still routes human chat and 策略, but no model drafts, auto-replies, or 智能体 工具 run. Select this for air-gapped 节点, CI fixtures, or when you need mesh connectivity without any LLM surface.

22.5 选择合适的模式

Start with **built-in only** for the simplest path. Add **external** when you already run HomeClaw/Hermes and want its plugins or 记忆 model. Use **both** only when you deliberately want two 智能体—otherwise pick one brain to avoid duplicate replies. Switch to **off** temporarily rather than uninstalling when testing connectivity alone.

22.6 切换当前外部智能体

External 智能体 are defined in `bridge-config.json` under `extAgents`; set `activeExtAgentId` to the entry you want. Each definition includes display name, base URL, bearer token, and 能力 flags. After editing, restart the 节点 or reload bridge config so the new destination binds to port `3031` (or your configured `bridgeListenPort`).

22.7 启动设置与运行时设置

`openclawEnabled` and `bridgeEnabled` are persisted in `node-config.json` and take effect on 节点 start—or immediately stop a running gateway when flipped off. Runtime status (`getOpenClawStatus`, `getBridgeStatus`) shows whether child processes are actually healthy, which can lag config during startup. Model provider mode, AI rules, and 联系人 preferences also persist to `node-config.json` and apply on the next 智能体 turn without restart.

22.8 诊断智能体可用性

If EnvoyAI shows **Stopped**, read `lastError` on the OpenClaw status panel—common causes are port `18789` in use, a missing OpenClaw binary, or repeated watchdog restart failures. For the bridge, verify loopback reachability, bearer token match, and that exactly one active 智能体 is selected. CLI helpers include connectivity status; Social's header badges mirror the same effective mode as Settings → AI → AI Engine.

23. 模型与提供商

23.1 模型路由概述

EnvoyMesh uses two tiers: the **native router** (`@envoymesh/models`) serves chat drafts, auto-replies, 终端 assist, and Team-job planning; **OpenClaw** serves Assistant/`@envoy` turns with its own LLM config. Native routing respects the 语义防火墙 (empty prompts rejected, 48K char cap, control-character filter). When OpenClaw is unavailable, Assistant requests fall back to the native provider you configured.

23.2 配置本地模型

Set provider mode to **ollama** in Settings → AI → Model (or `node-config.json`). Point `endpoint` at `http://127.0.0.1:11434/v1` and set `modelName` to your pulled tag (for example `llama3.1`). Local 通话 skip cloud 审批 gates and keep prompts on your machine—ideal for drafts and sensitive 保险箱 context.

23.3 配置远程提供商

Use **openai-compatible** or **anthropic-compatible** mode with the vendor base URL and `apiKey`. Set `modelName` to the remote model ID. Keep `requireApprovalForCloud: true` (default) so non-public context 触发器 an 审批 item before the request leaves your 节点.

23.4 配置 LiteLLM

litellm mode targets a LiteLLM proxy (typically `http://127.0.0.1:4000/v1`) that fans out to many backends. Set `modelName` to the LiteLLM route name and supply the proxy API 密钥 if required. This is the flexible choice when one home 节点 should switch models without editing EnvoyMesh config.

23.5 选择默认模型

Pick one native model for chat drafts and auto-replies; OpenClaw manages its own model separately in OpenClaw settings. Prefer a fast, inexpensive model for drafts and a stronger model (local or proxied) for Assistant if you split configs. Document your choice in the 资料 README so restores on a new machine stay consistent.

23.6 配置降级行为

When native mode is **disabled**, drafts and assist features return errors instead of calling a model. When OpenClaw is down, Assistant turns degrade to the native provider automatically. For LiteLLM or cloud 端点, verify fallback routes inside LiteLLM itself—EnvoyMesh does not chain multiple native providers in one request.

23.7 上下文窗口考量

Large 保险箱 RAG injections and long Team-job prompts consume context quickly. The 语义防火墙 caps prompt size at 48K characters for native 通话. Trim 知识库 `maxChunks` or lower per-联系人 syndication ceilings when you see truncated answers. OpenClaw 会话 记忆 is separate—very long Assistant threads may need manual 会话 reset.

23.8 提供商隐私

mock mode never 通话 an external 网络—useful for tests. **ollama** and local LiteLLM keep bytes on LAN. Cloud modes send prompt text to the configured vendor; pair with 敏感度 labels and `requireApprovalForCloud` so private notes do not leave without explicit consent. OpenClaw's own model 通话 follow OpenClaw config, not the native router.

23.9 成本控制

协作任务 and competitive award modes track spend in 授权; set `maxCost` and rebalance 策略 under chain defaults. For chat, prefer local models for high-volume auto-replies and reserve cloud models for occasional Assistant turns. Review Activity for correlated cloud 通话 after enabling auto-send rules.

23.10 排查模型调用

Empty or rejected prompts usually mean semantic-firewall validation failed—check for control characters or excessive length. Connection errors on Ollama/LiteLLM point to wrong `endpoint` or a stopped service. Persistent cloud denials often mean an 审批 is pending: open Approvals before retrying. Set mode to **mock** temporarily to confirm the 智能体 loop runs without external dependencies.

24. 智能体风格、模式与联系人行为

24.1 智能体沟通风格

Under Settings → AI → Identity, choose **transparent** (default), **invisible**, or **defensive** presentation. Transparent replies openly as an AI; invisible drafts as if you typed them (still signed with 智能体 role on the wire); defensive acts as a gatekeeper when you appear offline. Optional `debugPrefixInMessageText` adds a prefix in logs only—Social hides it in the UI.

24.2 智能体运行模式

Global defaults live in `aiSettings.defaultModeForNewContacts`: **manual** (draft only), **assistant** (suggest + confirm), or **auto** (send when 策略 allows). Online/offline behavior is controlled separately: `onlineAssistantEnabled` keeps suggestions while you are active; `offlineAgentEnabled` permits auto-reply when the 节点 thinks you are away. Set `statusMode` to manual if automatic presence detection misreads your schedule.

24.3 按联系人设置模式

Each 联系人 can override global defaults with `aiAccessLevel`: **none**, **assistant_only**, or **full**. None blocks AI participation for that 对等节点; assistant_only allows drafts and gated sends; full enables richer automation including rule 触发器. Set these from the 联系人 detail sheet or via `mesh.set-contact-mode` during 智能体-assisted setup.

24.4 按联系人设置披露规则

`knowledgeAccess` caps what 保险箱 material the 智能体 may cite for a 联系人 (`public`, `friends`, `trusted`, or `private`). Optional `syndicationMaxSensitivity` tightens inbound answers you syndicate to that 对等节点. `disclosure` settings (badges, collapse 对等节点 智能体 to 联系人) are local UI only—they do not change wire payloads. Align disclosure with 信任层级 before enabling auto-send.

24.5 社交代理行为

Social proxy (requires Trust mode) lets EnvoyAI mediate intros and standing social workflows under a signed 授权. Enable `socialProxyEnabled` only after `trustModeEnabled` is on and you have configured a 授权 ID. The 协调代理 respects `autonomousKillSwitch`—when kill switch is on, proxy passes stop even if the feature flag is set.

24.6 主动问候

Proactive behavior combines AI rules, 触发器, and friend autopilot (`friendAutopilotEnabled`). Rules match greetings, keywords, or 联系人 access levels and choose draft, auto_send, gatekeep, or defer actions. Rate limits (`autoReplyLimits`) cap hourly and daily auto-replies per 联系人 so a single thread cannot spam while you are away.

24.7 暂停或限制自动化

Toggle **autonomousKillSwitch** for an immediate global pause—every autonomous action becomes an 审批. Pause individual 触发器 from Settings or `mesh.update-trigger`. Lower a 联系人 to **assistant_only** or **none** to stop auto-send for one relationship without disabling EnvoyAI entirely.

24.8 重置智能体行为

Clear AI rules, reset 联系人 preferences to defaults, and turn off social proxy and autopilot flags in Settings → AI. Restart OpenClaw if 会话 tone drifted across long threads. For a hard reset, disable EnvoyAI, clear pending 审批 you no longer need, re-enable, and re-test with a single bonded 联系人 at **manual** mode.

25. 会话与记忆

25.1 什么是会话

An EnvoyAI 会话 binds your ongoing Assistant 对话 to OpenClaw's 记忆 store via a stable `sessionId`. Owner turns in Social's EnvoyAI chat, `@envoy` mentions, and 终端-correlated plans share this binding so follow-up questions stay coherent. Sessions are local to the home 节点—not replicated to EnvoyGo except through live RPC.

25.2 对话上下文

Each OpenClaw request carries 所有者 interests, bonded 联系人 names with trust levels, and the exported 工具 catalog. Native chat drafts use a slimmer context window through the model router. Correlation IDs in 审计 logs stitch a single turn across 工具 通话—use them when reviewing Activity after a complex exchange.

25.3 短期与长期记忆

OpenClaw retains short-term thread state inside the active 会话 and longer recall through its own 记忆 subsystem (including optional MCP bridges like Memex when configured). EnvoyMesh does not duplicate that long-term store in the 保险箱 by default. Treat OpenClaw's workspace and 记忆 plugins as the source of truth for "what the assistant remembers."

25.4 搜索记忆

Use OpenClaw-facing 工具 or configured MCP search (`memex_search` by default in 知识库 settings) to query external 记忆 indexes. Inside EnvoyMesh, `mesh.chat_rag_search` retrieves indexed chat and 资料库 snippets for 智能体 turns. Results inherit 敏感度 labels—do not expose private RAG chunks to public 联系人.

25.5 会话摘要

Call `mesh.session-summary` or list 会话 via `mesh.list-sessions` to inspect OpenClaw thread metadata without opening the gateway UI. Summaries help before handing off a 任务 to 协作任务 or filing 审计 notes. They are operator-oriented views, not wire 消息 to 联系人.

25.6 更正过时记忆

When OpenClaw states a stale fact, correct it in the Assistant thread and, if using Memex or similar, update or archive the source card. Adjust 资料库 notes that fed RAG so the next `mesh.chat_rag_search` returns current text. Per-联系人 preferences may also need updating if the error involved disclosure scope.

25.7 删除记忆

Revoke external 记忆 entries through the MCP 工具's archive/delete path configured in 知识库 settings. Clear OpenClaw 会话 state by starting a new 会话 ID (restart gateway for a full wipe). Removing local chat logs does not erase OpenClaw 记忆 until you delete on that side too.

25.8 保留与隐私

Session and 记忆 data live under your 资料 directory and OpenClaw workspace paths with `0600` 文件 modes. Back up the 资料 before OS migration. Cloud 记忆 plugins follow their vendor retention—disable them for air-gapped deployments.

25.9 跨设备记忆

EnvoyGo displays live Assistant replies from the home 节点 but does not host OpenClaw 记忆 locally. All persistent recall stays on the home machine where the gateway runs. Pairing a new phone does not copy 会话 history unless you restore the home 资料.

25.10 当前聊天历史集成的边界

Full recent-chat injection into every OpenClaw turn is not complete—绑定 and interests attach reliably; verbatim thread scrollback may be partial. Native auto-replies use current 消息 text only. Plan important continuity by referencing 资料库 notes or explicit summaries in your prompt until chat-log integration ships.

26. 工具

26.1 什么是智能体工具

A 工具 is a named, schema-described action the 智能体 can invoke—send chat, query 知识, list 审批, etc. EnvoyMesh registers 工具 in `ToolRegistry`, evaluates 绑定策略 and 敏感度, then executes or queues 审批. Every invocation produces an 审计事件 with 工具 name, latency, and correlation ID.

26.2 浏览可用 mesh 工具

In Social, open Settings → AI → Tools (or ask EnvoyAI to list 工具). CLI and bridge clients can 通话 `mesh.mcp.list_tools` when MCP proxying is enabled. The startup catalog exported to OpenClaw mirrors the same names—`mesh.*` prefix for mesh operations, plus standard chat/知识 entries.

26.3 知识与资料库工具

Use `mesh.library_list`, `mesh.library_read`, `mesh.library_discover`, and `mesh.chat_rag_search` to read local notes and query indexed content. `mesh.knowledge.query` (and 任务 variants) reaches bonded 对等节点' public or permitted indexes. Sensitivity ceilings on each 工具 prevent exfiltrating private 保险箱 paths to strangers.

26.4 联系人与消息工具

`chat.send` and mesh discovery/hello 工具 let the 智能体 find 联系人 and draft 消息. Sends to non-trivial 敏感度 usually enter the 审批队列 rather than delivering immediately. Trust intro 工具 (`mesh.intro.*`) appear only when Trust mode is enabled on the 节点.

26.5 文件共享工具

Sharing flows through `mesh.share_propose`, `mesh.library_request_share`, `mesh.transfer_status`, and gallery helpers. Raw 文件 transfer above 策略 ceiling requires 所有者 审批 and explicit 对等节点 accept. Check `mesh.share_list_pending` before assuming a transfer completed.

26.6 任务与智能体网络工具

`mesh.task.propose`, `mesh.task.await_result`, and `mesh.capability_provider.start` participate in 对等节点 任务 and 协作任务. Agent card 工具 (`mesh.agent_card.request`, `mesh.list_agent_network_workers`) support 工作节点 discovery. Competitive award flows may enqueue `chain_award` 审批 when spend or bid rules 触发器.

26.7 审批与升级工具

`mesh.list-pending`, `mesh.approve`, `mesh.reject`, `mesh.reject-all`, and `mesh.escalate` let the 智能体 surface work to you or pause when uncertain. Prefer escalation over silent failure when confidence is low or sentiment is negative. The 智能体 should not approve its own queued items unless 策略 explicitly allows auto-resolution.

26.8 MCP 工具

`mesh.mcp.list_tools` and `mesh.mcp.call_tool` proxy to configured MCP HTTP servers (for example Memex). Each 通话 inherits the same 审批 and 审计 path as native 工具. Register only MCP servers you trust— they execute with the 节点's local 网络 access.

26.9 启用或禁用访问

Disable Trust intro 工具 by turning off `trustModeEnabled`. Pause MCP servers in 知识库 settings. Use `autonomousKillSwitch` to block execution of autonomous 工具 chains without removing the catalog. Bridge 智能体 receive a filtered mesh 工具 list via the HTTP bridge— not the full registry.

26.10 审查工具执行

Open Activity and filter by 工具 or correlation ID. Each row shows allow/deny, remote 对等节点, and summary text. For bridge traffic, also check `mesh.list-external-agent-actions`. Cross-check pending 审批 if a 工具 returned "queued" instead of `ok: true`.

27. 触发器、计划与摘要

27.1 创建触发器

Triggers live in the 节点 触发器 store and fire proactive actions. Create time-based (cron, interval, or one-shot), event-based (消息 received, 联系人 online/offline), or topic-based (keyword match) 触发器 from Settings → AI → Automation or via `mesh.add-trigger`. Each 触发器 declares an action type—send chat, query 知识, send 摘要, notify 所有者, or follow up—and a daily fire cap.

27.2 更新或移除触发器

Edit conditions or pause a 触发器 with `mesh.update-trigger`; delete with `mesh.remove-trigger`. Paused 触发器 retain history but do not fire. After changing cron expressions, confirm the next scheduled time in the automation panel so timezone mistakes do not surprise you.

27.3 安排提醒与动作

Time 触发器 accept cron strings, ISO `at` timestamps, or `intervalMs` for repeating checks. The 节点 evaluates due 触发器 on its periodic loop and records `trigger.fired` 审计事件. Chat sends from 触发器 still pass 审批 策略—high-risk templates queue instead of auto-sending.

27.4 配置活动摘要

Digest settings (`mesh.set-digest-schedule`, `mesh.get-digest-config`) control **daily**, **weekly**, or **off** summaries written under your 资料 `digests/` directory. Toggle sections: 外部智能体 通话, discovery queries, 绑定 changes, proactive actions, and pending 审批. When a 摘要 is ready, Social emits a `digest:ready` event you can open from Activity.

27.5 晨报与发现摘要

Morning report (`getMorningReport`) ranks recent discovery events and trust-store signals—a separate, on-demand discovery 摘要 from periodic activity 摘要. Run it from Social discovery panels or CLI `morning-report` when evaluating new public 对等节点. It does not send mesh 消息 by itself.

27.6 跟进与主动检查

Follow-up actions re-open a 联系人 thread after delays you define in 触发器 metadata. Proactive check-ins combine offline detection (`offlineAgentEnabled`) with rules and 触发器— for example, defer a draft when sentiment is negative. Escalations from proactive passes appear in Approvals with `proactive_checkin` or `follow_up` action types.

27.7 勿扰时段与通知偏好

Per-domain **智能体 visibility** (`instant`, `brief`, `silent`, `approval`) controls push noise for 任务, intros, and reports without stopping the underlying automation. Use **silent** overnight and **审批** during focus blocks so only 审批-needed events interrupt you. This is 通知 loudness, not a separate cron quiet-hours clock—combine with paused 触发器 for true blackout windows.

27.8 审查自动化历史

Filter Activity for `trigger.fired`, 摘要 generation, and proactive 智能体 events. Each entry includes 触发器 name, action type, and correlation ID. Compare against `mesh.list-triggers` status fields (`firesToday`, `lastFiredAt`, `lastError`) when a schedule misfires.

27.9 停止自动化

Hit **autonomousKillSwitch** to halt all proactive firing immediately. Individually disable 触发器, turn off `offlineAgentEnabled`, or set 摘要 frequency to **off**. Cancel in-flight proactive chat by rejecting the 审批 item before it expires.

28. 审批与升级

28.1 为什么 EnvoyMesh 会请求审批

Approvals enforce 所有者 consent for actions that exceed 绑定 tier, 敏感度上限, or autonomous 策略: outbound chat drafts, 知识 shares, cloud model 通话, discovery forwards, 摘要, and Team-job awards. The queue is the control surface between 智能体 intent and mesh execution—nothing in the pending list has been sent yet.

28.2 审查待处理动作

Open **Approvals** in Social or 通话 `listPendingApprovals` from CLI. Each item shows title, draft content, action type, priority, and request timestamp. Read the draft as if it would send verbatim—edits after 审批 are not automatic unless you reject and ask the 智能体 to regenerate.

28.3 检查联系人、数据与能力范围

Inspect context fields: 联系人 所有者 ID, display name, 敏感度等级, requested 能力, and linked 触发器 name if automation-fired. Confirm the recipient matches your intent and the 敏感度 label fits the relationship tier. Reject if the 智能体 requested private data for a public or referred 联系人.

28.4 批准动作

Approve from the Approvals panel or CLI approve command; the executor runs the underlying 工具 or send path and marks the item resolved. Approved sends propagate as normal signed envelopes. Cloud model 审批 unblock the specific native router 通话 tied to the item.

28.5 拒绝单个或全部动作

Reject with an optional note so 审计 shows 所有者 intent. `mesh.reject-all` clears the queue when you distrust a batch—for example after a misconfigured auto rule. Rejection does not penalize the 联系人; it only blocks that draft.

28.6 升级原因

Items escalate to **escalated** status when confidence is below 0.6, sentiment is negative, or 敏感度 score exceeds the threshold. Manual escalation via `mesh.escalate` flags thorny threads for 所有者 attention even when 策略 might allow auto-send. Escalated items stay visible until acknowledged.

28.7 确认升级

Use **Acknowledge** in Approvals or `mesh.acknowledge-escalation` after you have read the context—even if you reject the underlying action. Acknowledgment clears urgent signaling without approving the draft. Pair with a 联系人 mode change if the 对等节点 should stay on manual assist going forward.

28.8 过期审批

Pending items expire after seven days by default; expired entries cannot be approved without a new 智能体 request. The 节点 periodically purges expired IDs and logs the count. If you routinely miss the window, switch risky 联系人 to manual mode and raise visibility to **审批** only.

28.9 智能体网络授予审批

Competitive 协作任务 may enqueue `chain_award` items when a 工作节点 bid needs 所有者 sign-off on spend or selection. Review bid price, 工作节点 身份, and 授权 budget before approving. Direct award mode skips bidding but still respects 授权 `maxCost`.

28.10 避免审批疲劳

Start new 联系人 in **manual** mode, enable auto-send only for trusted 对等节点, and use autonomous 策略 with tight 敏感度 ceilings. Prefer **brief** or **审批** 智能体 visibility so low-value activity does not ping you. Audit weekly: if the same rule generates noise, pause the 触发器 or narrow its keywords.

第 V 部分 —— 知识、资料库与网络内容

29. 知识体系概述

29.1 知识库、资料库、保险箱与 RAG

The 知识库 is the user experience, the 资料库 organizes discoverable items, the 保险箱 stores local 文件, and RAG retrieves relevant chunks for an 智能体 prompt. These layers work together but have different 安全 and lifecycle responsibilities.

29.2 本地优先存储

Your 保险箱 文件 and 资料 metadata live on the home 节点's disk first—typically under the 资料's 保险箱 directory with `notes/`, `documents/`, `inbox/`, and `.envoy/` metadata. Nothing requires a cloud sync service for ordinary reading or editing in Social desktop. Paired EnvoyGo reads and writes through home RPC; it does not hold a full 保险箱 replica on the phone by default.

29.3 笔记、文件与结构化信息

Markdown notes are created in the 资料库 UI and stored under `vault/notes/` with optional subfolders you define. Imported PDFs, Word 文件, images, and plain text land in `documents/` or legacy 保险箱 paths and join the same index. Structured `.envoy/sensitivity.json` overrides track per-item visibility independent of folder layout.

29.4 可见性与敏感度

Each item carries a 敏感度 tier—public, friends, trusted, or private—controlled by the Published toggle and Obsidian frontmatter when plugins are enabled. Bonds map 对等节点 trust tiers to the maximum 敏感度 they may read or receive in syndicated responses. Changing 敏感度 re-indexes RAG visibility without moving 文件 on disk.

29.5 搜索与检索

Local search scans indexed 保险箱 chunks; chat RAG retrieves the best matches to ground model answers with citations. Remote 对等节点 use `knowledge.query` for natural-language search or `library.read` for path-based byte retrieval when browsing published web content. These paths differ: search synthesizes or ranks text; 资料库 read serves 文件 verbatim.

29.6 受信任的远程知识

Bonded 联系人 may query syndicated 知识 within ceilings you set per relationship. Strangers on the public tier can query only public notes through rate-limited `knowledge.query` and see a stripped wiki-link graph. Federated RAG merges local and remote chunks when 策略 allows, preserving source attribution in responses.

29.7 来源与哈希

Content hashes fingerprint 保险箱 bytes and appear in discovery matches, 浏览器 verification, and IPFS export metadata. Hashes let recipients confirm they received unaltered 文件 without trusting filename alone. Publishing updates may change bytes at a stable path; verify hash when integrity matters more than friendly titles.

29.8 发布与浏览

Phase 45 adds URL-addressable mesh pages under `envoy://owner/path` served from the home `web/` directory with per-entry visibility. Social 浏览器 and paired EnvoyGo 浏览器 render Markdown, images, and PDFs like a lightweight web client. Feeds and topic 通知 (45E) alert followers when authors publish, but fetching remains pull-based via `library.read`.

29.9 IPFS 集成

Optional IPFS export publishes content-addressed copies of selected 资料库 items through Helia or Kubo integrations. CIDs complement mesh discovery but do not replace 绑定-gated `library.read` for authorized browsing. Treat IPFS as distribution and verification aid, not as implicit 权限 to ignore 敏感度 labels.

30. 创建与组织知识

30.1 创建 Markdown 笔记

Open the 资料库 tab in Social, choose New Note, and enter Markdown in the editor; saves land in `vault/notes/` automatically. The RAG pipeline re-indexes on save without restarting the 节点. Use `createNote` JSON-RPC when automating note creation from scripts or integrations.

30.2 编辑与预览笔记

Switch between edit and preview modes to validate formatting before sharing or publishing. Preview uses the same sanitization path as chat rendering so you see roughly what bonded readers will see. EnvoyMesh does not silently rewrite note bodies except through explicit plugin import flows.

30.3 组织文件夹

Create subfolders under `notes/` for research, work, or personal categories—the UI mirrors 保险箱 paths. Sensitivity remains per note, so one folder can mix public tutorials and private drafts. Obsidian users can organize the same directories externally while EnvoyMesh indexes changes on refresh.

30.4 添加文件

Drag or import 文件 into `documents/` for PDF, DOCX, images, and text formats the indexer supports. Large imports may take a moment to chunk for RAG; check 资料库 status if search lags. Received 对等节点 文件 arrive in `inbox/` with separate handling from authored notes.

30.5 Choose public, friends, trusted, or private visibility

Toggle Published in the 资料库 item editor or set Obsidian `published: true/false` frontmatter when the Obsidian plugin is enabled. Public items join the stranger-queryable mesh; friends items require at least referred 绑定 tier; private items stay local and 智能体-only. Review labels before bulk imports that default to private.

30.6 管理元数据

Titles, paths, tags, and 敏感度 overrides form the metadata layer the 资料库 displays and discovery matches against. `.envoy/sensitivity.json` persists overrides across restarts. Avoid hand-editing metadata 文件 while the 节点 is running unless you follow operator backup procedures.

30.7 使用 Obsidian 集成

Enable the Obsidian plugin under Settings → AI → 知识库 → Plugins, then point Obsidian at your 保险箱 directory for rich editing. The plugin parses frontmatter, builds a wiki-link graph, and strips private links from stranger-facing responses. EnvoyMesh never writes Obsidian 文件 directly—all mutations go through Social or RPC.

30.8 导入与导出内容

Export notes for offline archives or import markdown batches during migration from other 工具. Verify 敏感度 labels after import because external 工具 may not understand EnvoyMesh tiers. Keep a filesystem backup before bulk delete or path rewrites that could orphan index entries.

30.9 安全删除知识

Delete removes 保险箱 文件 and index entries together when using 资料库 delete actions or `deleteNote` RPC. Published web manifest entries may need separate unpublish steps if the item was exposed at an `envoy://` path. Confirm no bonded 对等节点 rely on syndicated copies before deleting authoritative originals.

31. 搜索与 RAG

31.1 搜索本地知识

Use 资料库 search for keyword retrieval across indexed 保险箱 chunks on your home 节点. Results show matching excerpts with paths so you can open the source note or document. Search respects 敏感度—you will not see private chunks in contexts meant for strangers.

31.2 让 EnvoyAI 搜索

Ask EnvoyAI in chat to find information; it invokes RAG 工具 that retrieve 保险箱 chunks before answering. Answers should cite paths or titles when attribution is enabled in your configuration. Remote model 通话 still pass through the 语义防火墙 and 绑定 checks on outbound context.

31.3 了解分块与匹配

RAG splits documents into chunks for embedding and retrieval; matches are ranked by relevance to your query. Chunk boundaries may split paragraphs, so read surrounding context in the source 文件 when precision matters. Re-indexing after large edits refreshes chunk boundaries automatically on save.

31.4 审查来源归属

Review citations in chat or 知识 responses to confirm which note or 文件 supplied each claim. Federated results include remote 所有者 identifiers so you know whether text came from your 保险箱 or a 对等节点's syndicated 资料库. Save attributed excerpts with MCP write-back when you want a durable local copy.

31.5 聊天 RAG 搜索

Chat RAG runs during assistant turns, combining retrieval with model generation in one flow. It differs from manual 资料库 search because the model synthesizes an answer grounded on retrieved chunks. Disable or narrow 工具 if you prefer search-only interactions without generative summarization.

31.6 跨受信任联系人的联邦 RAG

Federated RAG queries opted-in 联系人 libraries within syndication ceilings configured under trust settings. Private notes never leave your 节点; friends-tier material requires sufficient 绑定 tier on both sides. Conflicting facts from multiple 对等节点 should be resolved by reading originals, not trusting merged summaries alone.

31.7 处理冲突结果

When local and remote chunks disagree, open each cited source and compare hashes or timestamps. Models may over-merge paraphrases; treat RAG output as a map to evidence, not as authority. Adjust syndication settings if a 联系人's automated summaries are consistently misleading.

31.8 保存有用结果

Use MCP write-back or manual note creation to store useful query results in your 保险箱 with default friends 敏感度. Include source 对等节点 and query text in the note body for future 审计. Avoid saving strangers' private-leak attempts—verify 敏感度 before publishing saved summaries.

31.9 保护敏感信息

Keep credentials, medical, and legal material at private 敏感度 unless you explicitly accept broader exposure. Public mesh queries rate-limit strangers and strip non-public wiki links from responses. Audit 知识 queries periodically if you syndicate friends-tier content to referred 联系人.

32. 受信任的知识共享

32.1 向已绑定联系人询问知识

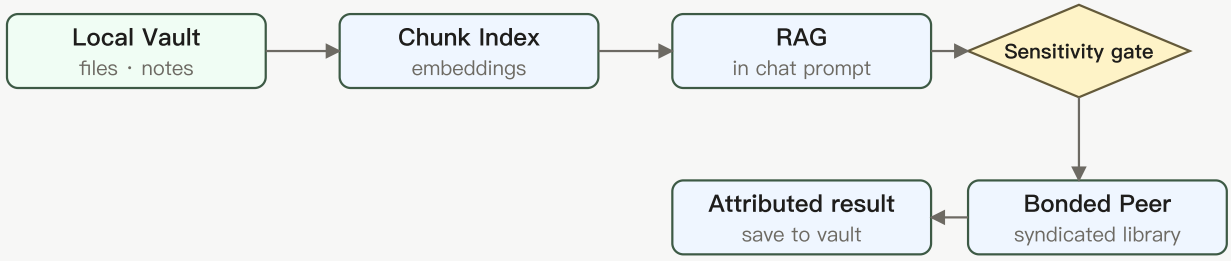


图 16 —— 联邦 RAG：本地保险箱分块直接供给 RAG；联邦路径通过按联系人的敏感度上限门查询已绑定对等节点的资料库，返回带归属的结果。

Send a `knowledge.query` intent to a bonded 联系人's 智能体 when you need their syndicated 资料库 summarized or searched. The remote 节点 applies its 绑定 tier, syndication ceilings, and model routing before answering. Ask precise questions and expect natural-language responses, not raw 文件 dumps.

32.2 公开、介绍与直接访问

Public tier allows stranger queries with tight rate limits; referred tier unlocks broader syndicated access; direct tier allows friends-敏感度 sharing. Each tier maps to deterministic 绑定策略 decisions logged in 审计. Upgrade 绑定 deliberately—referred access exposes more than public ping alone.

32.3 共享笔记或文件

Share notes or 文件 by sending chat attachments, data-transfer vouchers, or publishing with appropriate visibility. Vouchers copy bytes into the recipient inbox; publishing exposes metadata via discovery or bytes via `library.read`. Pick the mechanism that matches whether you want a point copy or ongoing browse access.

32.4 提议共享

Propose shares through 任务 or chat flows when your workflow requires explicit recipient acceptance. Proposals carry 敏感度 hints so recipients know what they are importing before indexing. Cancel proposals that stall to avoid ambiguous half-shared state.

32.5 接受共享请求

Accept inbound share requests only after verifying sender 绑定 tier and described 敏感度. Imported content lands in 保险箱 inbox or 资料库 lists with attribution to the remote 所有者. Re-index or adjust 敏感度 if you intend to re-share material further.

32.6 敏感度强制

The Bonds engine denies requests that exceed allowed 敏感度 for the sender's 信任层级, even when users believe they are friends. Syndication max settings cap what leaves your 节点 during automated queries. Test with a secondary 联系人 account if you tune syndication for a research 群组.

32.7 联系人范围发现

Contact-scoped discovery returns published 资料库 metadata for bonded 对等节点 without exposing private paths. Matches include titles, hashes, and optional CIDs—not full text until a follow-up read or query. Use scoped discovery before wide searches to respect relationship boundaries.

32.8 全网文档发现

Network-wide document discovery advertises public published 能力 on the DHT for strangers meeting 能力 and rate rules. It supports finding public material, not enumerating private vaults. Operators should monitor 审计 for unusual query volume from public 对等节点.

32.9 速率限制与防滥用

Stranger `knowledge.query` traffic is rate-limited (on the order of a few queries per minute and tens per hour in default configuration). Abuse protection complements 绑定 denials to reduce scanning of public notes. Report persistent abuse by blocking public-tier 对等节点.

32.10 防止意外披露

Double-check Published toggles before promoting notes to public or friends tiers, especially after Obsidian sync. Web manifest visibility uses separate ACL fields—including 联系人 pickers for 联系人-only pages. Anti-enumeration returns `not_found` for unauthorized `library.read` attempts rather than confirming hidden paths exist.

33. 发布与浏览 mesh 内容

33.1 `envoy://` 地址格式

Mesh content URLs follow `envoy://envoy:owner:<id>/path/to/page` using permanent 所有者 IDs, not display names. `@handle` syntax parses but is rejected at runtime until a future registry ships. Pairing URIs (`envoy://contact?...`) remain distinct and must not be confused with content URLs.

33.2 打开 mesh 页面

Open a mesh page from chat links, 浏览器 address bar, or feed 通知 in Social desktop. Paired EnvoyGo forwards `library.read` through the home 节点, so browsing away from home requires connectivity to that 节点. Pages render Markdown, images, and PDFs with sanitized HTML where applicable.

33.3 浏览历史

浏览器 back, forward, reload, and per-所有者 history behave similarly to a conventional web client within mesh constraints. Large binary bodies may arrive in ranged chunks with hash verification at display time. Navigation guards prevent overlapping in-flight reads from corrupting the view state.

33.4 创建书签

Bookmark frequently visited `envoy://` pages per 所有者 in 浏览器; autocomplete suggests recent paths as you type. Bookmarks stay local to your client 资料—they do not sync through a central server. Export bookmarks manually if you rebuild a 设备.

33.5 浏览作者

Browse an author's site by opening their 所有者 root URL, which serves `index.md` when present under `web/`. Blog, 资料, PhotoWall, and Bazaar templates organize paths by convention, not hard schema. Visibility still applies per 文件—seeing an index does not imply access to every subpath.

33.6 浏览 Bazaar 内容

Bazaar and feed views aggregate discoverable public or bonded content depending on manifest and topic subscriptions. Topic follow (Phase 45E) helps match interests without GossipSub push fanout on the wire. Discovery lists metadata first; opening an entry 触发器 `library.read` for bytes.

33.7 发布页面

Author pages in Social place Markdown or media into `~/EnvoyMesh/web/` (or 资料-equivalent) and register manifest entries with visibility. Choose public, bonded, or specific-联系人 ACL before sharing URLs in chat. Updating content changes bytes at the same path—notify followers via feeds when updates matter.

33.8 关注动态与主题

Follow feeds and topics to receive inbox 通知 when authors publish matching material. Notifications link into 浏览器 with the originating `envoy://` URL. Unfollow topics you no longer want to avoid 通知 noise.

33.9 更新已发布内容

Edit source 文件 locally, bump manifests, and republish when correcting typos or replacing media. Clients verify `contentHash` when reloading to detect changes since last visit. There is no built-in version history URL—keep 保险箱 git or snapshots if you need rollback.

33.10 外部 HTTP 网关 —— 计划中

Planned. The `envoy://` mesh-content path is available today; a public HTTP gateway for non-mesh browsing is forward-referenced as Phase 45F and is not part of the current release.

34. IPFS 与内容校验

34.1 为什么 EnvoyMesh 使用内容哈希

Hashes identify content independently of filenames so recipients detect tampering after transfer or IPFS fetch. EnvoyMesh surfaces hashes in discovery, 浏览器, and export dialogs. Treat hash mismatch as a hard stop before trusting quoted text or binaries.

34.2 将资料库内容导出到 IPFS

Export selected 资料库 items to IPFS when you want content-addressed sharing outside immediate mesh pull paths. Export respects 敏感度—do not pin material you would not publish at the same visibility tier. Record CIDs alongside mesh URLs when sharing with hybrid audiences.

34.3 Helia 集成

Helia integration embeds a lightweight IPFS 节点 suitable for desktop home 节点 exporting or verifying CIDs. Configure Helia when you need in-process pinning without a separate Kubo daemon. Monitor disk use because pinned blocks accumulate locally.

34.4 Kubo 集成

Kubo integration targets operators who already run a Kubo daemon and want EnvoyMesh to interoperate with its API. Point settings at your local Kubo 端点 and verify connectivity before bulk export jobs. Kubo and Helia are alternatives—typically enable one strategy per 节点.

34.5 通过网关校验内容

Public gateways help humans fetch IPFS CIDs through HTTPS for verification, but gateways are not authorization layers. Compare gateway bytes to expected mesh hashes before treating content as authentic. Sensitive material should not rely on public gateways for access control.

34.6 固定与可用性

Pinning keeps IPFS blocks reachable; unpinned CIDs may disappear when no 对等节点 hosts them. Mesh `library.read` remains authoritative for authorized live reads from the 所有者's home 节点. Use pinning for archival redundancy, not as a substitute for 保险箱 backups.

34.7 隐私考量

Publishing to IPFS or public mesh tiers exposes bytes to anyone who obtains the CID or URL regardless of friendly filenames. Private 保险箱 material should stay off export and unpublish lists. Review Phase 44 stranger-query behavior before marking research notes public.

34.8 Filecoin 持久化 —— 延期

Deferred. Helia and Kubo IPFS paths are available today; Filecoin-based long-term persistence is designed but not part of the current release. See Appendix J.9.

35. 备份与恢复知识

35.1 需要备份什么

Back up 所有者 密钥, 保险箱 directories, `.envoy/` metadata, web manifests, 资料 JSON state, and 审计 journals you need for compliance. 资料库 UI state alone is insufficient without underlying 保险箱 文件. Document your backup schedule alongside 中继 or model credentials stored outside EnvoyMesh.

35.2 备份保险箱

Copy the entire 保险箱 tree—including `notes/`, `documents/`, `inbox/`, and `.envoy/`—while the 节点 is stopped or quiesced to avoid partial 文件. Verify free space before large copies. Encrypt backups at rest if they contain friends-tier or private material.

35.3 备份资料库元数据

资料库 metadata such as 敏感度 overrides and published flags lives under `.envoy/sensitivity.json` and related stores—include these with 保险箱 backups. Published web manifests under the 资料 directory should backup with `web/` content. Missing metadata restores 文件 but may wrong-foot visibility until repaired.

35.4 在同一节点恢复

Restore 保险箱 and 资料 data into the same 资料 path, then restart the 节点 and run index refresh if search seems stale. Confirm 绑定 and trust stores if you restored partial 资料 trees. Test one private and one public query before returning to production use.

35.5 迁移到另一台电脑

Moving to a new computer requires copying 资料, 保险箱, and 所有者 密钥 material, then reinstalling EnvoyMesh and re-pairing EnvoyGo 设备. Update 中继 bootstrap or port settings if 网络 layout changed. Revoke old 设备证书 if the old hardware is discarded.

35.6 校验恢复内容

After restore, spot-check note hashes, open sample `envoy://` pages, and run 资料库 search for known keywords. Federated queries to 联系人 should still work once 绑定 reload. Log discrepancies before deleting the old machine's backup.

35.7 移动数据边界

EnvoyGo does not replace the home 保险箱 on the phone—it caches only what paired RPC 会话 fetch for UI display. Mobile backups mean backing up the home 节点 your phone pairs to, not expecting full 保险箱 export from EnvoyGo alone. Re-pair QR codes after home restore if 会话 tokens invalidate.

35.8 安全修复受损本地数据

If index corruption occurs, stop the 节点, restore 保险箱 from backup, and allow re-indexing rather than deleting unknown 文件 blindly. Use 审计 logs to identify which operations preceded corruption. Contact operator documentation before running manual JSONL edits on metadata stores.

第 VI 部分 —— 外部智能体

36. 外部智能体概述

36.1 什么是外部智能体

An 外部智能体 is a separately running assistant that receives selected 消息 and invokes allowed mesh 工具 through EnvoyMesh's local HTTP bridge. HomeClaw, Hermes, and OpenHuman use the shared `envoymesh-message` contract.

36.2 内置 EnvoyAI 与外部智能体

EnvoyAI/OpenClaw is bundled, deeper, and managed with the home runtime. External 智能体 are compatibility integrations with independently maintained 智能体-side code and should be enabled only when you trust that process.

36.3 为什么外部智能体使用桥接

The bridge converts between plain HTTP requests and signed mesh operations. This keeps networking 密钥, 绑定 checks, 能力 limits, and 审计 records inside EnvoyMesh.

36.4 为什么外部智能体绝不获得原始 P2P 访问

Raw libp2p access would let an 智能体 evade 身份 and 策略 boundaries. The bridge exposes intentional operations instead, such as sending a 消息, finding 知识, or executing an approved 工具.

36.5 外部智能体身份

The bridge 智能体 has its own mesh 对等身份 derived from the 所有者 授权, distinct from the external runtime's internal user or 会话 IDs. Bonded 对等节点 消息 that bridge 身份; EnvoyMesh signs outbound replies on its behalf.

36.6 可用桥接工具

When enabled, compatible 智能体 may 通话 `GET /bridge/list-tools` and `POST /bridge/execute-tool` on port 3031. The catalogue reflects 绑定策略, 授权, and 所有者 审批—not every 工具 on the home 节点.

36.7 会话与动作历史

External-智能体 会话 and action history appear in Settings for review. Each inbound mesh 消息 and 工具 invocation is correlated in the 审计 JSONL so you can trace what the bridge forwarded to the external process.

36.8 权限、审批与吊销

High-risk mesh operations may still require 所有者 审批 even when the bridge forwards the request. Revoke access by disabling the bridge, clearing the active preset, or rotating the Bearer secret shared with the 外部智能体.

36.9 每个桥接仅一个活动外部智能体 URL

A bridge resolves one active external-智能体 URL at a time. You may retain several presets, but switch the active preset rather than sending the same inbound event to several assistants and creating duplicate replies.

36.10 选择集成

Pick one integration path: bundled EnvoyAI (OpenClaw on port 18789), HomeClaw (8010), Hermes (8020), OpenHuman (8021), or a custom `envoymesh-message` adapter. Only one `agentUrl` is active per bridge 资料 at a time.

37. 安全的智能体桥接

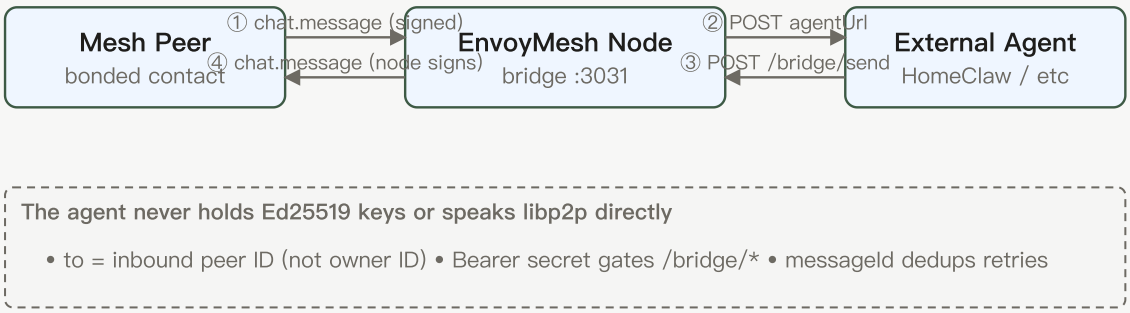


图 7 —— 外部智能体桥接：mesh 流量经由 EnvoyMesh 节点，由其代为签名。外部智能体接收纯 HTTP，并通过 /bridge/send 回复。

37.1 mesh 到智能体的消息流

When a bonded 对等节点 消息 the bridge 智能体, EnvoyMesh validates the signed envelope and POSTs a compact 消息 object to the configured 智能体 URL. The object includes sender routing information, display context, text, and a unique 消息 identifier.

37.2 智能体到 mesh 的回复流

The 外部智能体 replies by POSTing `{ to, text }` to `/bridge/send` on the local bridge, normally port `3031`. The `to` value is the inbound mesh 对等节点 ID, not the 所有者 ID; EnvoyMesh signs and sends the outbound envelope.

37.3 Bearer 令牌认证

Set a bridge secret so requests use `Authorization: Bearer <secret>`. Use a long random value, store it like a credential, and rotate it after suspected disclosure.

37.4 消息标识与去重保护

The inbound `messageId` lets an 智能体 suppress repeated webhook deliveries. The OpenClaw extension also has a short content-hash fallback for older bridges, but integrations should prefer exact 消息-ID deduplication.

37.5 关联标识与同步回复

Owner-to-智能体 synchronous asks include a correlation ID. A matching `/bridge/send` resolves the pending local request; an unknown correlation receives a gone response so the 智能体 can retry instead of silently losing the answer.

37.6 异步知识与发现回复

Discovery and 知识 responses can arrive after the initiating 工具 通话. Compatible 智能体 should handle `mesh.async_reply` events and associate them with the user's ongoing context.

37.7 列出与执行 mesh 工具

`GET /bridge/list-tools` returns the allowed 工具 catalogue and `POST /bridge/execute-tool` invokes a selected operation. Both remain subject to bridge authentication, 工具 schemas, 策略, and 审批.

37.8 提议文件共享

An 智能体 can propose sharing a 保险箱 item through `/bridge/agent-share-proposal`; it does not gain unrestricted filesystem access. The 所有者 or 策略 path decides whether the share proceeds.

37.9 本地默认与网络暴露

The bridge listens on loopback by default. Do not expose port `3031` directly to a LAN or the Internet; if remote access is necessary, place an authenticated, TLS-protected proxy in front and restrict its source 网络.

37.10 审计外部智能体活动

Filter 审计 logs for bridge intents and 工具 executions. Look for remote 对等节点 IDs, correlation IDs, allow/deny outcomes, and latency. This is the authoritative record when disputing what an 外部智能体 did on your behalf.

37.11 吊销外部智能体

Disable **Ext Agent** in Settings, clear or change `agentUrl` in bridge config, and rotate the Bearer secret. Stop the external process so it cannot keep calling port 3031 with a stale credential.

38. OpenClaw 与 EnvoyAI

38.1 OpenClaw 在 EnvoyMesh 中的角色

OpenClaw supplies EnvoyAI's bundled assistant runtime and also supports the canonical EnvoyMesh channel extension. That extension handles webhook 消息, reply routing, mesh 工具, asynchronous replies, and onboarding surfaces.

38.2 内置运行时与权威 EnvoyMesh 扩展

The packaged runtime and `OpenClawExtension/` are maintained with EnvoyMesh, which makes this integration richer than the generic compatibility presets. The extension is also installable into another OpenClaw checkout.

38.3 自动启动

The home 节点 normally starts OpenClaw automatically on gateway port `18789`. If it is disabled in startup configuration, enable it and restart the 节点 before expecting EnvoyAI responses.

38.4 在其他 OpenClaw 环境安装扩展

To install the channel in another checkout, run `./scripts/install-openclaw-extension.sh /path/to/openclaw --with-docs`, install that checkout's dependencies, and configure its EnvoyMesh webhook and bridge secret.

38.5 配置 EnvoyMesh 频道

In OpenClaw config, register the EnvoyMesh channel with webhook path `/webhook/envoymesh` on gateway port **18789** and set `bridgeUrl` to `http://127.0.0.1:3031/bridge/send`. Match the Bearer secret to `bridge-config.json` on the EnvoyMesh 节点.

38.6 发送与接收 mesh 消息

Bonded 对等节点 chat with the bridge 智能体 对等节点 ID; EnvoyMesh POSTs JSON to `http://127.0.0.1:18789/webhook/envoymesh`. Replies go to `/bridge/send` with `to` set to the sender's mesh 对等节点 ID (`envoy_…`), not the 所有者 DID.

38.7 列出与执行 mesh 工具

The OpenClaw extension exposes `envoymesh_list_mesh_tools` and `envoymesh_execute_mesh_tool`, which proxy to the bridge on 3031. Tool 通话 still pass 绑定 checks and 语义防火墙 rules on the home 节点.

38.8 处理异步 mesh 回复

Discovery and 知识 responses may arrive asynchronously. The extension handles `mesh.async_reply` POSTs to the webhook and surfaces them as in-channel 消息 so the model can continue the 对话.

38.9 使用入门与设置界面

Run `openclaw onboard` or use the bundled Social setup flow to seed workspace, bridge secret, and channel docs. Confirm the gateway log registers the EnvoyMesh HTTP route before testing with a bonded 联系人.

38.10 管理扩展与 ClawHub

Install optional OpenClaw extensions through ClawHub or symlinks; the EnvoyMesh channel lives in `OpenClawExtension/`. macOS bundles more extensions than Windows; add others manually when needed.

38.11 macOS 内置扩展选择

The macOS DMG includes a broader set of OpenClaw extensions for an integrated experience. This increases package size but reduces post-install setup for common workflows.

38.12 Windows 精简扩展包

The Windows installer packages the essential useful OpenClaw extensions rather than the full macOS set, keeping the bundle within practical size limits. Additional extensions can be installed separately when needed.

38.13 从 Hermes 迁移

Use the bundled migration extension to import Hermes memories, skills, or credentials into OpenClaw, then point `agentUrl` from `8020/message` to the OpenClaw webhook. Back up both environments and verify imported secrets before switching production traffic.

38.14 排查 OpenClaw

If chat fails: confirm gateway on 18789, bridge log shows 3031, webhook path matches, Bearer secrets align, and `to` on replies is a 对等节点 ID. Run `npm run smoke:openclaw-bridge` from the repo for a local round-trip check.

39. HomeClaw

39.1 HomeClaw 预设提供什么

HomeClaw is the default external-智能体 compatibility preset and conventionally receives 消息 at `http://127.0.0.1:8010/message`. EnvoyMesh supplies the bridge configuration; HomeClaw supplies its 智能体运行时 and channel implementation.

39.2 兼容预设状态

Compatibility preset. The EnvoyMesh side is available, but verify the compatible HomeClaw release and its `channels/envoymesh` support before production use.

39.3 启动 HomeClaw

Start HomeClaw so its EnvoyMesh channel listens on **8010** (default `http://127.0.0.1:8010/message`). Verify the process is bound to loopback unless you deliberately run 智能体 and 节点 on different hosts.

39.4 在设置中选择 HomeClaw

In **Settings → AI → Ext Agent**, enable the bridge and choose the HomeClaw preset. EnvoyMesh sets `agentUrl` to `http://127.0.0.1:8010/message` and starts forwarding bonded 对等节点 chat to that 端点.

39.5 配置消息 URL

Use the local default `http://127.0.0.1:8010/message` unless HomeClaw is intentionally bound elsewhere. Keep the 端点 on loopback whenever both processes run on the same host.

39.6 配置回复桥接

Configure HomeClaw to return replies to `http://127.0.0.1:3031/bridge/send`. Use the same Bearer secret on both sides.

39.7 添加共享密钥

Generate a long random secret in bridge settings and configure the same value in HomeClaw's EnvoyMesh channel. Both inbound POSTs to 8010 and outbound POSTs to 3031 should send `Authorization: Bearer <secret>`.

39.8 发送与接收消息

When a bonded 联系人 消息 your bridge 智能体, HomeClaw receives `{from, fromOwnerId, fromName, text, messageId}`. Replies POST to `http://127.0.0.1:3031/bridge/send` with `{to, text}` where `to` is the inbound `from` 对等节点 ID.

39.9 使用 mesh 工具

If HomeClaw's channel implements 工具 proxies, it 通话 list/execute 端点 on 3031. Each 工具 remains subject to 绑定 tier, 授权 bounds, and 所有者 审批 queues on the EnvoyMesh 节点.

39.10 权限与知识访问

Knowledge and 保险箱 reads flow through mesh 工具—not direct filesystem access. Tune HomeClaw's own 权限 separately; EnvoyMesh still enforces 敏感度 ceilings and 联系人 scope on every 工具 通话.

39.11 智能体端频道归属

The `channels/envoymesh` implementation lives in the HomeClaw repository. EnvoyMesh only configures URLs and secrets; upgrade or patch the channel on the HomeClaw side when wire behavior changes.

39.12 断开或吊销 HomeClaw

Disable Ext Agent in Settings, stop HomeClaw, and rotate the bridge secret. Clear `agentUrl` or switch to another preset so queued mesh 消息 are not delivered to a stopped process.

39.13 排查 HomeClaw

Common failures: HomeClaw not listening on 8010, secret mismatch, wrong reply `to` field, or bridge disabled. Check 节点 logs for `[bridge] HTTP on …3031` and curl 8010/消息 with a test payload plus Bearer header.

40. Hermes

40.1 Hermes 预设提供什么

Hermes is a built-in compatibility preset using the same 消息 contract, conventionally at `http://127.0.0.1:8020/message`. Its 知识-oriented runtime is maintained outside this repository.

40.2 兼容预设状态

Compatibility preset. EnvoyMesh provides selection and bridging, not a guarantee about every Hermes version. Test the exact release and configured 工具 before enabling it for 联系人.

40.3 启动 Hermes

Launch Hermes with its EnvoyMesh adapter on **8020** (`http://127.0.0.1:8020/message`). Confirm the release you run matches the compatibility preset expectations in release notes.

40.4 在设置中选择 Hermes

Select the Hermes preset under **Settings → AI → Ext Agent**. EnvoyMesh points `agentUrl` at 8020 and enables the bridge listener on 3031 for return traffic.

40.5 配置消息与回复 URL

Set inbound 消息 to `http://127.0.0.1:8020/message` and configure Hermes to reply via `http://127.0.0.1:3031/bridge/send`. Keep both URLs on loopback for same-machine setups.

40.6 添加共享密钥

Copy the bridge secret from EnvoyMesh settings into Hermes's EnvoyMesh channel configuration. Mismatched Bearer tokens produce 401 responses on both 8020 and 3031.

40.7 发送与接收消息

Hermes receives the standard bridge payload on 8020. Outbound replies must target the sender 对等节点 ID from `from`; 所有者 DIDs will not route correctly on the mesh.

40.8 使用知识与 mesh 工具

Hermes's 知识-oriented 工具 map to mesh list/execute 通话 when implemented in its adapter. 保险箱 and discovery results may return asynchronously through the same async-reply pattern OpenClaw uses.

40.9 权限与审批

Hermes-side prompts and 记忆 are outside EnvoyMesh 策略. Mesh-side 审批 still apply when a 工具 would exceed 授权 cost, 敏感度, or 联系人 scope.

40.10 智能体端集成归属

Hermes maintains its own integration code and release cadence. EnvoyMesh supplies the preset URLs and bridge 安全 boundary only.

40.11 从 Hermes 迁移 to OpenClaw

A bundled OpenClaw migration extension can import supported Hermes configuration, memories, skills, or credentials. Back up both environments and review imported secrets before switching the active runtime.

40.12 断开或吊销 Hermes

Turn off the Hermes preset, rotate secrets, and stop the Hermes process. Consider migrating to OpenClaw with the migration extension if you need a supported bundled path.

40.13 排查 Hermes

Verify 8020 is reachable, secrets match, and Hermes version supports `messageId` deduplication. Inspect bridge 审计事件 for denied 工具 通话 versus transport errors.

41. OpenHuman

41.1 OpenHuman 预设提供什么

OpenHuman is a built-in compatibility preset using the shared adapter, conventionally at `http://127.0.0.1:8021/message`. Its 智能体-side runtime remains an external project.

41.2 兼容预设状态

Compatibility preset. Verify the OpenHuman release, 端点 behavior, and consent model independently; EnvoyMesh secures only the mesh-facing bridge boundary.

41.3 为什么 OpenHuman 默认禁用

OpenHuman is disabled by default so installing EnvoyMesh never silently grants an unverified external process access to 对话 or 工具. Enable it only after configuration and trust review.

41.4 启动 OpenHuman

Start OpenHuman with its adapter listening on **8021** (`http://127.0.0.1:8021/message`). Because OpenHuman is disabled by default, confirm you intentionally enabled it after reviewing its consent model.

41.5 启用并选择 OpenHuman

Enable OpenHuman in bridge settings and select its preset. EnvoyMesh will not auto-start OpenHuman; both the external process and the Ext Agent toggle must be on.

41.6 配置消息与回复 URL

Configure `http://127.0.0.1:8021/message` for inbound mesh traffic and `http://127.0.0.1:3031/bridge/send` for replies. Document any non-default ports in both OpenHuman and `bridge-config.json`.

41.7 添加共享密钥

Set a shared Bearer secret in EnvoyMesh bridge settings and OpenHuman's channel config. Treat rotation like credential compromise response: update both sides before resuming traffic.

41.8 发送与接收消息

OpenHuman handles inbound `{from, text, messageId, …}` like other presets. Replies use the 对等节点 ID in `from`; duplicate `messageId` values should be ignored to absorb retries.

41.9 使用 mesh 工具

Tool access is limited to what the bridge exposes via list/execute on 3031. OpenHuman cannot bypass 绑定 or 授权 checks by calling libp2p directly.

41.10 同意、隐私与审批

Review OpenHuman's consent prompts and data retention separately from EnvoyMesh 策略. Owner 审批 on the home 节点 still gate sensitive mesh operations.

41.11 智能体端集成归属

OpenHuman ships its own integration layer; EnvoyMesh does not vet every 智能体-side behavior. Keep OpenHuman updated and disable the preset if its 安全 posture changes.

41.12 断开或吊销 OpenHuman

Disable the preset, revoke the secret, and stop OpenHuman. Clearing Ext Agent returns chat to EnvoyAI or another selected engine without exposing 8021.

41.13 排查 OpenHuman

Check that OpenHuman is enabled, listening on 8021, and using matching Bearer auth. Consent or 审批 denials may look like transport failures—inspect 审计 outcomes.

42. 自定义外部智能体

42.1 使用 `envoymesh-message` 适配器

A custom 智能体 can implement the `envoymesh-message` adapter without speaking libp2p. It accepts the bridge's inbound JSON, replies through the local bridge, and may list or execute only the 工具 exposed to it.

42.2 注册自定义智能体预设

Add a custom preset in bridge settings with your adapter's `agentUrl` (for example `http://127.0.0.1:9000/message`). One bridge 资料 points to one active URL.

42.3 实现入站消息端点

Implement `POST /your/message` accepting `{from, fromOwnerId, fromName, text, messageId}` and optional Bearer auth. Respond 200 quickly; deliver replies asynchronously via 3031 rather than echoing in the HTTP response body.

42.4 通过 `/bridge/send` 实现回复

POST `{to, text}` and optional `correlationId` to `http://127.0.0.1:3031/bridge/send`. Use `to` = inbound `from` 对等节点 ID. Sync asks resolve when `correlationId` matches a pending 所有者 request.

42.5 认证请求

Validate `Authorization: Bearer` on inbound mesh webhooks and on your outbound 通话 to 3031. Reject unsigned requests when a secret is configured.

42.6 处理重复消息

Track seen `messageId` values and drop duplicates within your retry window. This prevents double replies when the bridge retries a flaky webhook delivery.

42.7 列出与调用 mesh 工具

Call `GET /bridge/list-tools` then `POST /bridge/execute-tool` with JSON arguments. Handle structured errors and 审批-pending responses without crashing your 智能体 loop.

42.8 处理异步结果

Subscribe to or poll for async mesh results (`mesh.async_reply` shape when mimicking OpenClaw). Associate late discovery or 知识 responses with the user turn that triggered them.

42.9 定义能力与数据边界

Document which mesh 工具 your 智能体 may 通话 and what local data it stores. Never request raw libp2p 密钥 or 保险箱 paths outside approved 工具.

42.10 测试集成

Run bonded 对等节点 chat tests, 工具 通话, and secret-rotation drills. Use `npm run smoke:openclaw-bridge` patterns as a reference for mock round-trips.

42.11 安全清单

Loopback bind only, strong Bearer secret, least-privilege 工具, 审计 review, prompt secret rotation, and a documented revoke path. Do not expose 3031 to LAN/WAN without TLS and 网络 ACLs.

42.12 排查自定义智能体

Compare bridge logs with your adapter logs for 401/404/410 responses, wrong `to` IDs, and schema mismatches. Test with curl before involving live mesh 对等节点.

43. 管理外部智能体

43.1 审查当前智能体

Open **Settings → AI** and confirm which preset is active, its `agentUrl`, whether Ext Agent is enabled, and the bridge listen port (default 3031). Bundled EnvoyAI uses 18789 separately from Ext Agent presets.

43.2 审查外部智能体会话

Review external-智能体 会话 lists for active correlations and recent 对等节点 联系人. Sessions tie mesh senders to bridge forwarding state on the home 节点.

43.3 审查动作历史

Action history summarizes 工具 executions and forwarded 消息. Cross-check unusual entries against 审计 JSONL using the same correlation or 消息 IDs.

43.4 检查可用能力

Inspect the 工具 catalogue via Settings or `GET /bridge/list-tools` with auth. Capabilities change when 绑定, 授权, or 所有者 审批 change—not when the 外部智能体 restarts.

43.5 切换当前预设

Switch presets by selecting HomeClaw, Hermes, OpenHuman, OpenClaw webhook, or custom. Restart or reconnect the target external process after changing `agentUrl` or secrets.

43.6 禁用桥接

Toggle **Ext Agent** off to stop forwarding mesh chat to external URLs while keeping bundled EnvoyAI available. The bridge HTTP listener may remain for in-flight replies—rotate secrets if you need a hard stop.

43.7 吊销智能体会话

Revoke a 会话 by disabling the bridge, clearing credentials in the 外部智能体, and rotating the Bearer secret so old tokens cannot 通话 3031.

43.8 轮换共享密钥

Generate a new secret in EnvoyMesh, update the 外部智能体 config, then restart both sides. Expect brief 401 errors until configs match.

43.9 应对被入侵的外部智能体

Immediately disable Ext Agent, rotate secrets, block affected 对等节点 if needed, and review 审计 logs for exfiltration or 工具 abuse. Treat compromise of the external process as compromise of everything the bridge was allowed to do.

43.10 收集诊断

Gather bridge-config (redact secrets), recent 审计 excerpts, gateway/adapter logs, and results of `curl` probes to 3031 and your `agentUrl`. Include EnvoyMesh and external-智能体 versions when filing issues.

第 VII 部分 —— 智能体网络与协作任务

44. 智能体网络概述

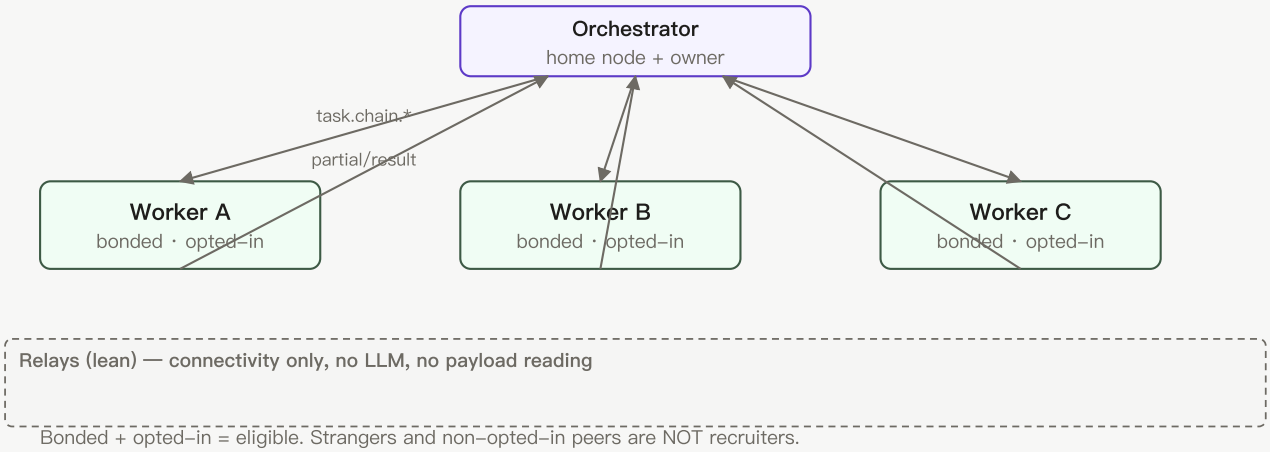


图 8 —— 智能体网络拓扑：协调代理招募已绑定、自愿加入的工作节点。中继仅承载连通性。没有公开市场——陌生人无法招募你的智能体。

44.1 智能体网络是什么

智能体网络 is the collaboration layer where bonded 所有者 opt their local 智能体 into work for 协作任务. It combines 身份, trust, 能力 discovery, 授权, orchestration, 交付物, and auditing.

44.2 已绑定的用户与自愿加入的本地智能体

A 工作节点 is eligible only when the 所有者 are bonded at an acceptable tier, the remote 所有者 enabled Join 智能体网络, and a fresh 智能体 card advertises the required membership and 能力.

44.3 智能体网络不是公开市场

There is no public marketplace where strangers can freely recruit your 智能体. Broad anonymous recruitment is deliberately outside the current product boundary.

44.4 你的智能体默认保持私有

The local 智能体 remains usable by its 所有者 whether or not it joins. Membership changes what bonded 对等节点 can discover and request, and can disclose 所有者-attested 资料 fields to those 对等节点.

44.5 工作节点成员资格

Worker membership is the opt-in flag (**Join 智能体网络**) that advertises `capability-provider` on your 智能体 card. Without it, bonded 对等节点 cannot recruit your 智能体 for 协作任务 even if trust is direct.

44.6 Agent Card 与能力

An **Agent Card** lists 能力, supported 任务 types, optional 资料 fields, and membership tags. Orchestrators index cards from bonded 对等节点 to decide who can execute each subtask.

44.7 协作任务

协作任务 (UI name for multi-智能体 chains) split an 所有者 goal into subtasks, assign them to opted-in 工作节点, and merge results into one report. Protocol code still uses `task.chain.*` intents.

44.8 智能家庭节点与精简中继

Home 节点 run LLMs, 保险箱 access, orchestration, and 工作节点 execution. **Relays** provide connectivity and discovery only—they never execute subtasks or read private payloads.

44.9 典型的个人、家庭与团队拓扑

Personal setups often pair two home laptops; families may add a child's 节点; teams use fleet manifest or LAN onboarding. Every topology still requires 绑定 plus 工作节点 opt-in before cross-home 协作任务 work.

44.10 当前范围与未来方向

Today, 智能体网络 covers bonded, opted-in collaboration: an 所有者 enables Join 智能体网络, bonded 对等节点 see the 工作节点 card, and the requesting 节点 orchestrates 协作任务 across those trusted 工作节点. There is no public marketplace, no anonymous 工作节点 recruitment, and 中继 stay lean (connectivity only). Forward directions — broader discovery, richer reputation, multi-hop commerce, and a complete hierarchical 中继 graph — are documented in Appendix J.5–J.11 as Planned, Parked, or Deferred; treat them as direction, not committed release dates.

45. 加入智能体网络

45.1 前提条件

Before joining: running home 节点, 所有者身份, configured AI engine, and at least one 绑定 if you expect to collaborate soon. Joining alone does not create 绑定 automatically.

45.2 Enable 加入智能体网络

Open **Settings → 智能体网络** and enable **Join 智能体网络**. The 节点 sets 能力-provider membership in its advertisements; it does not create 绑定 automatically.

45.3 成员资格广播什么

Membership advertises the `capability-provider` tag and, if configured, the 智能体网络 资料. Bonded 对等节点 can then index the card and consider the 工作节点 for compatible subtasks.

45.4 关闭成员资格

Disable **Join 智能体网络** in Settings to remove `capability-provider` from your card. In-flight subtasks may finish, but new orchestrators should stop recruiting you after refresh.

45.5 确认你的工作节点可见

On a 对等节点's 节点, open **Settings → 智能体网络 → Workers status** and click **Refresh 工作节点**. Your entry appears when 绑定 trust is eligible, you joined, and a fresh card synced.

45.6 未加入时本地智能体的行为

When not joined, your local 智能体 still serves chat, 保险箱, and personal 任务 on your 节点. Only recruitability to bonded 对等节点' 协作任务 is withheld.

45.7 隐私影响

Joining shares 能力 tags and optional 资料 fields with bonded 对等节点—not the public internet. Strangers cannot browse your 智能体; only 联系人 who already passed 绑定策略 see recruitability signals.

45.8 排查成员资格

If membership seems stuck: toggle Join off/on, restart the 节点, confirm 智能体 card publish, and ask a bonded 对等节点 to refresh 工作节点. Check 审计 for card fetch or index errors.

46. 智能体网络资料

46.1 所有者自证的工作节点资料

The 资料 is an 所有者-attested description used for soft 工作节点 ranking, not a centrally verified benchmark. It may include model freshness, spend posture, context window, strengths, and throughput.

46.2 模型新鲜度

Model freshness (1–10) is 所有者-attested signal for how current your models feel. Orchestrators use it as a soft tie-breaker after 能力 match, not as verified benchmark.

46.3 消费姿态

Spend posture (`subscription`, `metered`, `unknown`) hints whether long jobs may hit provider limits. It influences scoring but does not override 授权 cost ceilings.

46.4 上下文窗口

Context window (`128k` – `1M+`) helps orchestrators pick 工作节点 for large-document subtasks. Misstating window size may cause assignment mismatch or failed execution—keep it honest.

46.5 优势与技能标签

Strengths and skill tags (research, coding, summarization, etc.) improve soft ranking when several 工作节点 share the same 能力. They do not grant 能力 you did not advertise on the card.

46.6 吞吐量信息

Throughput information (when provided) helps Assigner estimate parallel capacity. It is informational; stall detection still relies on heartbeats and 协调代理 timers.

46.7 候选评分如何运作

Candidate scoring prioritizes 能力 match, then uses context, freshness, spend posture, strengths, and related signals. These factors guide assignment but do not override 绑定 and 授权 策略.

46.8 资料信任与局限

Profiles are **self-declared** by 所有者 you already trust via 绑定—not third-party ratings. Treat them as hints; verify outcomes through reports, 审计, and repeated collaboration.

46.9 更新或移除资料

Edit 资料 fields under **Settings → 智能体网络** anytime. Clearing the 资料 removes soft-ranking hints but does not disable Join; toggle membership separately to stop recruitment.

47. 智能体身份与 Agent Card

47.1 为什么智能体有独立身份

Agents have **independent 对等节点 IDs** (`envoy_agent_…`) derived from 所有者 + 智能体 密钥 so 对等节点 can verify an 智能体 acts under a specific 所有者 授权, separate from 设备 密钥.

47.2 所有者、设备、智能体与对等节点关系

Owner 身份 authorize 授权; **设备** run 节点; **智能体** execute 任务; **对等节点 IDs** sign envelopes at runtime. 协作任务 always address 智能体 对等节点 for 任务 traffic.

47.3 所有者授权的智能体凭据

Owner-signed **授权** link an 智能体 public 密钥 to an 所有者 DID. Workers should reject chain proposals that lack valid 授权 signatures within cost, 敏感度, and expiry bounds.

47.4 智能体公钥

Each 智能体 publishes a **public 密钥** on its card. Recipients verify envelope signatures before accepting `task.chain.*` or `task.result` payloads.

47.5 Agent Card

An Agent Card describes an 智能体's 身份, 能力, 任务 support, optional 工作节点 资料, and 端点. Native cards move through signed EnvoyMesh flows; an A2A bridge can publish a filtered external representation.

47.6 能力与支持的任务类型

Capabilities are string tags (`doc.translate`, `task.execute`, …) on the card. **Supported 任务 types** describe which wire intents the 智能体 accepts. Subtasks declare required 能力; mismatches exclude a 工作节点.

47.7 成员资格标签

Membership tags include `capability-provider` when Join 智能体网络 is enabled. Orchestrators filter on this tag before offering subtasks to a bonded 联系人.

47.8 获取与刷新已绑定智能体的名片

Cards auto-fetch when 绑定 form (eligible tiers) and cache ~24h. Use **Refresh 工作节点** or 绑定 events to force update before a critical 协作任务.

47.9 验证智能体的所有者

Verify `ownerId` on the card matches the bonded 联系人 you expect. Mandate signatures must chain to that 所有者; mismatches are grounds to reject work.

47.10 吊销智能体

Revoke an 智能体 by 所有者 授权 revocation and removing or rotating its 密钥 on the home 节点. Peers with stale cards should refresh; blocked trust stops new assignments immediately.

47.11 一个所有者的多个智能体

One 所有者 may run **multiple 智能体** with distinct 密钥 and cards. Each opts in and advertises 能力 independently; orchestrators treat them as separate 工作节点.

48. 绑定与工作节点资格

48.1 绑定信任层级

Trust tiers are **blocked**, **public**, **referred**, and **direct**. 协作任务 工作节点 generally require referred or direct trust; public strangers are not recruitable 工作节点.

48.2 Why 协作任务 require bonded contacts

协作任务 operate across bonded relationships because 工作节点 may receive objectives, data context, and delegated authority. Bond 策略 prevents unknown public 对等节点 from entering this workflow by default.

48.3 公开对等节点与陌生人

Public 对等节点 are not auto-fetched as 协作任务 工作节点. Bond them to referred or direct before expecting collaboration.

48.4 介绍级工作节点

Referred 工作节点 may participate under tighter 策略. 协调代理-side chain traffic typically requires referred or higher; confirm 绑定 tier before assigning sensitive subtasks.

48.5 直接级工作节点

Direct 绑定 unlock the full 工作节点 path: direct assign, bidding, and cross-home handoff subject to 授权. This is the usual friend/fleet configuration.

48.6 阻止级工作节点

Blocked 对等节点 cannot send or receive collaboration intents. Existing assignments should 失败即关闭; cancel active subtasks involving blocked 工作节点.

48.7 能力要求

Each subtask names a **required 能力**. Workers must advertise an exact or soft-matched tag plus `capability-provider` membership to be eligible.

48.8 成员资格与名片新鲜度

Stale cards may hide new 能力 or show revoked 智能体. Refresh after membership toggles, model changes, or 绑定 updates; orchestrators skip 工作节点 beyond freshness thresholds.

48.9 工作节点资格清单

Confirm: the 所有者 are bonded; trust is referred or direct as required; Join 智能体网络 is enabled; the card is fresh; `capability-provider` is present; the requested 能力 matches; and neither side is blocked.

48.10 更改或吊销信任

Lowering trust or blocking a 联系人 stops new recruitment immediately. Review active 协作任务 for in-flight subtasks awarded to that 对等节点 and cancel or reassign as needed.

49. 发现与接入工作节点

49.1 绑定现有联系人

Bond an existing 联系人 through chat intro, QR, or invite before recruiting them. 协作任务 never substitute anonymous discovery for trust establishment.

49.2 办公局域网接入

Office LAN onboarding combines same-Wi-Fi discovery with a shared token under **Settings → 智能体网络**. It accelerates bonding for coworkers on one 网络.

49.3 局域网自动绑定

LAN auto-绑定 pairs machines on the same subnet when enabled and tokens match. It creates trust, not membership—each 对等节点 must still enable Join 智能体网络 to become a 工作节点.

49.4 公司邀请链接

Company invitation links (`envoy://invite?…`) let teammates join your fleet with scoped trust. Distribute links through your normal secure channels; expired links stop working.

49.5 配对自助端

Pairing kiosk mints one-click invites for events or support desks. Keep kiosk mode off unless you actively supervise pairings—it reduces friction by design.

49.6 集群清单导入

Fleet Manifest import applies a signed roster of 对等节点 and trust hints for larger teams. Validate manifest signatures before import; manifests create 绑定, not automatic 工作节点 opt-in.

49.7 刷新工作节点状态

Click **Refresh 工作节点** on the 智能体网络 tab after onboarding changes. The 能力 index updates from freshly fetched 智能体 cards across bonded 联系人.

49.8 基于能力的匹配

Assigner matches subtask `requiredCapability` to 工作节点 card tags, then applies soft scoring (context, freshness, strengths, same-LAN hints). Missing 能力 excludes the 工作节点 entirely.

49.9 探测对等节点

Probe a 对等节点 sends a lightweight reachability check before awarding expensive subtasks. Failed probes remove unreachable 工作节点 from the current roster pass.

49.10 诊断零合格工作节点

If no 工作节点 is eligible, refresh cards, verify 绑定, confirm membership, inspect 能力 tags, and probe reachability. The UI should not start a multi-智能体 job when only the local 节点 is available.

49.11 广泛的匿名工作节点发现 —— 当前不提供

Planned boundary. Current 协作任务 recruit bonded, opted-in 工作节点. Network-wide anonymous 工作节点 search and public-marketplace behavior are not offered.

50. 协作任务基础

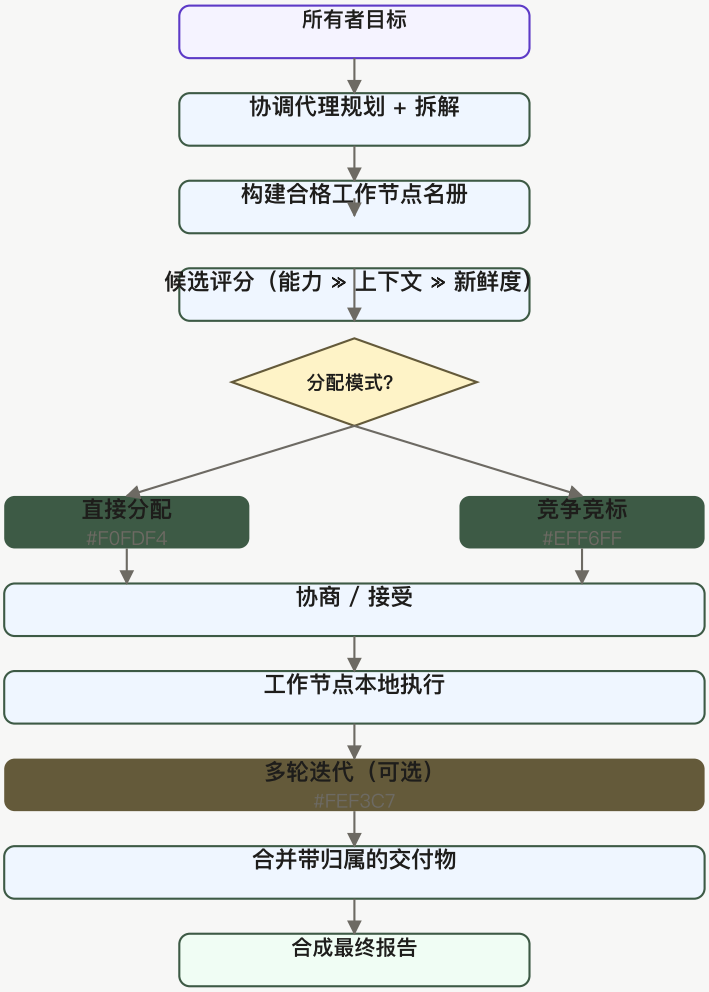


图 6 —— 协作任务编排：协调代理规划、构建名册、为候选评分，然后分支到直接分配或竞争竞标。工作节点本地执行；结果合并为一份带归属的报告。

50.1 什么是协作任务

A 协作任务 turns one 所有者 goal into coordinated subtasks executed by several eligible 智能体. The initiating home 节点 owns the budget and report and normally acts as 协调代理.

50.2 协作任务 and the older “chains” name

The UI says **协作任务**. Protocol intents, RPC names, storage, and older documents may use **chain**; treat that as the implementation name for the same product workflow.

50.3 所需工作节点

At least one eligible 远程工作节点 is required for meaningful multi-智能体 execution. A solo 节点 can use its personal 智能体, but the 协作任务 UI blocks or reports no-工作节点 rather than pretending to distribute work.

50.4 设定目标

Enter a clear, bounded goal in **协作任务 → New team job** or promote from chat. Good goals state deliverable, constraints, and 敏感度 so the planner can decompose realistically.

50.5 预览计划

Preview the plan shows proposed subtasks, 能力, and 工作节点 slots before spend starts. Edit or cancel here if the decomposition looks wrong.

50.6 从聊天发起

From chat, escalate a 对话 turn into a 协作任务 when the goal needs multiple 智能体. The 协调代理 inherits context subject to 授权 敏感度 limits.

50.7 Start from the 协作任务 view

The **协作任务** view is the primary control surface on desktop Social: start, monitor, approve, rebalance, and open reports. EnvoyGo mirrors status read-only.

50.8 跟踪进度

Watch lifecycle states (`discovering`, `running`, `partial`, `synthesizing`, …) and per-subtask rows. WebSocket `chain:iteration` events update iteration progress during Phase 47 multi-round jobs.

50.9 审查已完成报告

Open the published **chain report** for attributed sections, 工作节点 provenance, cost summary, and pinned 交付物. Draft rounds (Phase 47) appear in accordion before final publish.

50.10 取消或重试协作任务

Cancel from 协作任务 UI sends `task.chain.cancel` downstream. Retry may require a new job or 协调代理 re-plan depending on failure mode; check 审计 for 终端 reason.

51. 规划与拆解工作

51.1 协调代理的角色

The 协调代理 turns the 所有者's goal into a plan, finds 工作节点, awards subtasks, tracks execution, enforces budget and 策略, and merges results. Relays only carry connectivity and never assume this role.

51.2 将目标拆解为子任务

The 协调代理 decomposes the goal into subtasks with objectives, required 能力, inputs, deadlines, and cost ceilings. LLM-assisted planning may propose steps; 所有者 preview approves before dispatch.

51.3 所需能力

Each subtask declares a **required 能力** tag matching 智能体 card entries. Planning fails early if no bonded, opted-in 工作节点 advertises that 能力.

51.4 依赖与顺序

Dependencies order subtasks (for example research before synthesis). The 协调代理 respects DAG edges and will not award dependent work until prerequisites complete or partial results arrive.

51.5 工作节点数量上限

maxWorkers caps concurrent active 工作节点 会话 on a chain 授权. Finished or cancelled subtasks free slots for reassignment.

51.6 深度上限

Default chain **depth is 2** (协调代理 → 工作节点). Depth 3 requires `allowDepth3` on the 所有者-signed chain 授权; depth beyond 3 is rejected.

51.7 截止时间与敏感度

Set per-subtask deadlines and 敏感度 ceilings in the plan preview. Workers enforce local 绑定 and 保险箱策略 even when 协调代理 requests higher 敏感度.

51.8 预览与编辑计划

Edit subtask objectives, costs, or 能力 tags in preview when manual mode is available. Automatic LLM plans should be reviewed before start on high-stakes goals.

51.9 LLM 辅助规划

LLM-assisted planning uses the home model to propose decomposition when enabled. Failures fall back to keyword/heuristic templates or block start with a clear planning error.

51.10 规划失败与降级行为

When planning fails, check for zero eligible 工作节点, unsupported 能力, depth violations, or model errors. Reduce goal scope or add 工作节点 before retrying.

52. 发现与分配智能体

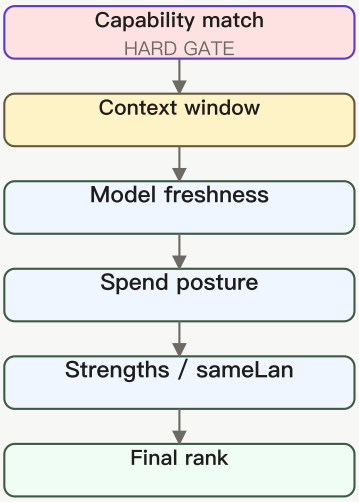
52.1 构建合格工作节点名册

The roster lists bonded 联系人 who joined 智能体网络 and pass 能力 filters for the current plan. Same-LAN 对等节点 may rank higher when dial hints show direct paths.

52.2 能力匹配

Capability matching is hard filter first: no tag, no assignment. Soft scoring breaks ties among remaining eligible 工作节点.

52.3 上下文、新鲜度、消费与优势评分



Priority decreases downward. Capability is

图 14 —— 候选评分漏斗：能力匹配是硬门；其下，软因素（上下文、新鲜度、消费、优势）贡献最终排名。

Scoring weighs **context window**, **freshness**, **spend posture**, and **strengths** after 能力 fit. Direct assign picks the top scored 工作节点; bidding still uses score as signal.

52.4 同网络考量

Workers on the same LAN may respond faster and rank higher (`sameLan` soft score). WAN paths use 中继 for connectivity without changing trust requirements.

52.5 直接分配模式

Direct assign is the default for personal and small-team use. It selects an eligible scored 工作节点 and awards work without exposing a bidding flow.

52.6 竞争竞标模式

Competitive bidding collects offers when cost, timing, or choice matters. It adds negotiation and 所有者 decisions, so enable it only when those controls justify the extra delay.

52.7 分配器选择

Assigner selection chooses which home 节点 plans and awards subtasks—usually yours. Remote assigner handoff delegates that role to another bonded 协调代理 with better 工作节点 visibility.

52.8 远程分配器交接

Remote assigner handoff transfers assignment authority via `task.chain.handoff` while preserving 授权 bounds and Phase 47 iteration knobs when configured.

52.9 无工作节点行为

With **no eligible 工作节点**, start is blocked (`no_workers`). Enable Join on 对等节点, refresh cards, or 绑定 additional 联系人—solo 节点 cannot fake multi-智能体 execution.

52.10 刷新与重新评估工作节点

Re-run roster build after refresh, 绑定 changes, or mid-job stalls. 协调代理 may swap 工作节点 when probes fail or bids expire.

53. 竞标与协商

53.1 何时使用竞标

Bidding is used only in competitive mode or a workflow that explicitly requests offers. Direct assign avoids this exchange.

53.2 请求竞标

In **competitive bidding** mode, the 协调代理 broadcasts `task.chain.propose` and collects `task.chain.bid` responses before accept. Direct assign skips this step.

53.3 审查提议的成本与时间

Compare each bid's proposed **cost** and **ETA** against subtask ceilings and chain budget. Reject bids that exceed 授权 limits without 所有者 审批.

53.4 审查信心与理由

Review bid **confidence** and textual justification when shown. Low-confidence bids may warrant counter-proposal or a different 工作节点.

53.5 比较候选

Side-by-side candidate comparison highlights cost, score, and 能力 fit. Owner picks accept when manual award mode is enabled.

53.6 还价

Counter-bids adjust cost, deadline, or scope via `task.chain` negotiation envelopes. Workers may accept revised terms or withdraw.

53.7 接受或拒绝工作

Accepting emits `task.chain.accept`; rejecting leaves the subtask open for other bidders or reassignment. Document decisions in 审计 for later cost disputes.

53.8 协商超时

Negotiation timers prevent indefinite stalls. Expired bids free the subtask for re-offer or fallback direct assign per 协作任务 defaults.

53.9 协商期间的人工审批

Sensitive or high-cost accepts may enter **所有者 审批** queues. Resolve 审批 in Social before 工作节点 start execution.

53.10 审计协商决策

Audit records capture bid amounts, accepted 对等节点, counter-proposal history, and 审批 outcomes. Export or filter by `chainId` for retrospective review.

54. 预算、成本与再平衡

54.1 设定协作任务预算

Set **maxChainCostUsd** and related limits when starting a job or in saved recipes. The 协调代理 tracks spend against the chain budget ledger throughout execution.

54.2 成本上限

Each subtask carries a **cost ceiling**; 工作节点 cannot bid or charge above it without rebalance or 所有者 审批.

54.3 工作节点成本分配

Initial allocation splits chain budget across subtasks in the plan. Manual rebalance moves funds; automatic rebalance adjusts within configured increments.

54.4 手动再平衡

Manual rebalance pauses for 所有者 review when allocation or 工作节点 conditions change. It maximizes control at the cost of requiring timely attention.

54.5 自动再平衡

Automatic rebalance lets the 协调代理 adjust within configured increments, ceiling, and retry limits. Use conservative limits and require 审批 for material cost increases.

54.6 从不再平衡策略

Never-rebalance preserves the original budget allocation. A stalled or underfunded subtask may then fail rather than consume additional funds.

54.7 再平衡增量与限制

Configure **rebalance increment** size and maximum automatic retries in 协作任务 defaults. Conservative increments reduce surprise spend.

54.8 高成本审批

Crossing high-cost thresholds 触发器 **审批 requirements** on the 授权. Watch for waiting-for-所有者 states when spend spikes.

54.9 导出成本数据

Export cost breakdowns from completed reports or 审计 CSV when enabled. Includes per-工作节点 attribution and rebalance events.

54.10 解读最终成本报告

Final cost reports show estimated versus actual spend, rebalance history, and unspent budget. Compare to 授权 `maxChainCostUsd` before starting similar jobs.

55. 运行与监控工作

55.1 工作节点接受

After accept, 工作节点 acknowledge and transition subtasks to **running**. Reject or timeout returns the subtask to negotiating or failed states.

55.2 运行状态

Chain lifecycle moves through `discovering → negotiating → running → partial → synthesizing → completed|failed`. UI maps these to human-readable 协作任务 status.

55.3 心跳

Workers send heartbeats so the 协调代理 can distinguish progress from disconnection. Missing heartbeats feed stall detection but should not be treated as proof of malicious behavior.

55.4 部分结果

Workers emit **`task.chain.partial`** with intermediate 交付物 when more output is coming. 协调代理 waits or merges partials per termination 策略.

55.5 停滞检测

Stall detection uses missed heartbeats and configured timeouts. Trigger retry, reassignment, or 所有者 prompt per stall 策略.

55.6 重试与重新分配

Retry re-offers a subtask; **reassignment** cancels the stalled 工作节点 slot and awards a backup. Release 工作节点 capacity before assigning replacements.

55.7 等待所有者输入

Jobs pause in **waiting_for_owner** when 审批, iteration continue/stop, or rebalance decisions are needed. Resolve in desktop Social—EnvoyGo shows the state but cannot act.

55.8 工作节点失败

Worker **failure** marks subtasks failed with reason codes. 协调代理 may synthesize partial reports or fail the chain per termination 策略.

55.9 取消子任务或整个任务

Cancel a single subtask or entire chain from 协作任务. Downstream 工作节点 receive cancel intents; 审计 records who initiated termination.

55.10 审计进度

Filter 审计 by `chainId` and correlation IDs to reconstruct timeline: plan, bids, accepts, partials, merge, iteration rounds, publish.

56. 多轮迭代

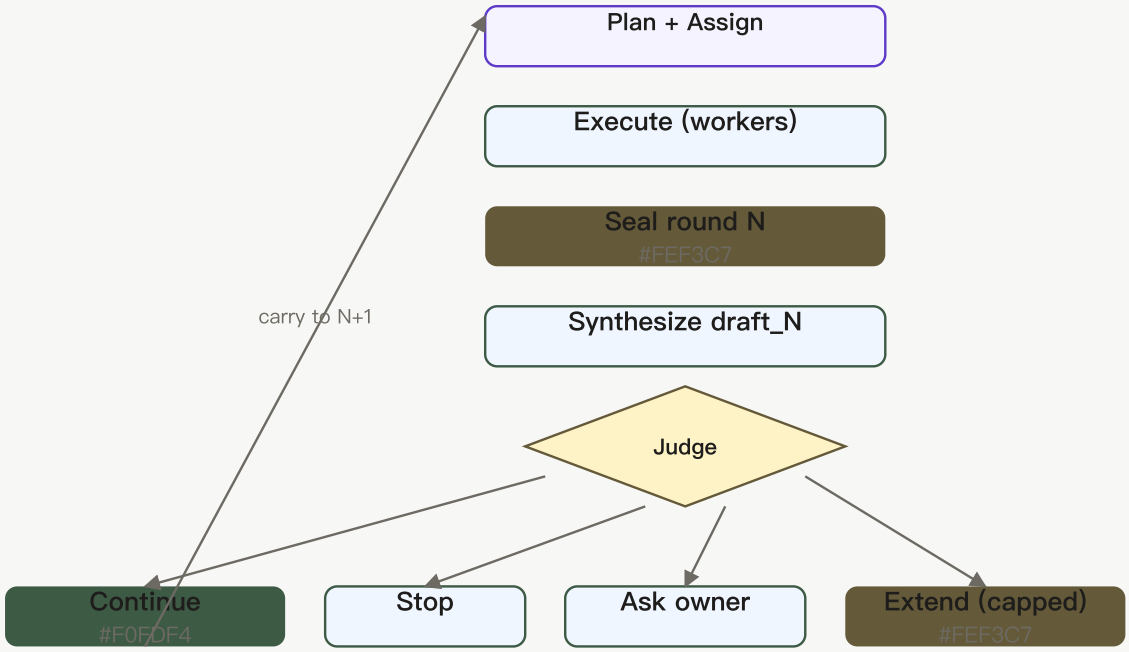


图 10 —— 多轮迭代：一轮内执行 规划 → 执行 → 封存 → 合成草稿。评审者随后继续、停止、询问所有者或扩展（有上限）。摘要带入下一轮。

56.1 为什么协作任务可能需要另一轮

Some goals benefit from reviewing a first draft and requesting targeted follow-up. Multi-round iteration adds that loop without allowing unbounded autonomous work.

56.2 草拟、评审与重新规划

Phase 47 **seal → draft → judge → replan** closes a round, synthesizes a draft report, decides whether to continue, and optionally launches another planning pass with carried summaries.

56.3 在一轮内扩展工作

Extend within a round (Phase 47B) appends capped extra steps before seal when local heuristics say more work in the same round will help.

56.4 最大轮次与扩展上限

Defaults preserve single-round behavior unless the 所有者 or template opts in. Maximum rounds and per-round extensions are hard caps that limit cost and duration.

56.5 LLM 评审模式

LLM judge mode asks the configured model whether another round would improve the result. The decision remains bounded by maximum rounds, budget, deadline, and 策略.

56.6 始终停止模式

Always-stop ends after the current sealed round, giving predictable cost and latency.

56.7 所有者决策模式

Owner-decision mode pauses after a draft and asks the 所有者 whether to stop, continue, or extend specific work.

56.8 将摘要带入下一轮

Summaries and `iterationState` blobs carry forward so the next round does not repeat completed subtasks blindly. Remote assigner handoff preserves this state when configured.

56.9 停止原因

Stop reasons include max rounds reached, 所有者 stop, budget exhaustion, judge always-stop, or failed seal. They appear in report metadata and 审计.

56.10 审查迭代历史

Review iteration history in 协作任务 accordion: each draft round, 所有者 Continue/Accept decisions, and final publish. EnvoyGo mirrors read-only.

57. 跨家庭与跨协调代理交接

57.1 何时交接编排

Handoff is useful when another bonded home has better 工作节点 visibility or should own a delegated sub-chain. It does not transfer the original 所有者's unlimited authority.

57.2 选择远程分配器

Pick a **remote assigner** bonded 对等节点 with `chain.orchestrate` or better 工作节点 roster for delegated assignment. Trust must be direct or 策略-allowed for handoff.

57.3 委派子链

Delegate a sub-chain via `task.chain.delegate` so another 协调代理 runs a subtree under your 授权 limits, not unlimited 所有者 authority transfer.

57.4 父与子职责

The **parent** 协调代理 retains chain ownership and budget; the **child** assigner executes delegated subtasks and returns results upstream.

57.5 中继链流量

Relay chain traffic uses circuit paths for WAN 对等节点. Relays forward envelopes without interpreting payloads or running models.

57.6 保留迭代状态

Handoff payloads include Phase 47 **iteration knobs** and optional `iterationState` so the remote assigner continues multi-round jobs seamlessly.

57.7 仲裁记录

Arbitration records resolve ordering disputes between orchestrators using seq and timestamp rules when cross-home coordination conflicts arise.

57.8 失败与恢复

On handoff failure, parent 协调代理 should reclaim assignment, fail the subtree, or cancel per 策略. Check 审计 for `handoff` reject reasons.

57.9 信任要求

Handoff requires compatible trust tiers, valid 授权, and mutual reachability. Blocked or public 对等节点 cannot become remote assigners.

57.10 审计交接

Audit handoff events with sender/receiver 协调代理 对等节点 IDs, delegated chain IDs, and iteration state checksums for compliance review.

58. 合并结果与生成报告

58.1 收集工作节点结果

The 协调代理 collects `task.result` and `task.chain.partial` payloads from each awarded 工作节点 before merge or synthesis.

58.2 文本交付物

Text 交付物 store narrative 工作节点 output with attribution metadata. Suitable for summaries and research sections.

58.3 结构化交付物

Structured 交付物 hold JSON or typed records (tables, extracted fields). Validators check shape on receipt.

58.4 文件交付物

File 交付物 reference 保险箱 items or chunked content by ID. Workers do not push raw filesystem paths across the mesh.

58.5 复合交付物

A 复合交付物 bundles attributed 工作节点 contributions and an aggregation method. It preserves provenance that would be lost if all text were flattened into one anonymous answer.

58.6 加权贡献

Weighted contributions let synthesis emphasize higher-confidence or 所有者-prioritized 工作节点 sections in the composite report.

58.7 合并策略

Merge strategies (concatenate, summarize, vote, template-driven) are chosen per report type. Phase 47 draft rounds may use lighter merge before final publish.

58.8 工作节点归属与来源

Reports preserve **工作节点 attribution** and provenance so readers know which 对等节点 produced each section—critical for accountability.

58.9 合成最终报告

Synthesize the final report after all required subtasks complete or partial 策略 allows best-effort merge. Only one 终端 publish per iteration round.

58.10 置顶与导出报告

Pin important reports in 协作任务 for quick access; **export** when CSV or 文件 export is enabled in your build.

58.11 所有者审查

Owner review accepts draft iterations (Phase 47), rejects unsafe content, or requests another round before final publish.

59. 协作任务配方与默认值

59.1 保存可复用任务模板

Save **job templates** with default budgets, award mode, stall/rebalance/iteration 策略, and 敏感度. Templates are local to your 节点—not a marketplace.

59.2 选择授予默认值

Choose **direct assign vs competitive bidding** default in **Settings → AI → 协作任务 defaults**. Most personal teams should stay on direct assign.

59.3 配置停滞策略

Configure **stall 策略** (timeouts, auto-rebid, notify 所有者) per template or globally. Aggressive timeouts reduce cost but increase reassignment churn.

59.4 配置再平衡策略

Set **rebalance 策略** to manual, automatic, or never. Match your appetite for autonomous budget shifts mid-job.

59.5 配置迭代默认值

Phase 47 **iteration defaults** (`iterationMaxRounds`, judge mode, extend caps) live in defaults or templates. Default `iterationMaxRounds=1` preserves single-round behavior.

59.6 配置成本可见性

Cost visibility toggles whether 工作节点 and 所有者 see bid amounts in UI during competitive mode. Hidden cost UI does not remove ledger tracking.

59.7 使用已保存的配方

Start from a **saved recipe** to pre-fill 策略 and award mode. Edit goal and preview before committing spend.

59.8 更新或移除配方

Update recipes when your team workflow changes; delete obsolete templates to avoid accidental use of stale 策略.

59.9 模板市场 —— 暂缓

Parked. Saved recipes are local product features; a mesh-wide marketplace for exchanging templates has no committed release.

60. EnvoyGo 上的协作任务

60.1 View active 协作任务

EnvoyGo **协作任务** tab lists active jobs mirrored from the home 节点 over JSON-RPC. Status updates when the phone is connected; offline viewing may lag.

60.2 查看近期任务

Recent jobs shows completed or failed chains with timestamps. Use it to reopen reports on mobile without starting new work.

60.3 打开任务详情

Tap a job for detail: lifecycle state, subtask summary, iteration progress line, and links to published reports. Controls that change orchestration are hidden.

60.4 阅读报告与交付物

Read **reports and 交付物** inline when synced. Large 文件 交付物 may require opening on desktop if not cached on the phone.

60.5 了解只读移动行为

EnvoyGo presents a read-only mirror of active and recent 协作任务. Start, award, rebalance, and orchestration controls remain on the home/desktop experience.

60.6 回到桌面进行编排控制

For start, cancel, rebalance, bid accept, or iteration Continue/Accept, switch to **desktop Social** on the home 节点. EnvoyGo intentionally omits these mutating RPCs.

60.7 移动通知

Mobile 通知 (when enabled) alert for job completion or 所有者–审批 waits. Tapping opens read–only detail; act on 审批 from desktop.

60.8 排查 EnvoyGo 移动镜像

If the mirror is empty: confirm EnvoyGo pairing, home 节点 reachability, and that jobs were started on desktop. Reload after WebSocket reconnect.

61. 智能体网络信任与安全

61.1 验证工作节点身份

Verify 工作节点 **对等节点 ID** and card signatures match envelopes on every `task.chain.*` 消息. Reject mismatched 密钥 or expired 授权.

61.2 验证所有者授权

Confirm the 工作节点's **所有者 授权** authorizes the chain 能力 and 敏感度 requested. Owner DID on card must match bonded 联系人.

61.3 绑定策略与能力门

Bond 策略 and 能力 gates run before 协调代理 logic. Blocked intents never reach 工作节点 execution even if UI allowed planning.

61.4 授权限制

Every 工作节点 request is bounded by a signed 授权 that specifies objective, actions, 对等节点 scope, 敏感度, cost, expiry, and 审批 requirements. A 工作节点 should reject work outside those bounds.

61.5 数据敏感度边界

Subtasks declare 敏感度; 工作节点 downgrade or reject over–limit data per 保险箱 策略. Do not exfiltrate friends–tier content to public–tier 对等节点.

61.6 成本与截止时间限制

Mandate **cost and deadline** limits apply on both 协调代理 and 工作节点 节点. Task runtime guards cancel expired or over–budget work.

61.7 审批要求

Actions listed in `requiresApprovalFor` pause until 所有者 allow. External 智能体 observing chains cannot bypass 审批 queues.

61.8 运行时任务守卫

Task runtime guards enforce cancellation, collect–N termination, and 授权 expiry on 工作节点 mid–flight.

61.9 保险箱与模型隔离

Workers run models and 保险箱 access locally under Diplomat → Bond → Brain → 保险箱 isolation. Remote orchestrators never receive raw 保险箱 filesystem paths.

61.10 阻止与吊销工作节点

Block trust to stop all collaboration with a 对等节点. **Revoke** 智能体 授权 on your 节点 if your 工作节点 should reject new inbound chain proposals.

61.11 应对恶意或错误配置的智能体

For misconfigured 智能体, disable Join, rotate 密钥, block 对等节点, and cancel active chains. Collect 审计 evidence before re-enabling.

61.12 审查端到端审计轨迹

Stitch **审计 JSONL** by `chainId` and `correlationId` across 协调代理 and 工作节点 节点 (each side logs its view). No central server holds the trail.

62. 智能体网络连通性

62.1 局域网发现

mDNS / LAN discovery helps find 对等节点 on the same 网络 for bonding and lower-latency paths. It does not replace 绑定 establishment for 协作任务.

62.2 直接对等连接

Direct TCP/QUIC connections are preferred when dial hints show reachable private addresses. Same-LAN 工作节点 score higher in Assigner.

62.3 中继辅助连接

Relay-assisted circuit paths connect WAN 对等节点 when NAT blocks direct dial. Relays do not terminate TLS for chain payloads beyond transport 中继.

62.4 Agent Card 同步

Agent cards sync over 绑定-triggered fetch and 能力 index updates. Stale sync manifests as missing 工作节点 until refresh.

62.5 能力发现

Capability discovery queries the index built from bonded 对等节点' cards. Only opted-in 工作节点 with matching tags appear.

62.6 离线工作节点

Offline 工作节点 fail probes and heartbeats; 协调代理 marks subtasks stalled and may reassign per 策略.

62.7 重连与重试

libp2p reconnects automatically when 对等节点 return. Retry discovery after 网络 changes; use 中继 bootstrap if direct paths fail.

62.8 多中继协调

Multi-中继 setups use community or private bootstrap 对等节点 for DHT and circuit 中继. Override `TEST_RELAY_ADDR` only in tests—operators configure bootstrap in 节点 settings.

62.9 NAT 与防火墙考量

Open firewall ports for outbound mesh traffic; inbound direct dial may require port mapping or 中继 fallback. 协作任务 reliability improves with working circuit reservations.

62.10 诊断工作节点可达性

Run 对等节点 **probe** from 智能体网络 settings, inspect dial hints, and verify 中继 reservation logs. Compare LAN vs WAN paths when 工作节点 are reachable but slow.

63. 智能体网络故障排查

63.1 加入开关不生效

If Join toggle does not stick, restart 节点, check `capabilityProviderEnabled` in config, and confirm no fleet script overwrote settings. Re-enable and refresh 工作节点 on a 对等节点.

63.2 工作节点不可见

Invisible 工作节点: verify 绑定 tier, remote Join enabled, 能力 tag present, and click **Refresh 工作节点**. Public-tier 绑定 do not auto-fetch cards.

63.3 Agent Card 过期或缺失

Force card refresh by re-bonding or manual fetch; cards older than cache TTL may hide new 能力. Check 审计 for `agent.card.response` errors.

63.4 无合格工作节点

Fix **no eligible 工作节点** by bonding 联系人, having them Join, aligning 能力 tags with plan, and refreshing. UI should block start rather than run solo multi-智能体 fiction.

63.5 无法创建计划

Plan cannot be created when LLM planner fails, 能力 mismatch, or depth/budget constraints violate 授权. Simplify goal or add 工作节点.

63.6 竞标或协商未完成

Stuck **bids**: check competitive mode timeouts, 工作节点 Join status, and 绑定策略. Counter-bid loops exhaust when max negotiation rounds reached.

63.7 任务停滞

Stalled jobs: inspect heartbeats, stall 策略, 工作节点 offline state, and 所有者-审批 waits. Manual rebalance or cancel may unblock.

63.8 工作节点未返回结果

Empty **results** often mean 工作节点 rejected 授权, hit 敏感度 wall, or crashed locally. Worker-side 审计 shows deny vs fail reason.

63.9 无法打开交付物

Artifact open failures: verify 保险箱 path on 协调代理 节点, 敏感度 审批, and that 文件交付物 IDs still exist. Re-sync 资料库 if chunked content missing.

63.10 预算或审批阻塞任务

Budget or **审批 blocks** show as `waiting_for_owner`. Resolve 审批 or raise 授权 limits on desktop, then resume.

63.11 交接失败

Handoff fails when remote assigner unreachable, trust insufficient, or iteration state rejected. Parent should fail over or cancel subtree; check `task.chain.handoff` 审计.

63.12 报告不完整

Partial reports may publish under best-effort termination 策略 when some subtasks fail. Review attribution to see which sections are missing.

63.13 收集诊断

Collect **diagnostics**: chain ID, 审计 excerpts, 工作节点 roster snapshot, 绑定 tiers, card timestamps, and 网络 probe results from both 协调代理 and 工作节点 节点.

第 VIII 部分 —— 任务、授权与交付物

64. 任务基础

64.1 什么是 EnvoyMesh 任务

An EnvoyMesh 任务 is a signed, 策略-checked request between 智能体 with an objective, requested result, constraints, lifecycle, and attributable 交付物. It is narrower than a 协作任务, which coordinates several subtasks.

64.2 任务目标与请求结果

State a clear **objective** and **requestedResult** so 工作节点 can judge fit before accepting. Vague goals cause unnecessary `task.negotiate` rounds or early `task.reject`; include 敏感度 hints when 保险箱 content is involved.

64.3 创建任务

On mesh, 智能体 open a 任务 with `task.mandate` then `task.propose`. Over A2A, `message/send` 触发器 the production executor: Bonds gate → home-所有者-signed 授权 → `task.propose` → `handleDaemonTaskInbound` (runtime guard + journal).

64.4 提议与协商

`task.propose` offers concrete work under an accepted 授权; `task.negotiate` adjusts terms. Both are signed 智能体 envelopes—the daemon inbound handler validates 授权 bounds before advancing lifecycle state.

64.5 接受或拒绝工作

Workers reply with `task.accept` or `task.reject`. Acceptance requires 绑定 tier and 授权 ceilings still satisfied; rejection should carry a reason auditors can correlate via `correlationId`.

64.6 跟踪任务状态

Track progress in Social, 审计 JSONL, or A2A `tasks/get` when bridged. Map internal twelve-state lifecycle to nine A2A states when presenting status to external clients.

64.7 心跳 and partial results

Emit `task.heartbeat` during long runs so orchestrators do not stall waiting. Partial `task.result` payloads record interim 交付物 while 授权 `collectCompletedResults` and expiry rules remain enforced.

64.8 已完成与失败的任务

终端 success requires a signed `task.result` in `completed` state; failures land in `failed` with an auditable reason. A2A pollers see mapped `completed` / `failed` after `tasks/get`.

64.9 取消任务

Send native `task.cancel` on mesh or `tasks/cancel` over A2A JSON-RPC. Tokens are 所有者-scoped—a bearer mapped to one 所有者 cannot cancel another 所有者's tracked 任务.

64.10 任务反馈

Attach post-completion feedback to the 任务 record for operator review. Feedback does not widen 授权 authority or alter 交付物 内容哈希 already published.

65. 任务生命周期

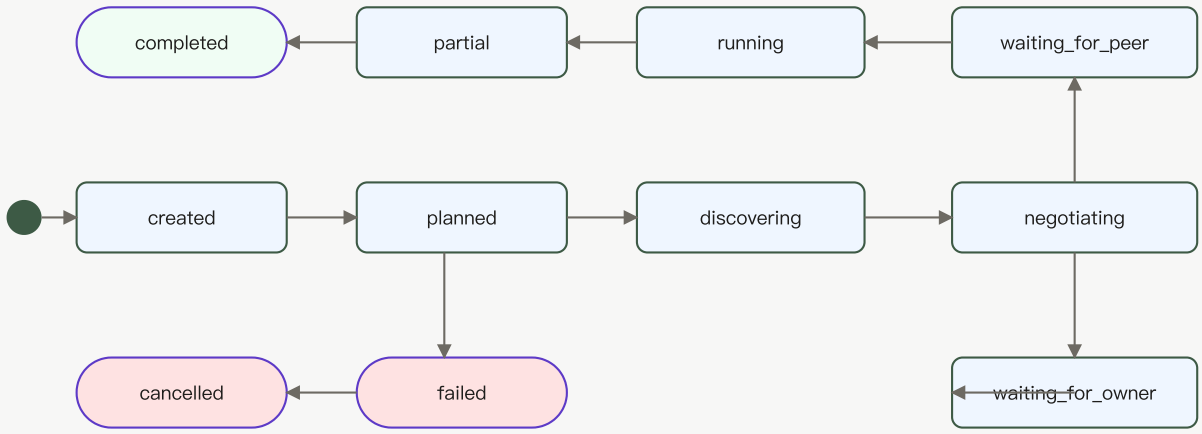


图 4 —— 任务生命周期：12 个状态与三个终态（completed 绿色、failed/cancelled 红色）。箭头表示合法转换；协商可分支到等待状态；部分结果可继续到完成。

65.1 Created

Created means the 任务 record exists but planning or 对等节点 interaction has not begun. It is a lifecycle state, not a statement that the product feature is merely planned.

65.2 Task planned

Task planned means the 节点 has derived an execution approach and can proceed to discovery or proposal. The source schema names this state `planned`.

65.3 Discovering

The 协调代理 scans bonded 对等节点 and 能力-index entries within 授权 联系人 scope. Discovery stalls when no 工作节点 meets 绑定 tier (`direct` / `referred`) or 敏感度 requirements—refresh 智能体 cards before blaming mesh outage.

65.4 Negotiating

Active `task.negotiate` exchanges adjust deliverables, cost, or 敏感度. Either party rejects when a counter-offer exceeds 授权 `maxCost`, `maxSensitivity`, or disallowed actions.

65.5 Waiting for a peer

State `waiting_for_peer` means no remote 智能体 has accepted yet. Verify remote Join toggles, 绑定 tier, and dial hints; time out and reassign per 协调代理 策略 if heartbeats stop.

65.6 Waiting for the owner

`waiting_for_owner` follows `requiresApprovalFor` hits or 绑定策略 that demands 所有者 consent. Clear the Social 审批队列 on the home 节点—A2A clients may see `input-required` until then.

65.7 Running

Models, 保险箱 reads, and 工具 execute under Brain/保险箱 isolation on the 工作节点. The A2A bridge defaults to leaving 任务 `running` until a real mesh `task.result` arrives (unless `autoCompleteLocal` is enabled for smoke).

65.8 Partial

Partial state records one or more 交付物 while work continues. Mandate `closeOnFirstCompletedResult` may terminate the 任务 as soon as the first acceptable 交付物 lands.

65.9 Synthesizing

Team and multi-工作节点 flows merge child 交付物 into a composite result. Weighted child references preserve 工作节点 lineage through the synthesis step.

65.10 Completed

Completed is 终端: the requested work ended successfully and the available result and 交付物 were recorded.

65.11 Failed

Failed is 终端: execution ended without a successful result. Preserve the reason and 审计轨迹 before retrying.

65.12 Cancelled

Cancelled is 终端 after an 所有者, 设备, 对等节点, or 策略 path stops the 任务. A2A spells the mapped external state `canceled` with one “l”.

66. 授权与委派权限

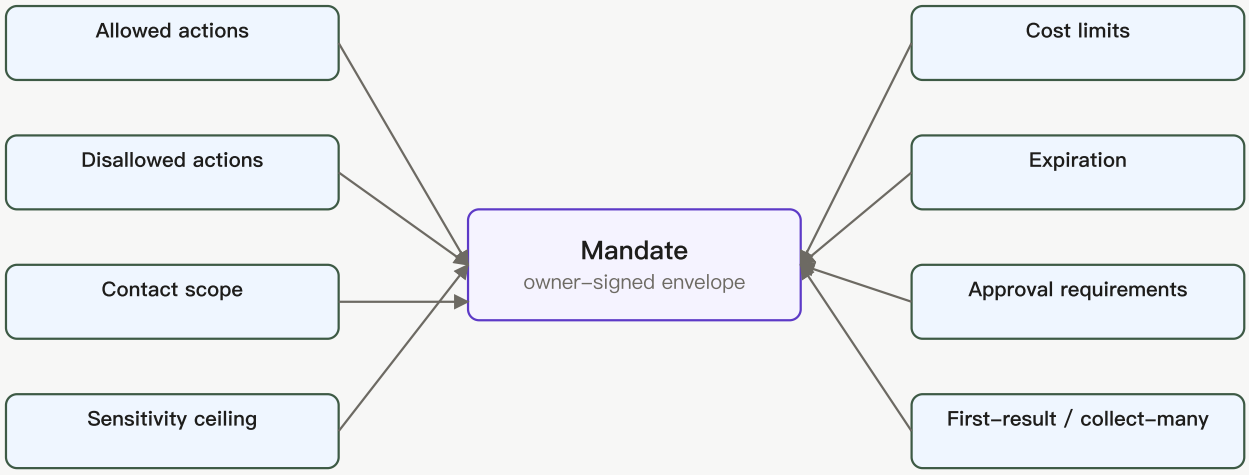


图 15 —— 授权剖析：八个正交维度限制智能体可做什么。所有者签名信封；每个维度独立可执行。

66.1 为什么智能体需要授权

A 授权 prevents an 智能体 from interpreting a broad goal as unlimited authority. It defines a verifiable envelope within which planning, 工具 use, 对等节点 联系人, and spending are allowed.

66.2 谁签发授权

Mandates are always home-所有者 signed. External A2A bearer tokens identify the 所有者 context but do not sign 授权—the production executor uses the 所有者's 密钥. Agents act under 所有者-delegated credentials verified via 授权 signature.

66.3 允许与禁止的动作

`allowedActions` and `disallowedActions` constrain which intents and 工具 may run. The 任务 runtime guard denies transitions that would invoke a disallowed action even mid-execution.

66.4 联系人与对等节点范围

Mandates may restrict participating 对等节点 IDs or 联系人 lists. Bonds tiers `self`, `direct`, and `referred` further limit who may receive `task.propose`—strangers cannot accept delegated work.

66.5 数据敏感度上限

`maxSensitivity` caps 保险箱 and 知识 exposure for the whole 任务. Workers must not return 交付物 above the 授权 ceiling even when local 绑定策略 would normally allow higher 敏感度.

66.6 成本限制

`maxCost` bounds authorized spend. Exceeding it stops execution unless the 所有者 issues a new 授权 or approves an extension through the 审批队列.

66.7 过期

`expiresAt` is enforced by the 任务 runtime guard on every inbound intent. Post-expiry proposals, heartbeats, and results are rejected before model or 保险箱 access.

66.8 首结果与多结果策略

Set `closeOnFirstCompletedResult` to stop after the first successful 工作节点; use `collectCompletedResults` when fan-out jobs need N completions before synthesis.

66.9 审批要求

List sensitive actions in `requiresApprovalFor` to pause until 所有者 allow in Social. Bridged A2A callers stall in `waiting_for_owner` or see `input-required` until 审批 clears.

66.10 智能体专属授权

Bind 授权 to `envoy:agent:<hash>` via 所有者-signed 智能体 credentials. Remote 对等节点 verify the 智能体 is authorized by the stated 所有者 before accepting proposals.

66.11 意图证明

Optional signed proof-of-intent documents why the 智能体 initiated work. Use for 审计 trails—it does not bypass Bonds checks or replace 授权 signatures.

66.12 吊销或取消权限

Owners revoke 授权 or send `task.cancel` to halt further work. Revocation prevents new proposals under the same 授权 ID; completed 交付物 and 审计 history remain intact.

67. 交付物与结果

67.1 文本交付物

A 文本交付物 contains human-readable output and may include a media type. Use it for summaries, explanations, and reports that do not require a structured schema.

67.2 文件交付物

A 文件交付物 refers to a 保险箱 path and 内容哈希, with optional name, media type, and size. Recipients should verify the hash before trusting downloaded bytes.

67.3 结构化交付物

A 结构化交付物 carries a schema reference and object data. It is suitable for machine-readable results, tables, records, and interoperability payloads.

67.4 复合交付物

A 复合交付物 contains weighted, attributed child 交付物 and an aggregation strategy. 协作任务 use it to retain 工作节点 lineage through merging.

67.5 内容哈希

File and structured 交付物 carry sha256 hashes. Verify hash before trusting downloaded bytes—especially when fetching via authenticated `GET /vault/<path>?hash=…` on the home bridge or 中继 proxy.

67.6 显示名称与媒体类型

Set `displayName` and `mediaType` for UI rendering and A2A Part translation. These labels aid presentation; they never substitute for hash verification on 文件 content.

67.7 工作节点来源

Artifacts record producing 智能体 对等节点 IDs. Composite merges retain weighted child references so 协作任务 attribution survives synthesis.

67.8 将结果存入保险箱

File 交付物 reference 保险箱 paths on the executing 节点. Path-safety and 敏感度 checks run before write; bridged File Parts expose gateway URIs instead of raw filesystem paths.

67.9 共享结果

Publish 交付物 IDs inside signed `task.result` envelopes or share within bonded `knowledge.query` bounds. Do not hand cross-tier 对等节点 direct 保险箱 paths outside 授权 敏感度.

67.10 校验结果

Check 授权 ID, 交付物 hashes, result-envelope signature, and 绑定 tier at delivery time. Re-fetch 文件 bytes through 保险箱 HTTP with matching `?hash=` before acting on content.

67.11 MCP 内容映射

Phase 48 maps MCP TextContent, ImageContent, AudioContent, resource_link, and structuredContent into EnvoyMesh 交付物 via `mesh.mcp.call_tool`. The MCP server adapter reverses the mapping when external clients 通话 `mesh.*` 工具.

67.12 A2A Part 映射

Text, Data, and File Parts translate through `a2a-artifact-map.ts` into native 交付物 kinds. File Parts advertise `<gateway>/vault/<encodedPath>?hash=…` URLs served from the home bridge (中继 forwards via home-tunnel).

第 IX 部分 —— MCP 与 A2A 互操作

68. 互操作概述

68.1 原生 EnvoyMesh 通信

Native EnvoyMesh communication uses signed envelopes, 所有者 and 智能体 身份, 绑定策略, and typed intents. It remains the preferred path between EnvoyMesh 节点.

68.2 为什么需要桥接

Claude Desktop, Cursor, and A2A SDKs speak MCP or JSON-RPC—not libp2p envelopes. Opt-in bridge 端点 translate external 通话 into signed 授权 and 工具-registry invocations without handing clients a raw mesh socket.

68.3 用于工具的 MCP

MCP target support focuses on the 2025-06-18 工具 interfaces: stdio or Streamable HTTP with `tools/list` and `tools/call`. Resources, prompts, and OAuth are future scope.

68.4 用于智能体发现与任务的 A2A

A2A target support follows v1.0.0 concepts for Agent Card, unified Parts, 任务 methods, polling, and streaming. EnvoyMesh maps these external 通话 into its signed 任务 system.

68.5 信任边界

Bridges sit above Diplomat: authenticate callers, enforce size limits, then delegate to Bonds and 授权. Bridge tokens must not exceed the mapped 所有者身份's intended authority.

68.6 认证

MCP server adapter: `ENVOYMESH_BRIDGE_SECRET` or `--bridge-token` matching `bridge.secret` on the 节点. A2A JSON-RPC: `Authorization: Bearer` from `a2aBridge.bearerTokens[]` (中继: `ENVOYMESH_A2A_BEARER_TOKENS` as `token:envoy:owner:…`). Missing auth 失败即关闭.

68.7 审计

Bridge invocations emit 审计事件 (`auditTag: "mcp-server"`, A2A method names). Correlate external request IDs with internal 任务 IDs in JSONL when debugging cross-boundary flows.

68.8 当前兼容范围

Phase 48 shipped: MCP consumer (`mesh.mcp.*` + `mcpConsumers` config), MCP server (`npx envoymesh mcp-server`), Agent Card at 中继 `/.well-known/agent-card.json`, JSON-RPC `message/send|stream`, `tasks/get|cancel`, and 保险箱 FileArtifact `GET /vault`. OAuth, MCP resources/prompts, and anonymous A2A remain future scope.

69. 使用外部 MCP 服务器

69.1 MCP 消费者模式做什么

Lets the home 智能体 通话 external MCP servers through `mesh.mcp.list_tools` and `mesh.mcp.call_tool`, backed by `@modelcontextprotocol/sdk` and entries in `node-config.json` → `mcpConsumers`.

69.2 添加 MCP 服务器

Add to `mcpConsumers: [{ name, transport, command?, url?, bearerToken?, allowRemoteHttp?, env? }]`, reload config, then run `mesh.mcp.list_tools` with the consumer `name` to confirm the 会话 starts.

69.3 Stdio 传输

Stdio launches a configured local process and exchanges MCP 消息 over standard input and output. Treat the command as executable code: use only trusted binaries and fixed arguments.

69.4 流式 HTTP 传输

Streamable HTTP connects to an MCP 端点. EnvoyMesh defaults to safe local or HTTPS destinations and requires an explicit override for remote plain HTTP.

69.5 列出外部工具

Call `mesh.mcp.list_tools` naming the configured consumer. Returns MCP 工具 schemas for 智能体 planning; empty or error responses usually mean process exit, bad URL, or bearer mismatch.

69.6 调用外部工具

Invoke `mesh.mcp.call_tool` with 工具 name and JSON arguments. MCP content blocks map into EnvoyMesh 交付物 suitable for 任务 results and 审计.

69.7 内容与交付物映射

Text, image, audio, resource links, and structured MCP content become typed 交付物. Inspect mapped output before publishing to bonded 对等节点 above public 敏感度.

69.8 超时与响应限制

Consumer 会话 honor SDK timeouts and 节点 payload size caps. Oversized MCP responses are rejected before entering the 语义防火墙 or 保险箱.

69.9 远程 URL 安全

Remote URL validation reduces SSRF risk: prefer HTTPS, avoid private metadata services, keep loopback as the default, and enable remote plain HTTP only for a controlled development 网络.

69.10 排查 MCP 消费者

Verify `command` vs `url`, stdio vs Streamable HTTP transport, `allowRemoteHttp` for dev plain-HTTP 端点, and `bearerToken`. Audit JSONL distinguishes connection failures from schema validation errors.

70. 将 EnvoyMesh 作为 MCP 服务器

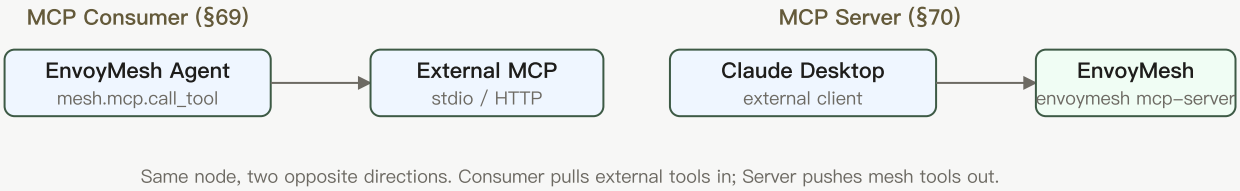


图 9 —— MCP 消费者 vs 服务器：同一个 EnvoyMesh 节点可消费外部 MCP 工具（方向 A），或向 Claude Desktop 等 MCP 客户端暴露 mesh 工具（方向 B）。数据方向相反。

70.1 MCP 服务器模式暴露什么

The stdio adapter answers MCP JSON-RPC (`initialize`, `tools/list`, `tools/call`) and forwards to the home bridge HTTP listener (default `http://127.0.0.1:3031`), exposing registered `mesh.*` 工具.

70.2 启动 `envoymesh mcp-server`

Start the adapter through the EnvoyMesh CLI `mcp-server` command (for example, the packaged or workspace CLI invocation documented for your release). It communicates by stdio and forwards 通话 to the configured local bridge.

70.3 连接 Claude Desktop

Edit `~/Library/Application Support/Claude/claude_desktop_config.json` (macOS) or `%APPDATA%\Claude\claude_desktop_config.json` (Windows):

```
"mcpServers": {
  "envoymesh": {
    "command": "npx",
    "args": ["envoymesh", "mcp-server"],
    "env": { "ENVOYMESH_BRIDGE_SECRET": "YOUR_SECRET" }
  }
}
```

Restart Claude Desktop; confirm **envoymesh** appears under MCP servers. Match `ENVOYMESH_BRIDGE_SECRET` to the 节点's `bridge.secret`.

70.4 列出 EnvoyMesh 工具

Ask Claude or Cursor to list MCP 工具—you should see `mesh.*` entries from the home 工具 registry. An empty list usually means the bridge listener is down or the bridge secret mismatches.

70.5 调用 mesh 工具

MCP `tools/call` reaches the bridge; the 节点 runs Bonds checks and 工具 handlers locally. Start with read-only 工具 (联系人, ping) before invoking 保险箱 or spend actions.

70.6 桥接认证

Set `bridge.secret` on the 节点 and the same value in `ENVOYMESH_BRIDGE_SECRET` or pass `--bridge-token YOUR_SECRET` to the adapter. Misaligned secrets return 401 before any 工具 runs.

70.7 本地与远程桥接 URL

Default: `npx envoymesh mcp-server --bridge http://127.0.0.1:3031`. For LAN hosts add `--bridge-allow-remote` and point `--bridge` at the 节点's bridge URL—avoid plain HTTP with live secrets on untrusted 网络.

70.8 错误处理与审计标签

Adapter failures surface as MCP 工具 errors; successful 通话 log `auditTag: "mcp-server"` on the 节点. Distinguish bridge 401 (auth) from 工具 deny (绑定/授权) in 审计 summaries.

70.9 当前仅工具范围

Current server scope exposes 工具. MCP resources and prompts are not automatically translated into the 保险箱 or 资料库.

70.10 OAuth 与 MCP 资源 —— 未来工作

Future. Bearer authentication is current; OAuth 2.1 and broader MCP resources/prompts support are deferred until required by a deployment.

70.11 排查 MCP 服务器

Run manually: `npx envoymesh mcp-server --bridge http://127.0.0.1:3031`. Confirm the 节点 bridge listener is up, secrets match, and 工具 are enabled in 节点 config. See `docs/phase-48-interop-smoke.md` for the full checklist.

71. A2A Agent Card

71.1 什么是 Agent Card

A2A Agent Card JSON describes name, skills, 能力, 安全 schemes, and the JSON-RPC interface URL. EnvoyMesh translates native 智能体 cards through `toA2AAgentCard()` before 中继 publication.

71.2 发现 well-known Agent Card

An A2A client fetches `/.well-known/agent-card.json` from the configured 中继 HTTP origin. Publication is opt-in through A2A bridge settings.

71.3 身份与提供商字段

Fields derive from EnvoyMesh 资料 and 智能体-网络 metadata—display name, provider URL, 所有者-linked hints. The 中继 may attach optional Ed25519 signatures (`type: "envoymesh-ed25519"`) so clients can detect tampering.

71.4 技能与能力

Native 能力 map to A2A skills with strength tags from the 能力 index. Clients use skills for discovery fit—not as authorization; bearer tokens and Bonds still gate 任务 execution.

71.5 支持的接口

`supportedInterfaces[0].url` targets `/.well-known/a2a/jsonrpc` on the configured gateway. Fetch the card first, then POST JSON-RPC to that URL with the same bearer token used for 任务.

71.6 流式能力

When `capabilities.streaming: true` and metadata includes `x-envoymesh-taskBridgeStatus: "available"`, clients may 通话 `message/stream` for SSE 任务 updates instead of polling `tasks/get` alone.

71.7 已签名 Agent Card

The 中继 can sign the Agent Card with its Ed25519 control 身份 so clients can detect alteration. Consumers must still decide whether they trust that signer and 端点.

71.8 中继发布

Enable with `--a2a-bridge` / `ENVOYMESH_A2A_BRIDGE=1` and set `--a2a-gateway-url` / `ENVOYMESH_A2A_GATEWAY_URL`. The card is served at `GET /.well-known/agent-card.json` on the 中继 HTTP port (commonly `:15432`).

71.9 隐私与字段过滤

Sensitive 资料 fields may be omitted from the public card. Treat published cards as discovery metadata—任务 authorization still requires bearer tokens, Bonds tiers, and home-所有者-signed 授权.

71.10 排查名片发现

Run `curl -sS https://relay:15432/.well-known/agent-card.json | jq .` — expect HTTP 200 when the bridge is enabled, 503 when disabled. Verify the gateway URL hostname matches the TLS certificate clients use.

72. A2A 任务

72.1 A2A JSON-RPC 端点

The public 中继 exposes `POST /.well-known/a2a/jsonrpc`; the home bridge uses the loopback `/a2a/jsonrpc` path. The 中继 authenticates and forwards rather than executing the model itself.

72.2 Bearer 令牌认证

Bearer tokens map an external caller to an EnvoyMesh 所有者身份. Keep tokens unique, rotate them, and bind them to the minimum intended trust relationship.

72.3 用 `message/send` 发送任务

`message/send` supplies user 消息 Parts and receives an A2A Task. The production executor applies 绑定策略, mints an 所有者-authorized 授权, and dispatches through the native 任务 runtime.

72.4 用 `message/stream` 流式更新

`message/stream` returns server-sent 任务 updates for clients that need progress without polling. Close abandoned streams and observe gateway timeouts.

72.5 用 `tasks/get` 轮询

`tasks/get` retrieves the current persisted 任务 mapping and state. Use it after a synchronous request returns working or after reconnecting.

72.6 用 `tasks/cancel` 取消

`tasks/cancel` requests native cancellation for the authenticated 所有者's 任务. Owner scoping prevents one token from controlling another 所有者's 任务.

72.7 A2A 到 EnvoyMesh 的策略门

The production executor 通话 Bonds `evaluatePolicy` for tiers `self`, `direct`, and `referred`. Bond denial returns A2A state **`auth-required`**—no home-所有者-signed 授权 is minted for blocked or public strangers.

72.8 生产任务执行

Pipeline: bearer auth → Bonds gate → home-所有者-signed `task.mandate` + `task.propose` → `handleDaemonTaskInbound` (runtime guard + journal) → persist mapping for `tasks/get` and `tasks/cancel`. Default leaves 任务 **`running`** until a mesh `task.result` arrives.

72.9 中继到家庭转发

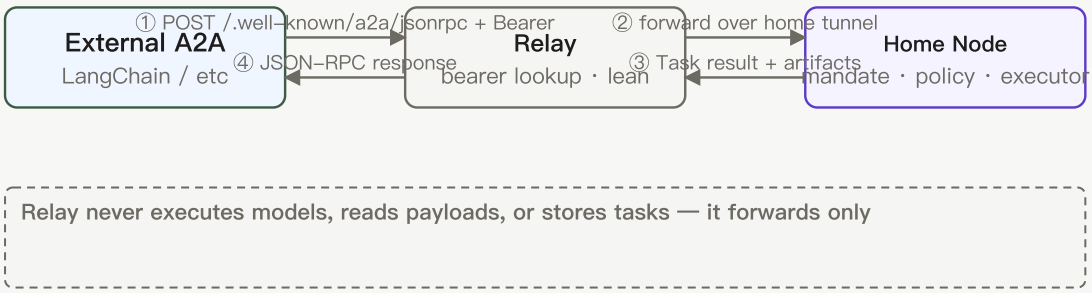


图 11 —— A2A 中继到家庭转发：中继认证 Bearer 令牌并转发到所有者的家庭节点，后者持有授权、策略、模型与任务存储。中继保持精简。

The 中继 looks up the token 所有者's home and forwards over the home tunnel. It remains lean: 策略, 授权, model execution, 任务 storage, and 交付物 stay on the home 节点.

72.10 错误码

JSON-RPC errors follow A2A conventions; Bonds denial surfaces as 任务状态 `auth-required`. Relay `forwardToHome` preserves upstream HTTP status from the home bridge when the tunnel or 节点 rejects a 通话.

72.11 排查 A2A 任务

Confirm bearer token maps to the intended 所有者, home tunnel is up for 中继 forwarding, and 审计 shows 授权/propose acceptance. Poll `tasks/get` with the returned 任务 id; use `tasks/cancel` only for that 所有者's tracked 任务.

73. A2A 状态、交付物与文件映射

73.1 EnvoyMesh 到 A2A 的任务状态

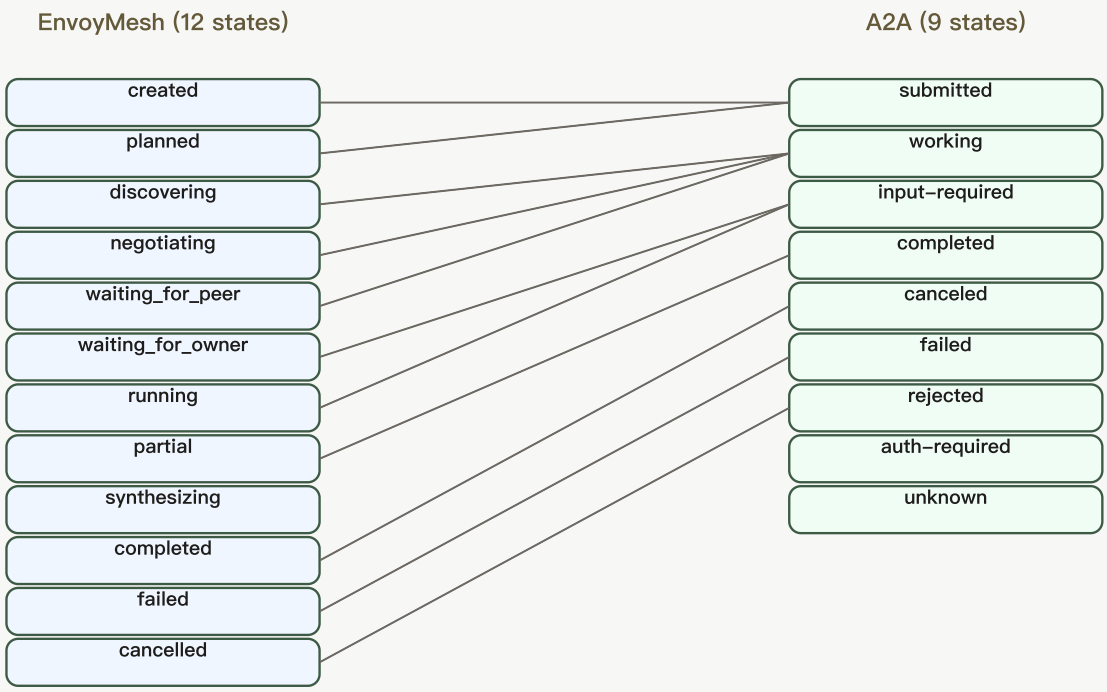


图 17 —— EnvoyMesh 到 A2A 状态映射：12 个内部状态收敛为 9 个 A2A 状态。多对一合并（如 waiting_for_peer + waiting_for_owner → input-required）由 a2a-state-map.ts 处理。

Twelve internal lifecycle states collapse to nine A2A states via `a2a-state-map.ts`. Document the mapping when building client UX that polls `tasks/get` or renders SSE events from `message/stream`.

73.2 submitted、working 与 input-required

Fresh A2A 任务 often appear **`submitted`**, then **`working`** after 授权 acceptance through `handleDaemonTaskInbound`. **`input-required`** mirrors 所有者-审批 stalls or missing parameters the executor cannot infer.

73.3 completed、failed 与 canceled

终端 A2A states align with mesh `completed`, `failed`, and `cancelled` (A2A spells **`canceled`** with one "l"). Artifacts attach on completed paths when the mapper finds native results.

73.4 rejected、auth-required 与 unknown

`rejected` follows 工作节点 `task.reject` or executor refusal; `auth-required` signals Bonds failure for the bearer-mapped 所有者; `unknown` covers untracked or expired 任务 IDs not in `a2a-bridge-tasks.json`.

73.5 文本 Part

Inbound 消息 text becomes objective context and may map to TextArtifacts in results. Outbound text 交付物 become A2A Text Parts in bridged 任务 payloads.

73.6 数据 Part

Structured JSON Parts map to structured 交付物 with schema hints. Validate schema and 敏感度 before acting on machine-readable output from external 智能体.

73.7 文件 Part

File Parts carry URIs like `<gateway>/vault/<encodedPath>?hash=…`. Fetch with the same A2A bearer used for JSON-RPC—the 中继 proxies `GET /vault/*` to the home bridge via home-tunnel.

73.8 复合结果

Composite EnvoyMesh 交付物 expand into multiple A2A Parts where the mapper supports child weights and attribution metadata.

73.9 保险箱支撑的文件 URL

File 交付物 may be represented as authenticated 保险箱-backed URLs. The 端点 validates path safety and can check the expected 内容哈希 before serving bytes.

73.10 哈希校验与访问控制

保险箱 HTTP verifies path safety, A2A bearer auth, and optional `?hash=` against sha256 (hex, base64url, or `sha256:` prefix). Hash mismatch returns 403/404 without leaking whether the path exists.

第 X 部分 —— 网络与中继

74. 点对点网络

74.1 本地与互联网连接

Nodes can discover and dial 对等节点 on a local 网络 or across the Internet. The final path depends on advertised addresses, NAT, 中继 availability, and transport compatibility.

74.2 TCP、QUIC 与 WebSocket 路径

EnvoyMesh uses libp2p over TCP and QUIC for direct 对等节点 links, and WebSocket where 中继 or NAT require HTTP-friendly transport. The Social UI and EnvoyGo usually reach the home 节点 over WebSocket when you are off-LAN. Transport choice affects reachability only; application 消息 still require signed envelopes and 绑定策略 after the link is up.

74.3 本地发现

On the same 网络, 节点 can find each other through mDNS without typing multiaddrs. Use local discovery when testing two machines on one Wi-Fi segment before adding WAN bootstrap 对等节点. Guest 网络, VPN split tunneling, or disabled multicast can block mDNS—fall back to printed multiaddrs or 中继 check-in when LAN discovery fails.

74.4 分布式发现

Across the Internet, 节点 publish and resolve rendezvous records through configured bootstrap 对等节点 and 中继 (DHT plus 中继 lookup intents). WAN discovery needs reachable bootstrap multiaddrs and a compatible discovery 资料 (for example `wan-default` in source runs). Zero bootstrap 对等节点 or an empty 中继 roster in `connectivity-status` usually indicates bootstrap or firewall misconfiguration, not a missing 身份.

74.5 直接连接

When both sides expose reachable addresses, libp2p prefers a direct dial before any 中继 hop. Direct paths reduce latency and keep 中继 operators out of the signed-envelope data plane. After every 节点 restart, copy the latest `Listening on:` multiaddr—dynamic ports invalidate saved addresses.

74.6 NAT 与防火墙行为

Home routers and corporate firewalls often block inbound TCP unless you forward a port or use circuit 中继. Allow outbound TCP from the 节点 process on both 对等节点 when diagnosing WAN connectivity. `--connectivity-strict` intentionally fails startup when all bootstrap probes fail; disable it only temporarily for diagnosis, then restore strict mode.

74.7 连接升级

libp2p negotiates identify, stream muxers, and optional 中继 reservations below the application layer. Successful transport upgrade does not grant trust—绑定策略 still applies to every intent. Enable `--p2p-debug` or 审计 `p2p.trace` rows when a 对等节点 connects but signed envelope exchange fails afterward.

74.8 已签名信封流

Application traffic travels as Ed25519-signed `EnvoyEnvelope` records on libp2p streams, not as opaque bytes trusted by IP alone. The inbound guard checks size, schema, signature, and replay before the 绑定 engine runs. A live TCP 会话 without valid signatures still produces deny or reject outcomes in 审计.

74.9 离线对等节点与重试

Peers that restart, sleep, or roam may be unreachable until 中继 registration and advertised addresses refresh. Clients retry discovery with updated multiaddrs; temporary offline status is not the same as a blocked 绑定. Confirm 中继 roster freshness and remote check-in before treating a failure as a trust problem.

74.10 网络诊断

Run `npm run cli -w @envoymesh/node -- connectivity-status --profile <path>` for bootstrap counts and 中继 hints; add `--rich` for a text snapshot. Export 审计 timelines with `--include-p2p-trace` when sharing connectivity evidence. Use the same absolute 资料 path for the 节点, CLI, and Social—a mismatched path makes diagnostics look empty even when traffic exists.

75. 中继服务

75.1 何时需要中继

Relays help when NAT, firewalls, or mobility prevent a direct libp2p dial. They provide rendezvous, lookup, optional WebSocket entry, and circuit forwarding—not account login or 消息 decryption authority. Try direct paths first; add a 中继 when 对等节点 cannot learn each other's reachable addresses.

75.2 中继能做什么与不能做什么

A 中继 helps with rendezvous, lookup, WebSocket access, and forwarding. It does not run user models, become an 身份 authority, or receive 权限 to bypass signed-envelope 策略.

75.3 选择中继

Choose a 中继 you trust for connectivity metadata: community bootstrap presets, an operator-run fleet 节点, or a private 中继 you administer. Record its bootstrap multiaddr and verify it supports the 中继 protocols your build expects (check-in, lookup, and circuit reservation on current releases). Avoid switching 中继 frequently while debugging—stale registrations confuse lookup results.

75.4 通过中继连接

Start the 节点 with `--relay` and `--bootstrap "<relay-multiaddr>"` (or an equivalent Settings entry) so it checks in and publishes a circuit address. Remote 对等节点 dial `/p2p-circuit/p2p/<your-peer-id>` when direct paths fail. Confirm both sides use compatible bootstrap lists and the same major protocol

version.

75.5 中继签到与查询

Checked-in 节点 register with `relay.checkin`; seekers resolve them through `relay.lookup` without learning private home IPs by default. Audit rows such as `relay.checkin.ok` and `relay.lookup.response` confirm healthy registration. An empty roster on the 中继 usually means clients never completed check-in or used the wrong 资料 path.

75.6 路由提示

Relays may return sibling or fleet hints so clients try alternate bootstrap paths before giving up. Hints affect where to dial next, not who may send which intent. Treat hints as optimization; 绑定策略 and signatures still gate every application 消息.

75.7 使用多个中继

Configure several bootstrap 中继 for redundancy when one host is down or geographically distant. Multi-homed clients can check in to more than one 中继 while keeping a bounded 中继 book locally. More 中继 improve reachability options; they do not merge trust stores or 身份.

75.8 使用中继时的隐私

Relays see connection metadata—对等节点 IDs, timing, and forwarding paths—not decrypted application payloads inside signed envelopes. Choose 中继 operators accordingly, especially for sensitive workflows. End-to-end intent authorization still depends on 绑定 and 授权, not on hiding traffic from your chosen 中继.

75.9 更改或移除中继

Update bootstrap multiaddrs in Settings or launch flags, restart the 节点, and verify fresh check-in before removing an old 中继 from your book. Peers caching stale circuit addresses may fail until they rediscover you. Document the change for 联系人 who pinned your old 中继-dependent multiaddr.

75.10 中继故障排查

Run `relay-status` on the 中继 资料 and `connectivity-status` on clients; compare roster totals, bootstrap counts, and recent `p2p.trace` rows. Common fixes: correct `--bootstrap` multiaddr, open outbound TCP, align 资料 paths, and recopy post-restart listen addresses. See QuickStart WAN troubleshooting and Appendix K for command references.

76. 运营中继

76.1 运维要求

Operator. Run a 中继 only if you can maintain a stable host, public reachability, 密钥 material, TLS for public HTTP/WebSocket surfaces, access controls, monitoring, upgrades, and abuse response.

76.2 安装中继

Build or deploy `apps/relay` from a current repository release on a stable host with a public TCP listener. Package installs and source runs both work; keep the 中继 version aligned with client 节点 to avoid reservation handshake skew. Document the bootstrap multiaddr you will give to fleet clients.

76.3 配置身份与监听地址

Assign the 中继 its own libp2p 密钥 material and bind to `/ip4/0.0.0.0/tcp/<port>` (or your operator standard). Print and archive the resulting `/ip4/.../tcp/.../p2p/...` multiaddr for bootstrap configuration. Separate 中继 身份 from any personal EnvoyMesh 所有者 资料 you use elsewhere.

76.4 配置公开模式

Public mode advertises an externally reachable address (`--advertise-addr` on current builds) so circuit 中继 reservations work across NAT. Without it, a 中继 may appear discovery-only—clients connect for lookup but fail reservation handshakes. Match advertised addresses to DNS or firewall rules you actually expose.

76.5 配置 WebSocket 访问

Enable the 中继 HTTP/WebSocket surface when thin clients or 浏览器 Social instances must tunnel through the 中继. Terminate TLS at the edge for production hostnames. Restrict administrative routes from the public Internet even when user WebSocket paths are open.

76.6 配置管理员访问

Protect 中继 admin APIs and metrics with operator credentials, 网络 ACLs, or mutual TLS as your deployment model requires. Never expose unauthenticated admin 端点 on public interfaces. Rotate credentials when operators leave and 审计 access changes.

76.7 发布 DNS 与 TLS 端点

Map stable DNS names to 中继 listen addresses and install valid TLS certificates for HTTPS and secure WebSocket. Clients embed these names in bootstrap presets and pairing flows. Keep certificate renewal automated—expired TLS breaks mobile and 浏览器 clients silently.

76.8 监控健康、指标、名册与日志

Track process health, 中继 roster size, lookup latency, and error rates from 中继 审计 snapshots and host metrics. Alert when roster drops unexpectedly or check-in failures spike. Correlate 中继-side traces with client `connectivity-status` during incidents.

76.9 升级与备份中继

Back up 中继 密钥 material and configuration before upgrades; schedule maintenance when client traffic is low. Roll forward one 中继 at a time in multi-中继 fleets so bootstrap lists always include a healthy 对等节点. Test circuit reservation after upgrade before decommissioning the old binary.

76.10 应对滥用

Rate-limit or block 对等节点 IDs that flood lookup, reservation, or WebSocket 端点. Preserve 审计 evidence with correlation IDs when escalating. Document your abuse 联系人 and takedown process for fleet customers—中继 carry connectivity metadata even though they do not read envelope payloads.

77. 多中继集群

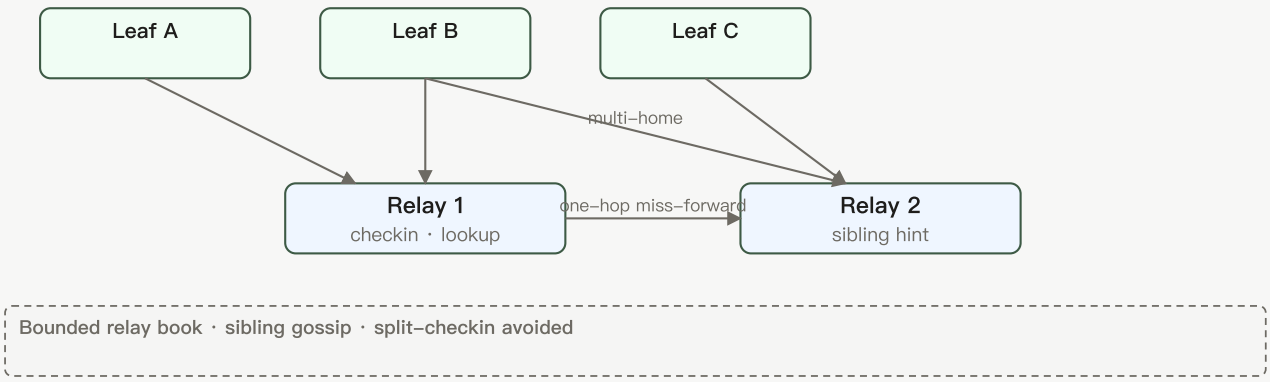


图 13 —— 多中继集群：叶子节点在多个中继间多归属；兄弟中继交换提示并一跳错失转发查询。有界中继簿防止签到分裂故障。

77.1 为什么使用多个中继

Multiple 中继 improve geographic coverage, uptime, and bootstrap redundancy. Clients can home to several bootstrap entries while keeping a bounded local 中继 book. Fleet operators standardize presets so end users are not locked to a single community 节点.

77.2 配置引导预设

Bootstrap presets bundle known-good 中继 multiaddrs (for example `public-libp2p` in source runs) so new 节点 start with WAN discovery enabled. Operators can ship private presets for enterprise fleets. Presets seed connectivity—they do not import 联系人 or trust relationships.

77.3 客户端多归属

A 节点 may check in to several 中继 and maintain multiple circuit addresses simultaneously. Multi-homing helps roaming users stay reachable when one 中继 region is degraded. Local 中继-book pruning keeps storage bounded; stale entries drop after configured freshness windows.

77.4 有界中继簿

Each 节点 stores a capped 中继 book (`relay-book.json` in the 资料 directory) rather than an unbounded global directory. Eviction 策略 favor recently verified 中继. Operators should monitor whether legitimate 中继 are aged out too aggressively in long-idle deployments.

77.5 兄弟中继提示

Sibling hints tell a lookup client about alternate 中继 in the same fleet when the primary miss occurs. They reduce failed dials during partial outages. Hints are optional optimization; clients must still complete check-in and lookup on the chosen target.

77.6 一跳查询转发

When a 中继 does not hold a registration, it may forward lookup to a sibling once rather than building a full hierarchical graph. This covers many fleet topologies today without ancestor/parent/child coordination. Deep multi-hop forwarding remains limited—see 77.10 for deferred hierarchical work.

77.7 集群健康与诊断

Compare roster counts, check-in rates, and lookup success across fleet 中继 using `relay-status` and 中继 审计 snapshots. Run live WAN validation tests from representative client 资料 after configuration changes. Standardize 资料 paths in runbooks so CLI and UI diagnostics align.

77.8 真实 WAN 校验

Prove cross-网络 paths with two 资料 on different 网络 bootstrapping to the same 中继, then exercise ping, chat, or 审计-verified intents. QuickStart's cross-网络 中继 walkthrough and `npm run poc:discovery` smoke modes are reference flows. Record correlation IDs from both sides when filing connectivity bugs.

77.9 当前协调限制

Today's multi-中继 support covers bounded books, sibling hints, and one-hop miss forwarding—not a complete hierarchical 中继 graph or global 中继 marketplace. Plan fleet layouts accordingly. Features marked **延期** in 77.10 are design targets, not hidden toggles.

77.10 完整分层中继图 —— 延期

Deferred. Current multi-中继 coordination supports bounded books, sibling hints, and one-hop miss forwarding; the complete hierarchical ancestor/parent/child graph remains future work.

第 XI 部分 —— 终端、浏览器与高级用法

78. 终端

78.1 打开终端视图

Open 终端 from Social's navigation or start a 会话 from an eligible chat thread. The view lists active PTY 会话 on the home 节点 and offers controls to attach, resize, or end them. EnvoyGo exposes the same 能力 through its 终端 screen when paired to home.

78.2 创建与管理终端会话

Create a 会话 to spawn a shell PTY on the home desktop 节点; name or tag 会话 when the UI supports it so you can find long-running work. Multiple clients can attach read/write depending on 策略. Sessions persist until closed or until the 节点 restarts—save important output elsewhere.

78.3 了解家庭端 PTY

The 终端 process runs as a PTY on the home desktop 节点. EnvoyGo and the Social UI are clients of that 会话, so commands execute with the home user's operating-system 权限.

78.4 使用终端输入与输出

Type commands in the 终端 pane; stdout and stderr stream back over the authenticated WebSocket or JSON-RPC tunnel. Large output may be truncated in mobile clients—prefer desktop Social for heavy logs. Copy/paste behavior follows your platform and 浏览器 constraints.

78.5 使用智能体辅助终端模式

When 智能体 assist is enabled, EnvoyAI may propose shell commands based on your 对话 context. Review every proposed command before execution—智能体 assist does not bypass 审批 you configured. Denied commands should appear in 审计 with an explicit outcome.

78.6 从 EnvoyGo 访问终端

EnvoyGo attaches to home-节点 PTY 会话 over the paired JSON-RPC transport; commands still execute on the desktop with its OS 权限. Keep the home 节点 awake and reachable via 中继 or LAN while using mobile 终端. Treat phone access as remote control of a powerful surface.

78.7 Security and approvals

终端 access is powerful and can alter 文件, credentials, or software. Restrict pairing, require 审批 for 智能体-suggested commands, inspect commands before execution, and close abandoned 会话.

78.8 安全关闭会话

Exit long-running programs cleanly (`exit`, `Ctrl+D`, or application-specific stop commands) before closing the 终端 tab. Abrupt disconnects may leave background jobs running on the home 节点. Revoke pairing or change 审批 if you shared a 会话 unintentionally.

78.9 Troubleshoot terminals

If attach fails, confirm the home 节点 is running, WebSocket or 中继 paths are healthy, and your 会话 token is valid. Check 审计 for auth-required or deny rows tied to 终端 RPCs. Restart the 节点 only after closing sensitive 会话 you do not want orphaned.

78.10 外部终端集成

Some releases integrate external 终端 products through the same home PTY boundary rather than granting them libp2p 密钥. Configure integrations in Settings and restrict them to trusted 网络. External 工具 inherit home-节点 OS privileges—apply the same caution as local shell access.

79. 浏览器

79.1 打开浏览器视图

Open 浏览器 from Social or EnvoyGo to browse permitted mesh content. The view resolves `envoy://` URLs through your home 节点's 策略 boundary, not the public web by default. Pairing or local 节点 availability is required before content loads on mobile.

79.2 Navigate `envoy://` content

An `envoy://` URL identifies mesh-hosted content by author and path rather than a public web server. Resolution passes through the paired or local 节点 and its trust 策略.

79.3 Browse authors and topics

Browse by author DID, published topics, or feeds your 绑定策略 exposes. Strangers may see only public-敏感度 material; bonded 联系人 may see friends-level notes when authors published them. Empty lists often mean 策略 denial, not a broken index—check 信任层级 and 敏感度 labels.

79.4 Use history and bookmarks

浏览器 history and bookmarks are stored locally in your 资料 for quick return to mesh pages you already accessed. Clearing history does not unpublish remote content. Bookmarks reference `envoy://` paths; if an author moves content, update or remove stale entries.

79.5 从浏览器发布

Publishing creates or updates mesh-visible content from notes and pages you own, subject to per-item 敏感度 toggles in 资料库. Public items become queryable via `knowledge.query` within rate limits; friends-level items require appropriate 绑定. Preview 敏感度 before publishing sensitive drafts.

79.6 订阅动态更新

Subscribe to authors or topics to receive feed updates when new mesh content appears and 策略 allows delivery. Subscriptions respect 绑定 and 敏感度 rules—dropping a 绑定 may silently stop updates. Push 通知 on EnvoyGo depend on home-节点 forwarding and platform 权限 settings.

79.7 在 EnvoyGo 上使用浏览器

EnvoyGo renders 浏览器 through the paired home 节点, mirroring desktop 策略 results on a smaller screen. Keep the home 节点 online; cached pages may be stale when offline. Read-only browsing does not substitute for 资料库 editing—create notes on desktop when possible.

79.8 Paired-mode requirements

Mobile 浏览器 requires a completed EnvoyGo pairing with a healthy home JSON-RPC 会话. Without pairing, the phone has no 保险箱, 绑定 store, or signing context to resolve `envoy://` URLs. Re-pair if 会话 tokens expire or after major home-节点 身份 changes.

79.9 Troubleshoot Browser content

When a page fails to load, verify the URL author exists, 敏感度 allows your 信任层级, and the home 节点 can reach the publishing 对等节点. Audit may show 绑定 deny or schema reject for fetches—not generic HTTP 404 semantics. Retry after 绑定 acceptance or author republish.

80. 高级设置

80.1 Node settings

Node settings cover 资料 身份, display name, discovery 资料, listen addresses, and service ports for the home runtime. Changes often require a restart to take effect. Note your 资料 directory path before editing paths or ports so CLI and Social stay aligned.

80.2 Network and bootstrap settings

Configure bootstrap multiaddrs, presets, strict connectivity mode, and advertised listen addresses here or via equivalent launch flags. Misconfigured bootstrap lists are the most common WAN failure mode. After changes, run `connectivity-status` and inspect 审计 for bootstrap probe results.

80.3 Relay settings

Enable client 中继 mode, set bootstrap 中继, and manage the local 中继 book from 中继 settings. These controls affect how others dial you—not whom you trust. Pair 中继 changes with `relay-status` on both client and 中继 operator 资料 during rollout.

80.4 AI and model settings

AI settings select providers, model routes, 语义防火墙 behavior, and EnvoyAI/OpenClaw gateway integration. API 密钥 and model credentials live in 资料 configuration—back them up securely and never paste them into support bundles. Disable remote models when air-gapped 策略 requires local-only inference.

80.5 External-agent settings

External-智能体 settings configure HomeClaw, Hermes, OpenHuman, or custom HTTP bridges, including ports, bearer secrets, and enabled presets. Bridges forward 策略-checked 工具—they do not receive raw libp2p 密钥. Rotate bridge secrets after compromise and review action history against 审计 JSONL.

80.6 Agent Network settings

智能体网络 settings control opt-in collaboration, 工作节点 visibility, 协作任务 budgets, and orchestration limits. Both sides must opt in and hold appropriate 绑定 before 智能体 collaborate. Start with manual 审批 and small 授权 before enabling automatic spend rebalance.

80.7 Knowledge and storage settings

Point the 保险箱 at `shared_vault/` (default in source runs) or a configured path; enable 资料库 plugins such as Obsidian or MCP under 知识库. Sensitivity defaults and indexing options live here. Large 保险箱 moves require re-index time and disk space on the home 节点.

80.8 Call and TURN settings

Voice 通话 settings include STUN/TURN URLs, credentials, and platform-specific push topics for EnvoyGo. Misconfigured TURN prevents 通话 across strict NAT. Video remains limited on current releases—确认功能状态 before training users on video workflows.

80.9 Notification settings

Configure push 通知 providers (APNS/FCM) on the home 节点 and 权限 prompts on EnvoyGo. Delivery depends on home-节点 forwarding, 中继 reachability, and OS battery 策略. Test with a low-noise channel before enabling alerts for every chat 消息.

80.10 Logging and diagnostics

Adjust verbosity, p2p trace capture, and diagnostic exports from logging settings or CLI flags such as `--p2p-debug`. Audit JSONL remains the authoritative allow/deny trail even when console logging is quiet. Redact secrets before sharing logs—see Appendix K.

80.11 Experimental settings

Experimental toggles gate features still receiving validation; interfaces and defaults may change between releases. Enable them only on non-production 资料 until release notes mark them **可用**. Document which toggles you enabled when reporting bugs.

80.12 Restore recommended defaults

Restore recommended defaults resets risky or nonstandard settings while preserving 身份 密钥 and 保险箱 content. Use this after connectivity experiments or failed 智能体-bridge trials before escalating support. Export a 资料 backup first if you customized many fields.

第 XII 部分 —— 隐私、信任与安全

81. 身份与密钥安全

81.1 所有者身份

The 所有者身份 is the long-lived root representing the human. It signs 设备证书, 授权, and other authorizations, so its private 密钥 deserves the strongest backup and access protection.

81.2 设备身份

Each 设备 has its own 身份 authorized by the 所有者. This lets you revoke one lost machine without changing the human's 所有者身份 everywhere.

81.3 智能体身份

An 智能体 has a distinct 密钥 and an 所有者-signed credential linking it to the 所有者. Peers can therefore verify which 所有者 authorized an 智能体 without treating the 智能体 密钥 as the 所有者 密钥.

81.4 对等身份

A 对等身份 is the runtime sender 身份 used for signed envelopes and networking. It is not interchangeable with 所有者, 设备, or 智能体身份 even when one 节点 holds several of them.

81.5 Ed25519 签名的通俗解释

Ed25519 lets a sender create a compact signature with a private 密钥 and lets others verify it with the public 密钥. Verification proves 消息 integrity and 密钥 possession, not that the human is trustworthy.

81.6 DID 呈现

DIDs (`envoy:owner:…`, `envoy:device:…`, `envoy:agent:…`) label 身份 in UI and 审计 without replacing 密钥 verification. Present DIDs alongside fingerprints when teaching someone to verify you. A matching string is not proof unless signature checks succeed.

81.7 密钥存储

Private 密钥 stay in the 节点 资料 with restrictive 文件 modes (`0o600` on sensitive JSON). EnvoyGo stores pairing secrets in OS secure storage, not 所有者 root 密钥. Never copy private 密钥 文件 into chat, email, or cloud drives without encryption.

81.8 备份与恢复

Back up 所有者 密钥 material, 设备证书, trust store, and 保险箱 together on encrypted media tested with a restore drill. Losing the only 所有者 密钥 backup may require a new 身份. Separate hot backups from offline copies to limit ransomware spread.

81.9 设备证书

Device certificates are 所有者-signed documents binding a 设备 public 密钥 to your 所有者身份. Pairing EnvoyGo or adding a laptop mints a new certificate chain. Revoke certificates promptly when hardware is lost—see Chapter 88.

81.10 吊销

Revocation records invalidate 设备证书 or 授权 without rotating the 所有者 密钥. Publish revocations from a still-trusted 设备 and 审计 that 对等节点 reject stale credentials on next handshake. Owner-密钥 compromise requires 身份 migration, not certificate revoke alone.

82. 绑定与信任策略

信任层级	含义	允许的动作	敏感度上限
blocked 阻止	全部拒绝	—	—
public 公开	陌生人	ping · 窄发现	public
referred 介绍	已介绍	知识 · 受限任务	friends
direct 直接	好友	完整协作 + 协作任务	friends · trusted

图 3 —— 绑定信任层级：每个层级限制联系人可执行的动作与最高数据敏感度。更高层级解锁更丰富的协作；阻止层级拒绝一切。

82.1 绑定的含义

A 绑定 records a local 信任层级 for another 所有者 and drives deterministic 策略. Identity answers “who signed”; the 绑定 answers “what may this relationship do.”

82.2 自身信任

Self is the local 所有者’s highest trust context and can reach private 敏感度 within local 策略.

82.3 直接信任

Direct represents a deliberately trusted 联系人 and permits the broadest remote workflows, generally up to friends-level 敏感度 unless additional 策略 restricts them.

82.4 介绍信任

Referred represents limited trust established through introduction or constrained onboarding. Knowledge and 协作任务 operations remain more restricted and may require 审批.

82.5 公开信任

Public is the stranger/default tier. Only narrow discovery, ping, introduction, and public-知识 behaviors are eligible; it is not sufficient for 协作任务 recruitment.

82.6 阻止信任

Blocked denies communication regardless of advertised 能力. Use it for abuse, compromise, or a relationship that should no longer reach the 节点.

82.7 能力门

Capabilities map intents and 工具 actions to allow, deny, challenge, or 审批 outcomes. A 联系人 may be direct yet still denied a specific 保险箱 action if 授权 or 敏感度 forbids it. Inspect 审计 `deny` rows for the missing 能力 name.

82.8 敏感度上限

Each 信任层级 caps maximum **敏感度** (`public` / `friends` / `trusted` / `private`) for 知识 and data operations. Requests above the ceiling 失败即关闭 even when the intent is otherwise allowed. Lower 敏感度 before sharing with referred 联系人.

82.9 挑战与审批

Stranger-tier or high-risk actions may return **challenge** or **审批** outcomes instead of immediate allow. Human 审批 land in the 审批队列 on the home 节点. Do not bypass 审批 by retrying the same payload repeatedly.

82.10 更改或吊销信任

Change 信任层级 in 联系人 settings or issue signed revocation for 设备 and 授权. Downgrade takes effect on the next inbound operation; already shared 文件 remain on 对等节点 节点 until they delete local copies. Document tier changes for future incident review.

83. 已签名消息与协议安全

83.1 已签名消息

Every envelope is Ed25519-signed over canonical JSON so tampering is detectable. Unsigned or wrongly signed payloads fail inbound guard before 策略 runs. Signatures prove 密钥 possession, not moral trust—pair with 绑定.

83.2 发送者与接收者角色

Roles (`human`, `agent`, `system`) are schema-enforced per intent—`chat.message` requires 人对人, 任务 intents require 智能体对智能体. Role mismatch rejects at validation. UI choices must match the intended role path.

83.3 类型化意图

Intents are typed (`chat.message`, `knowledge.query`, `task.propose`, …) with Zod-validated payloads. Unknown intents 失败即关闭. Agents and integrations must use the correct intent for the operation, not opaque blobs.

83.4 消息与关联标识

`messageId` identifies one envelope; `correlationId` stitches multi-step flows in 审计 across 对等节点. Include correlation IDs when sharing diagnostics. Replay dedup uses 消息 IDs within the inbound guard window.

83.5 模式校验

Inbound payloads pass schema validation before 绑定 evaluation. Malformed JSON or field violations return structured errors without touching 保险箱 or models. Client bugs show up as validation failures in 审计, not silent drops.

83.6 签名验证

Verification recomputes canonical JSON and checks Ed25519 signatures against the sender public 密钥, which must hash to `senderPeerId`. Failed verification denies before 策略. Never disable verification for convenience.

83.7 重放保护

The inbound guard rejects duplicate `messageId` values within a replay window to limit replay attacks. Clock skew affects ordering but not signature validity. Restarting 节点 does not reset 对等节点 replay state mid-会话.

83.8 速率与大小限制

Diplomat enforces rate and size caps on streams before expensive work. Oversized chat or 文件 payloads deny early. Burst traffic from one 对等节点 may throttle—back off rather than splitting into many tiny 消息.

83.9 格式错误消息处理

Malformed 消息 are rejected with 审计 summaries; guards do not crash the 节点 on bad input. Persistent malformed traffic from a 对等节点 is grounds for block tier. Capture one sample for diagnostics without enabling verbose payload logging in production.

83.10 协议版本管理

Protocol version fields gate incompatible 对等节点 during handshake. Mixed-version fleets should upgrade 中继 and 节点 together per release notes. Version mismatch manifests as connect failures, not partial silently broken chat.

84. 安全架构

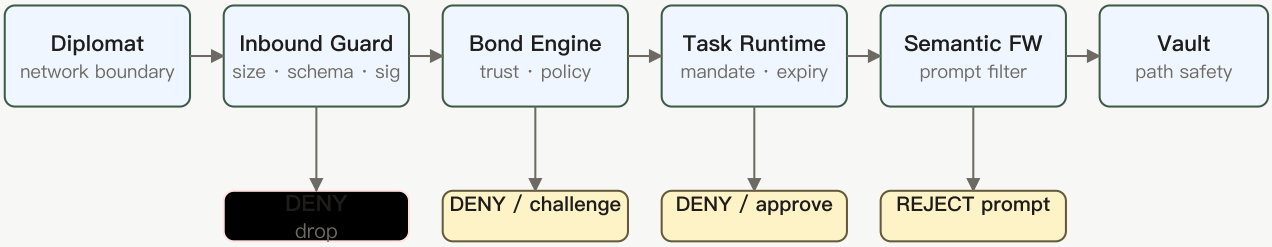


图 5 —— 安全流水线：从网络边界到保险箱的六个有序层。每层可拒绝、质询或要求审批；链条失败即关闭，且不单独信任任何一层。

84.1 网络边界

The Diplomat (网络 boundary) accepts bytes and connections but has no direct model or filesystem authority. It parses, limits, and forwards only validated requests.

84.2 入站守卫

Inbound guard checks size, schema, signature, and replay before 绑定 engine. It has no 保险箱 or model access—only accept or reject. Most user-visible "消息 failed" traces start here or at 绑定 deny.

84.3 绑定策略引擎

The 绑定引擎 turns 信任层级, intent, 能力, and 敏感度 into an allow, deny, challenge, or 审批 decision before privileged work proceeds.

84.4 任务运行时守卫

Task runtime guard enforces 授权 expiry, cancellation, collect-N termination, and action lists during 智能体 work. Even allowed 绑定 cannot exceed an expired 授权. Review 任务 journal alongside 审计 when jobs stall mid-flight.

84.5 Semantic firewall

The 语义防火墙 (part of the model boundary, historically called the Brain layer) rejects empty, oversized, or control-character-laden model prompts and normalizes excessive newline runs before a model sees them.

84.6 模型边界

The model router receives only approved context after 绑定 and 任务 guards; external 智能体 never bypass it via bridge 工具. Prompts pass the 语义防火墙 before provider 通话. Model output does not auto-execute 保险箱 writes.

84.7 保险箱隔离

The 保险箱 enforces path safety and explicit operations. Neither a remote 对等节点 nor an 外部智能体 receives unrestricted filesystem access.

84.8 路径与文件安全

保险箱 operations resolve paths against an allow-list; `../` and unsafe symlinks deny. Remote 对等节点 never get arbitrary filesystem paths—only explicit 保险箱 intents. Validate local paths when indexing new 资料库 folders.

84.9 SSRF 防护

MCP and bridge URL validation restricts unsafe remote destinations and plain HTTP defaults, reducing the chance that an integration can probe internal services.

84.10 外部智能体隔离

Bridge and MCP adapters expose curated 工具, not shell or libp2p. Tokens authenticate bridge HTTP; 授权 scope each 工具 通话. Compromise of an 外部智能体 is contained by 绑定 + 授权, not full 节点 access.

84.11 中继信任边界

A 中继 is a connectivity service, not a trusted brain. End-to-end signatures and home-节点 策略 remain necessary even when the 中继 operator is reputable.

84.12 纵深防御

Security stacks Diplomat → inbound guard → 绑定 engine → 任务 guard → 语义防火墙 → 保险箱 path checks. No single layer is sufficient—connectivity success does not imply authorization. Design integrations assuming deny-by-default at each hop.

85. 隐私控制

85.1 资料可见性

Choose which 资料 fields each 信任层级 may fetch—public bios vs friends-only 照片. Publishing overly open defaults exposes you on discovery topics. Revisit after changing trust relationships.

85.2 联系人披露

Contact cards show only what 绑定策略 and your disclosure settings permit. Introduction flows reveal minimal proof text until upgrade. Do not embed third-party phone numbers in signed proof unless intended.

85.3 知识敏感度

Index and query 知识 with 敏感度 tags; referred and public tiers cannot read private-indexed chunks. Re-tag before sharing summaries in 协作任务. Mis-tagged content is a 隐私 bug, not a crypto failure.

85.4 对话保留

Conversation history stays on your home 节点 unless you export it. Retention controls (where offered) prune local indexes, not remote 对等节点 copies of 消息 they already accepted. Align retention with backup 策略 (Chapter 89).

85.5 智能体记忆

Agent 记忆 and 会话 context live in home-节点 stores governed by 授权 and settings. Clearing 智能体 记忆 does not delete 对等节点 chat logs. Scoped 授权 limit how much history 工具 may retrieve.

85.6 保险箱共享

保险箱 sharing uses explicit intents with 敏感度 and path safety—no hidden folder sync to strangers. 协作 任务 pull only mandated 保险箱 slices. Audit 保险箱 retrieve denials when 智能体 "cannot find" a 文件.

85.7 模型提供商隐私

Cloud model providers receive prompts you send through the configured adapter—review their terms and prefer local models for sensitive topics. Semantic firewall reduces exfiltration patterns but is not a full DLP suite. Disable cloud routing for classified workflows.

85.8 中继隐私

Relays see connection metadata and encrypted/signed frames they forward—they are not trusted readers of plaintext chat. Avoid putting secrets in 中继-visible routing hints. Choose 中继 you tolerate for availability, not for confidentiality.

85.9 审计日志隐私

Audit JSONL stores structured summaries, not full 消息 bodies by default. Protect 审计 文件 like 密钥—`0o600` and encrypted backup. Redact tokens before sharing logs externally.

85.10 删除本地数据

Local delete removes threads, 保险箱 objects, or 资料 fields from **your** 节点; 对等节点 may retain copies. Use block and revoke for ongoing abuse. Secure-delete media if OS support exists before decommissioning hardware.

86. 审计与活动历史

86.1 为什么 EnvoyMesh 记录活动

Audit records make 策略 and automation reviewable. EnvoyMesh records structured summaries and correlation identifiers rather than treating 智能体 activity as an opaque model transcript.

86.2 审计事件字段

Audit events carry `eventId`, `createdAt`, `type`, `intent`, `outcome`, `summary`, optional `remotePeerId`, `correlationId`, and `latencyMs`. Learn the field glossary in Settings → Activity help. Summaries are human-readable; correlate IDs for multi-hop traces.

86.3 跨对等节点关联

Use shared `correlationId` values to follow one user action across 绑定, 中继 forward, and 工具 通话 on both sides. CLI `audit --include-p2p-trace` expands traces when debugging WAN paths. Ask 联系人 for their side's ID only over a verified channel.

86.4 策略允许与拒绝记录

Allow and deny rows prove 策略 decisions with intent names and reasons—essential for "why was this blocked?" disputes. Approvals appear as separate outcomes. Export filtered 审计 slices for support, redacted.

86.5 任务与协作任务记录

协作任务 lifecycle events append to 任务 journal and 审计 with state transitions (`discovering`, `running`, `completed`, …). Correlate job ID with chat threads that spawned the work. Failed jobs retain error summaries without raw model transcripts.

86.6 工具与审批记录

Tool invocations and human 审批队列 entries log 授权 action, 工具 name, and outcome. Denied 工具 name missing 能力. Use these rows to tune 授权 without disabling 审计.

86.7 外部智能体记录

Bridge and MCP traffic tags external–智能体身份 separately from native mesh 对等节点. Cross-check bearer auth failures vs 绑定 denials. External compromise investigations start in these rows.

86.8 网络诊断

Network diagnostic 审计 entries record 中继 reservation, dial failures, and connectivity snapshots—no 消息 plaintext. Pair with `connectivity-status` CLI during outages. Do not enable verbose libp2p logging routinely in production.

86.9 检查端到端流程

Pick one failed 消息 or job, note its correlation ID, and walk 审计 chronologically on home and 对等节点 if available. Identify whether failure was guard, 绑定, transport, or model. Stop after the first definitive deny reason—avoid random setting changes.

86.10 保留、备份与保护

Audit logs grow unbounded without operator rotation—archive JSONL to encrypted backup media. Include 审计 in disaster-recovery drills. Restrict read access to 所有者-trusted 设备.

87. 应对安全事件

87.1 丢失设备

From a trusted 设备, revoke the lost 设备, rotate any bridge or 中继 tokens it held, and review recent activity. If the lost 设备 held the only 所有者-密钥 backup, recovery may require creating a new 身份.

87.2 所有者密钥被入侵

Treat an exposed 所有者 private 密钥 as a root compromise. Disconnect affected 节点, preserve evidence, rotate dependent credentials, notify trusted 联系人, and migrate to a new 所有者身份 because signatures from the old 密钥 can no longer be trusted.

87.3 可疑联系人

Lower trust to public or block, preserve 审计 and recent threads, and verify 身份 out of band before restoring direct tier. Do not execute 文件 opens or 工具 审批 from suspicious threads. Report coordinated harassment via block and documented 审计 export.

87.4 行为异常的智能体

Pause or revoke the 智能体 授权, disable bridge tokens, and inspect 工具 审计 for unexpected 保险箱 or mesh 通话. Narrow `allowedActions` before re-enabling. Treat repeated 授权 violations as potential prompt injection or compromised integration.

87.5 被入侵的外部智能体

Rotate bridge bearer tokens, disable the 外部智能体's MCP registration, and review all 工具 通话 since last known good. External 智能体 never had libp2p—containment is token + 授权 scope. Re-enable only with fresh secrets and tighter 授权.

87.6 恶意文件或知识内容

Do not open unknown attachments; quarantine downloads outside default 保险箱 open paths. Re-index 知识 sources if malicious content was ingested. Warn 联系人 if your 节点 forwarded malware-signed-as-you due to 密钥 compromise.

87.7 中继事件

If a 中继 operator reports abuse or outage, rotate home tunnel tokens and verify Agent Card URLs still point to your 节点. Relays cannot decrypt chat but can disrupt availability—have a secondary bootstrap 中继 in config. Document incident time window for 审计 review.

87.8 吊销、阻止与暂停

Use block tier for 联系人, revoke for 设备 and 智能体, and pause 协作任务 from 智能体网络 UI. Order: stop ongoing harm (revoke/block), then investigate 审计, then restore with tighter 策略. Pausing is reversible; blocked 联系人 need deliberate unblock.

87.9 保留诊断

Copy relevant 审计 JSONL segments and correlation IDs before clearing logs or reinstalling. Remove bearer tokens, private 密钥, and recovery phrases from shared bundles. Store evidence encrypted with incident date in filename.

87.10 报告漏洞

Report 安全 defects through the project's coordinated disclosure channel listed in release notes or repository SECURITY 策略. Include reproduction steps and version—not live 密钥. Do not test exploits against production 对等节点 without 权限.

第 XIII 部分 —— 管理设备与数据

88. 设备管理

88.1 查看设备

Open Settings → Devices to list 所有者-authorized machines and EnvoyGo pairings with creation dates and last activity hints. Each entry maps to a 设备证书, not the 所有者 root 密钥. Use this view before revoking stale hardware.

88.2 添加桌面设备

Install Social/Tauri on the new computer, restore or create 设备身份 from 所有者 authorization flow, and approve the new certificate from an existing trusted 设备. Copy 资料 data via backup restore (Chapter 89) rather than hand-copying 密钥 文件 over chat.

88.3 配对 EnvoyGo

On desktop Social open Pairing → show QR; in EnvoyGo tap Pair and scan `envoy://pair?…`. Approve the pending 设备 on home if the queue prompts. Confirm chat loads through HomeRemote before retiring an old phone pairing.

88.4 了解独立的设备身份

Devices can share one 所有者身份 while retaining independent 设备 密钥 and certificates. This supports targeted revocation and 审计 attribution.

88.5 审查设备活动

Filter 审计 and 设备 list by 设备 ID to see which machine sent 消息 or invoked 工具. Unexpected 设备 IDs after travel warrant revoke. EnvoyGo actions appear attributed to the paired phone 设备, not desktop.

88.6 吊销设备

Select the 设备 → Revoke certificate; the 节点 rejects new 会话 immediately. Rotate bridge or 中继 tokens that 设备 held. Physical access after revoke still reads old local caches—encrypt disk on shared PCs.

88.7 迁移到新电脑

Take a full 资料 backup, install on the new host, restore 密钥 and 保险箱, then revoke certificates for the old PC if retiring it. Verify mesh listen addresses and update 联系人 if your public multiaddr changed. Send a test DM before decommissioning.

88.8 更换丢失的手机

Revoke lost EnvoyGo pairing from desktop first, then pair a replacement phone with a fresh QR. Assume the lost phone's pairing token is compromised if unlocked. Do not clone pairing 文件 between phones manually.

88.9 设备同步边界

EnvoyGo syncs selected NodeService views—not a full mesh replica. Desktop and mobile may show different Settings depth. Conversation state authoritative on home; mobile cache clears on re-pair.

89. 备份与恢复

89.1 备份策略

Use a layered backup: protect 所有者 and 设备 credentials separately from replaceable application binaries, and back up configuration, trust, 保险箱 content, and important records on a tested schedule.

89.2 身份密钥

Export 所有者 and 设备 private 密钥 only into encrypted backup archives; never store plaintext 密钥 in cloud sync folders. Test import on an isolated machine yearly. Loss of 所有者 密钥 without backup is 身份 loss.

89.3 配置

Back up `node-config`, 中继 tokens, model provider settings, and bridge secrets with secrets redacted in secondary copies. Version-control non-secret config templates separately. Restore config before starting 节点 after OS reinstall.

89.4 联系人与信任

Include `trust-records.json` and 对等节点 directory in backup—losing trust store turns friends into strangers locally. Export before major migrations. Restored trust must match still-valid remote 密钥.

89.5 对话 and sessions

Conversation indexes and 会话 stores live in 资料 JSON/JSONL; back them with the home 资料. Mobile holds minimal cache—re-pair refreshes from home. Large media may live in 保险箱 paths included in 89.6.

89.6 保险箱与资料库

保险箱 and 资料库 文件 need filesystem-level backup alongside indexes. Chunk stores and search indexes rebuild slowly—prefer consistent snapshot while 节点 stopped. Verify random 文件 hash after restore.

89.7 审计与任务历史

Archive `audit-events.jsonl`, `task-journal.jsonl`, and 审批 queues for compliance. Rotation 策略 prevent unbounded disk use. Restored 审计 on new hardware preserves historical correlation IDs.

89.8 恢复与校验

Restore to a clean install, import 密钥 and data, start 节点 offline to verify, then enable 网络 and send test 消息. Compare 所有者 DID and 设备 list with pre-disaster records. Revoke 设备 that should not return post-restore.

89.9 灾难恢复清单

Maintain a printed or offline checklist: 所有者 密钥 backup location, 中继 bootstrap, trusted 联系人 to notify, revoke order for 设备, and last verified restore date. Run tabletop exercise annually. Store checklist without live secrets.

90. 更新与迁移

90.1 检查已安装版本

Check **About** in Social/Tauri or `envoy --version` on CLI against release notes before upgrading. Note 中继 and mobile app versions separately—mixed versions cause handshake surprises. Record build hash when reporting bugs.

90.2 更新桌面应用

Quit the app cleanly, run the installer or bundle update, relaunch, and confirm 身份 loaded in status. Desktop updates replace binaries only—资料 directory persists. Roll back binary if startup fails, not by deleting 资料.

90.3 更新 EnvoyGo

Update EnvoyGo from the app store or sideload channel your fleet uses; re-pair if release notes require new pairing schema. Test home connection and one 语音通话 after update. Keep desktop home 节点 on compatible API version.

90.4 更新 OpenClaw 扩展

Update OpenClaw/HomeClaw extensions per bundled compatibility matrix in release notes. Restart bridge after extension update. Mismatch shows as gateway errors in 智能体 status, not mesh failures.

90.5 更新中继

Upgrade 中继 binaries with `--advertise-addr` preserved; restart during low traffic if operator. Community 中继 users depend on operator schedule—private 中继 you control should follow same version as 节点. Verify reservation after upgrade.

90.6 配置 compatibility

Read migration notes for renamed config 密钥 or JSONL schema bumps. Automatic migrations run at startup; failed migration backs up `.bak` 文件 beside originals. Do not hand-edit migrated 文件 while 节点 is running.

90.7 数据迁移

Large data migrations may re-index 保险箱 or rebuild trust views—allow time on first boot after upgrade. Monitor 审计 for migration summary events. Keep pre-migration backup until indexes stabilize.

90.8 安全回滚

To roll back, install previous binary version and restore 资料 backup if new version wrote incompatible data. Revoke tokens issued only on new build if 安全 fix motivated rollback. Never roll back 所有者 密钥—only application bits.

90.9 查看发行说明

Read release notes for 安全 fixes, breaking protocol changes, and experimental flags before clicking update. Appendix J lists maturity labels for planned features. Schedule upgrades after backup, not before travel.

第 XIV 部分 —— 帮助与故障排查

91. 故障排查基础

91.1 检查节点状态

Start with the application status surfaces: confirm the 节点 service is running, 身份 loaded, model/智能体 state is expected, and at least one 网络 path is available.

91.2 安全重启

Quit Social or the Tauri wrapper cleanly so the 节点 can flush JSONL appenders. Restart the 节点 process (or relaunch the desktop app) and wait until status shows 身份 loaded and mesh listening. If the 资料 was mid-write, check for `.tmp` 文件 beside `trust-records.json` before deleting anything.

91.3 检查连通性

Run `connectivity-status --rich` from the CLI and confirm at least one of: mDNS 对等节点, bootstrap dial, or 中继 reservation. Compare your bootstrap multiaddrs with the 联系人's advertised addresses. If direct dial fails but 中继 works, treat it as NAT/firewall—not a 绑定 or 身份 problem.

91.4 检查智能体状态

Open Settings → AI and confirm the 智能体 授权 is present and not expired. For EnvoyAI, verify OpenClaw Gateway responds on its configured port (default 18789). External 智能体 should show bridge health on port 3031; 审计 rows tagged `bridge` explain auth or timeout failures.

91.5 审查近期活动

Open Activity or run `audit --limit 40 --include-p2p-trace` and sort by time around the failure. Follow `correlationId` across rows—绑定 deny, guard reject, and 中继 forward each produce distinct summaries. Note the intent name (`chat.message`, `knowledge.query`, etc.) before changing trust or 网络 settings.

91.6 查找日志

Operational history lives in your 资料 directory as JSONL: `audit-events.jsonl`, `task-journal.jsonl`, `approval-queue.jsonl`, and `discovery-events.jsonl`. Relay operators also get 中继-manager snapshots in 中继 资料 审计 logs. Console output from `npm run node:dev` supplements but does not replace these 文件.

91.7 收集诊断报告

A useful diagnostic bundle includes version, platform, relevant configuration with secrets removed, recent logs, 审计 correlation IDs, 对等节点/中继 status, and exact reproduction steps.

91.8 分享诊断前移除隐私数据

Before sharing logs, copy only the relevant time window and redact `owner-key*`, 设备 密钥, `bridge-config.json` bearer tokens, model API 密钥, and raw envelope payloads. Replace 对等节点 display names with labels if needed; keep correlation IDs intact so support can trace flows.

91.9 向社区求助

Gather version, platform, 资料 path, reproduction steps, and redacted 审计 excerpts with correlation IDs. State which feature status label applies (Available, Beta, Experimental). Community channels are announced in release notes for 0.1.0—avoid posting secrets in public threads.

92. 安装与启动问题

92.1 安装程序无法运行

Confirm the download matches your CPU architecture and macOS/Windows version in release notes. On macOS, if Gatekeeper blocks the DMG, use System Settings → Privacy & Security → Open Anyway once. On Windows, unblock the installer 文件 property if SmartScreen quarantined it.

92.2 操作系统阻止应用

macOS: approve the app under Privacy & Security after first launch; notarized builds should not require disabling SIP. Windows: allow the app through Defender/Firewall when prompted for inbound mesh traffic. Corporate MDM may block unsigned or unknown publishers—request an exception or install from source with your own signing.

92.3 应用无法启动

Launch from 终端 with logging enabled (`npm run node:dev -- --profile <path>`) to capture startup exceptions. Verify the 资料 directory is writable and not on a sync folder that locks 文件 (iCloud, OneDrive). A corrupt `trust-records.json` or missing 所有者 密钥 prevents UI load—restore from backup rather than deleting the 资料.

92.4 节点运行时不启动

Check Node.js version against `package.json` engines and rerun `npm install` from the repo root for source installs. Packaged desktop builds embed the runtime—reinstall if the bundled binary was quarantined. Look for port conflicts on WebSocket/API ports configured in 节点 settings.

92.5 OpenClaw 运行时不可用

Confirm OpenClaw Gateway is installed and listening (default 18789). Run `./scripts/setup.sh` or `.\scripts\setup.ps1` after upgrades to refresh extensions. Windows slim bundles may omit optional extensions—compare with macOS bundle list in release notes.

92.6 所需扩展缺失

List enabled OpenClaw extensions in Gateway settings and compare with the platform bundle in Chapter 9. Re-run setup scripts to copy missing extensions into the expected paths. Mesh chat and 绑定 do not require optional channel extensions—only enable what your 智能体 workflow needs.

92.7 防火墙或杀毒警告

Allow outbound TCP/QUIC to bootstrap 对等节点 and 中继; inbound direct dial may need a firewall rule on the home 节点. Antivirus hooks on `%AppData%` or `~/.local/share/envoymesh` can block JSONL writes—add an exclusion for the 资料 path. Document which ports you opened before retrying WAN discovery.

92.8 更新失败

Ensure the updater can write beside the install directory and 资料 path. Back up the 资料 and 保险箱 before major upgrades. If auto-update fails, download the new installer manually and install over the existing app without deleting user data.

92.9 重装但不丢数据

Uninstall or replace the application bundle only—never delete the 资料 directory or `shared_vault/`. Note your absolute 资料 path from Settings or Appendix K before reinstalling. After reinstall, point the app at the same `--profile` path or restore from your encrypted backup.

93. 身份与配对问题

93.1 身份创建失败

Ensure the 资料 directory is empty or use a new `--profile` path for a fresh 所有者. Disk full or 权限 denied on 密钥 write shows in console as ENOENT/EACCES—fix filesystem access first. Do not run two 节点 against the same 资料 simultaneously.

93.2 二维码无法扫描

Increase screen brightness and 关闭相机微距模糊; QR must include the full `envoy://pair?` payload. Regenerate the invitation if it expired—tokens are time-bound. For LAN onboarding, confirm both 设备 share the same 网络 segment without client isolation.

93.3 邀请无效

Compare the scanned URI with what the sender displayed—truncated copies break signature verification. Check clock skew; some invitation formats embed expiry timestamps. Ask the sender to regenerate from Contacts → Invite rather than forwarding a screenshot.

93.4 身份验证失败

Verify the sender's public 密钥 hashes to the claimed 对等节点 ID and the envelope signature verifies. If verification fails after a 密钥 rotation, ensure revocation records propagated and both sides refreshed trust. Audit rows `malformed or unsigned envelope` indicate transport corruption or version skew, not necessarily malice.

93.5 绑定请求缺失

Bond requests require the recipient to be online or reachable via 中继 for `bond.request` delivery. Public-tier 对等节点 receive a challenge flow—not an automatic 绑定; complete referral or manual 审批. Check Activity on both sides for `bond.request` / `bond.challenge` intents.

93.6 局域网接入失败

Confirm mDNS is not blocked by guest Wi-Fi or VPN split tunneling. Print multiaddrs from the host 节点 and dial manually if discovery fails. Firewall on the host must allow inbound mesh ports for LAN onboarding handoff.

93.7 EnvoyGo 配对失败

EnvoyGo must scan a home-节点 QR while the desktop 节点 is running and WebSocket-reachable. Off-LAN pairing needs 中继/circuit path to home—verify home tunnel and pairing token in Settings → Devices. Revoke stale 设备证书 if an old phone retains a broken 会话.

93.8 恢复缺失的身份数据

Restore `owner-key.pem` and 设备 密钥 from your encrypted backup into the original 资料 path. Never invent new 密钥 for the same 所有者 ID—对等节点 will reject mismatched signatures. If only 保险箱 data is missing, re-index from backup; 身份 loss without backup cannot be cryptographically recovered.

94. 消息、文件与通话

94.1 联系人显示离线

Offline usually means no active libp2p connection—not necessarily a blocked 绑定. Run connectivity checks and confirm the 联系人's 节点 is running. Relay-assisted paths may lag behind direct; wait one heartbeat interval before assuming permanent offline.

94.2 消息未投递

Confirm 绑定 tier allows `chat.message` (direct or referred with 审批). Inspect 审计 for deny vs guard reject vs 中继 forward failure. Large payloads may hit envelope size caps—try a smaller 消息 or 文件 chunk path.

94.3 群组消息缺失

Verify all members share the same room ID and room sync completed (`chat.room.sync`). A member on an old build may not decode new room envelope versions—align versions. Check whether the missing 消息 was sent while you were offline; request room sync from the host 对等节点.

94.4 文件传输失败

Raw 文件 sharing may require 所有者 审批 when `allowRawFiles` 触发器 绑定策略. Confirm 保险箱 path safety and size limits on both 节点. If transfer stalls mid-stream, inspect 中继 circuit stability—resume after reconnect rather than duplicating sends.

94.5 语音消息无法播放

Confirm the audio codec and container match what Social/EnvoyGo expects for the release. Download completed before play—partial 文件 fail decode silently in some clients. Check 绑定策略 did not strip attachments from the chat envelope.

94.6 语音通话无法连接

Voice 通话 need working 对等节点 connectivity plus TURN/STUN when NAT blocks direct media. Verify TURN credentials in Settings and that UDP is not blocked on restrictive 网络. Both parties must be on builds that support voice signaling for the current protocol version.

94.7 后台通话通知缺失

On mobile, confirm 通知 权限 and that EnvoyGo background refresh is enabled. iOS Focus modes and Android battery savers can delay push until the app foregrounds. Incoming 通话 signaling still requires home 节点 reachability—check 中继 path if away from LAN.

94.8 TURN 配置问题

Validate TURN server URL, username, and credential expiry—stale credentials produce ICE failed states. Test with a known-good public TURN service before blaming mesh signaling. Document whether the failure is gather timeout (firewall) or 中继 allocate reject (bad creds).

94.9 重复或延迟事件

Duplicate 消息 often indicate reconnect replay—check inbound guard dedup and whether two 设备 share one 身份. Delayed events on 中继 paths are normal under load; compare timestamps in 审计, not UI order alone. If duplicates persist, ensure only one active 会话 per 设备证书.

95. 智能体、模型与工具问题

95.1 EnvoyAI 无响应

Verify OpenClaw Gateway is up and the bridge on 3031 accepts the configured bearer token. Check model router configuration and 语义防火墙 rejections in 审计. An expired 智能体 授权 stops all 智能体-role intents until renewed on desktop.

95.2 模型提供商失败

Confirm API 密钥 and base URLs for the selected provider in model settings. LiteLLM or adapter errors surface in 审计 with provider name and HTTP status. Try a local model or different provider to isolate 网络 vs quota failures.

95.3 工具缺失

Open the 工具 registry on the home 节点 and confirm the 工具 is registered for your 智能体 授权. MCP-imported 工具 require an active MCP client 会话; mesh-native 工具 need matching 能力 on the 智能体 card. Restart the 智能体运行时 after adding 工具 so the registry reloads.

95.4 工具调用被拒

Tool denial usually means 授权 `allowedActions`, 绑定 tier, or missing 能力 (`vault.retrieve`, `task.execute`, etc.). Read the 审计 deny reason verbatim—it distinguishes 绑定 deny from 授权 approval_required. Raise trust or approve the pending action on desktop rather than retrying blindly.

95.5 审批待处理

Open Approvals on desktop and resolve items matching the 任务's correlation ID. Approvals expire with 授权—check `expiresAt` if the queue looks empty but 任务 stay waiting. External 智能体 cannot approve on your behalf; only the 所有者 设备 can clear 所有者 prompts.

95.6 触发器未运行

Verify 触发器 schedules, cron expressions, and that the 节点 was running at fire time. Triggers respect 授权 bounds—disallowed actions fail silently in 审计, not always in UI toasts. Check whether experimental toggles gate the 触发器 feature for your build.

95.7 摘要缺失

Digests batch activity on a schedule—confirm 摘要 generation is enabled and the 智能体 had events in the window. Empty 摘要 may mean no qualifying 审计 rows or 语义防火墙 dropped the summary prompt. Inspect `task-journal.jsonl` for failed 摘要 任务.

95.8 记忆或会话结果异常

Session 记忆 is local to the 智能体运行时—clearing bridge cache or rotating 会话 resets context. Compare what the model returned with 保险箱 retrieval citations; RAG may pull unexpected public items. If 记忆 looks like another user's data, stop and verify 资料 path—never share one 资料 between 所有者.

95.9 外部智能体无回复

Ping the 外部智能体's 消息 port (HomeClaw 8010, Hermes 8020, OpenHuman 8021) from the home 节点 host. Confirm bridge bearer token matches on both sides and the 智能体 process logs show inbound `envoymesh-message` traffic. Compatibility presets do not guarantee the external runtime is running—start it separately.

96. 知识与浏览器问题

96.1 文件无法添加

保险箱 enforces path safety—illegal paths or symlinks outside allowed roots 拒绝添加. Check disk space and 文件 权限 under `shared_vault/`. Very large 文件 may need chunking settings; watch 审计 for size cap denials.

96.2 搜索无结果

Confirm the item was indexed: run 资料库 refresh and check 敏感度 labels match your query scope. Local search only covers your 保险箱; remote queries need 绑定-permitted `knowledge.query`. Rebuild the index if `shared_vault` was restored from backup without re-indexing.

96.3 远程知识被拒

Remote deny usually means 绑定 tier caps 敏感度—public 对等节点 only see `public` items; referred caps at `public` for 知识.query; direct caps at `friends`. Public 知识 queries are rate-limited (default 5/min)—wait for window reset. Inspect 审计 for `requested sensitivity exceeds` vs `peer is blocked`.

96.4 共享内容无法打开

浏览器 and 资料库 require `library.read` 权限 for the reader's 绑定 tier and item visibility. Confirm 内容哈希 matches if integrity check failed—do not open mismatched blobs. EnvoyGo mirrors 浏览器 through home—home 节点 must be online.

96.5 内容哈希不匹配

Re-download or re-export the item; hash mismatch means bytes changed in transit or on disk. Compare sha256 from publisher metadata with local 文件. If IPFS pin is stale, fetch from the original publisher rather than a cached gateway.

96.6 IPFS 导出失败

IPFS export is optional—confirm Helia/Kubo sidecar is running if your build includes it. Check sidecar logs for connect errors to the IPFS daemon. Omit IPFS entirely if you only need local 保险箱 storage—export is not required for mesh sharing.

96.7 `envoy://` 页面无法加载

`envoy://` pages resolve through home 浏览器 routing—verify the URI scheme handler and that the item is published. Off-LAN access needs home reachability via EnvoyGo or desktop with 中继 path. Broken hashes or missing 保险箱 paths show as blank pages with errors in home 审计.

96.8 动态更新缺失

Feed notify requires referred+ 绑定 for inbound 通知; publisher must have sent `feed.notify`. Check 绑定 tier and that feed subscription is enabled on the reader 节点. Metadata-only notify does not push full content—open 资料库/浏览器 to fetch the item.

96.9 恢复受损内容

Restore 文件 from backup into the same 保险箱 path layout; run re-index afterward. Do not hand-edit chunk manifests unless you understand 保险箱 chunking—prefer republishing from source. If corruption is widespread, isolate the 资料 and scan disk health before continuing.

97. 网络与中继问题

97.1 直接连接失败

Collect dial hints from both 对等节点 and attempt manual dial from CLI if UI shows disconnected. Symmetric NAT often blocks direct TCP—configure 中继 bootstrap and verify circuit reservation succeeds. Compare libp2p versions if identify handshake fails immediately.

97.2 本地发现 fails

mDNS requires multicast on the LAN—guest 网络 and VPNs frequently block it. Use printed multiaddrs for lab setups. Confirm both 节点 advertise the same discovery 资料 (e.g. local vs wan-default).

97.3 中继查询失败

校验引导 multiaddr includes `/p2p/<relay-id>` and 中继 HTTP check-in succeeds. Run `relay-status` and inspect 审计 for `relay.lookup` failures. Override with a private 中继 if community 中继 is down—do not disable bootstrap entirely.

97.4 社区中继不可用

Community 中继 at the default bootstrap may be busy or undergoing deploy—retry with backoff. For production, run a private 中继 with `--advertise-addr` in public mode. Circuit reservation failure often means version skew on the 中继 host, not client misconfig.

97.5 多个中继不一致

节点可能检查 in to different 中继 with 分叉的路由提示—standardize bootstrap lists across your fleet. Compare 中继-manager snapshots in 审计 for conflicting parent/child records. Prefer one organization 中继 as primary bootstrap to reduce split views.

97.6 防火墙或 NAT 限制

Map required outbound ports for bootstrap and 中继 TCP. Inbound direct dial needs port forwarding or UPnP where supported; otherwise rely on circuit 中继. Document corporate proxy rules—libp2p does not traverse HTTP proxies without explicit tunnel setup.

97.7 对等节点持续离线

Peer offline on your UI may still be online to others—verify from a third mutual 联系人 if possible. Check last-seen in 对等节点 directory and recent `system.ping` results. Long offline periods may mean sleep, 资料 migration, or revoked 设备 cert.

97.8 Agent Card 无法获取

Agent cards fetch over bonded paths—public 绑定 do not auto-fetch 工作节点 cards. Force refresh from 智能体网络 settings after 绑定 upgrade. Audit `agent.card.request` / `agent.card.response` for deny or timeout; stale cards hide 能力.

97.9 收集网络诊断

Bundle: app version, OS, 资料 path, bootstrap multiaddrs, `connectivity-status --rich` output, redacted `audit-events.jsonl` with correlation IDs, and 中继 reservation result. Include both 对等节点' perspectives for connection issues. See Appendix K.5 for CLI commands.

98. 集成问题

98.1 OpenClaw 扩展缺失

Compare installed OpenClaw extensions with Chapter 9 platform bundle lists. Re-run setup script after clone or upgrade. Windows essential set is slimmer—install missing extensions manually or switch to source/macOS bundle.

98.2 HomeClaw 无法连接

Default HomeClaw 消息 port is 8010—confirm process is listening on the home 节点 host. Bridge bearer token in `bridge-config.json` must match HomeClaw's expected secret. HomeClaw runtime is externally maintained; verify its logs independently from EnvoyMesh 审计.

98.3 Hermes 无法连接

Hermes defaults to port 8020—test with curl or netcat from localhost on the home machine. Apply the Hermes compatibility preset then restart both bridge and Hermes after config edits. Check firewall loopback rules if bridge is containerized.

98.4 OpenHuman 无法连接

OpenHuman listens on 8021 by default in compatibility presets. Confirm OpenHuman's envoymesh adapter is enabled and using the same bridge URL as desktop Settings. Treat 智能体-side errors as external-runtime issues once bridge auth succeeds.

98.5 桥接认证 fails

A 401 response usually means the Bearer token is missing or mismatched. Confirm both sides use the same secret, the header uses `Bearer`, and the URL points at the correct bridge rather than the OpenClaw gateway.

98.6 外部工具调用失败

Inspect bridge logs for 工具 name, 授权 action, and 绑定 decision on the failing 通话. External 工具 map to mesh 能力—missing `vault.retrieve` denies 知识 工具. Retry with a minimal 工具 invocation to isolate schema vs 策略 failures.

98.7 MCP 客户端无法连接

MCP 客户端连接 to the home MCP adapter—确认端口, bearer token, and that the 节点 exposes MCP when bridge is enabled. stdio MCP servers need correct command paths in registry config. Client and server must agree on protocol version supported by your build.

98.8 MCP 服务器被拒

Rejected MCP servers usually fail 能力 or auth checks at registration time. Verify server manifest 工具 do not require disallowed actions for the active 授权. Check 审计 for `missing capability for` 消息 naming the intent.

98.9 A2A Agent Card 不可用

Fetch `/.well-known/agent-card.json` from the home or 中继 public base URL with a valid bearer when required. Relay forwarding needs an active home tunnel for the token 所有者. Card JSON must be signed and fresh—republish after 能力 changes.

98.10 A2A 任务失败或未找到

Locate the 任务 id from `tasks/send` response and poll `tasks/get` with the same bearer. Map internal states via Chapter 73—`auth-required` means 绑定/授权 denial, not transport failure. Cancel only 任务 owned by the bearer-mapped 所有者; unknown id means expired or never created on this 节点.

99. 常见问题

99.1 EnvoyMesh 需要账号吗?

No central EnvoyMesh account is required. You create local cryptographic 身份 and may optionally use third-party model providers, 中继, mobile push services, or integrations that have their own accounts.

99.2 我的数据存储在哪里?

Everything lives on **your 设备**, primarily the home 节点 资料 directory: 保险箱 文件, trust store, 对话 indexes, 审计 JSONL, and 身份 密钥. EnvoyGo keeps pairing tokens and cached UI state on the phone—not a second copy of the full 保险箱 unless a feature explicitly caches media. Relays forward traffic; they are not your data store.

99.3 中继能阅读我的消息吗?

Relays can observe connection metadata and forward encrypted/signed application traffic, but they are not authorized to impersonate a sender or bypass home 策略. Avoid placing unnecessary sensitive data in routable metadata.

99.4 我可以不使用中继吗?

Yes, on the same LAN with mDNS or direct multiaddrs, or over known 对等节点 routes without 中继 reservation. Many WAN setups still use a **中继** for discovery and circuit 中继 when NAT blocks direct dial. Relay assists connectivity; it does not replace home-节点 策略 or signing.

99.5 我可以使用多台设备吗?

Yes. One **所有者身份** can authorize multiple **设备证书**—desktop Social/Tauri, additional computers, and EnvoyGo via QR pairing. Each 设备 has its own 密钥 for 审计 and revocation. Mobile is a thin client to home; it does not duplicate the full mesh 节点.

99.6 我可以使用自己的模型吗?

Yes. Configure providers in Settings → AI → Model (LiteLLM-compatible 端点, local runners, or cloud APIs you trust). The 语义防火墙 still filters prompts; 绑定 and 授权 still gate 工具 use. Provider traffic is subject to that provider's 隐私 terms.

99.7 我可以使用外部智能体吗?

Yes, through the **Ext Agent bridge** (HomeClaw, Hermes, or OpenHuman) and MCP adapters on the home 节点. External 智能体 通话 mesh 工具 (`mesh.findKnowledge`, etc.)—they do not receive raw libp2p sockets. Enable bridge auth, scope 授权, and review 审计 for external 工具 通话.

99.8 联系人离线时会怎样?

Signed 消息 queue on the sender's home 节点 and retry when a path opens—direct LAN, 中继 circuit, or later online presence. Delivery indicators may lag until the remote 节点 acknowledges. Neither side loses 消息 integrity; duplicates are avoided by protocol IDs where implemented.

99.9 陌生人能招募我的智能体吗?

No. 协作任务 require bonded 联系人 and opted-in 能力 providers. Public strangers are not recruitable 工作节点 in the current product.

99.10 我可以吊销智能体或设备吗?

Yes. **Revoke 设备证书** for lost laptops or phones from a trusted desktop 节点; revoke or narrow **智能体 授权** to stop automation. Blocked trust stops new 联系人 operations. Revocation is local and signed—对等节点 learn on next verified interaction.

99.11 EnvoyMesh 是 MCP 或 A2A 的替代品吗?

No. EnvoyMesh uses its own signed native protocol and provides MCP and A2A bridges so other ecosystems can use selected 工具, discovery, and 任务.

99.12 哪些功能是实验或计划中的?

See Appendix J for the authoritative list. In short: Beta/Experimental items are implemented but still being validated (interfaces may change); Planned items are designed but not shipped as complete features (notably video calling and broad anonymous 工作节点 discovery); Parked items are intentionally deferred without a committed date (EnvoyGo full-节点 mode, global reputation, multi-hop commerce); Deferred items are designed but not yet built (Filecoin persistence, full hierarchical 中继 graph); Future items are scoped for later interop work (MCP resources/prompts, OAuth 2.1). Always confirm against the current release notes before relying on any non-Available 能力.

第 XV 部分 —— 网站与内容体系 *（面向编辑与运维）*

> Part XV is a website and editorial content map. End users can skip this part; use Parts I–XIV and the appendices instead.

100. 网站信息架构

100.1 主页

Lead with the one-sentence value proposition (private mesh for people and 智能体), the primary install CTA, and three feature pillars (私信, personal AI, 智能体网络). Link to Use Cases and Downloads; avoid protocol jargon above the fold.

100.2 产品概述

One paragraph per pillar linking to the dedicated product page. Attach the Available/Desktop/Mobile labels and link each pillar to its in-guide chapter (messaging → Part III, personal AI → Part IV, 知识 → Part V, external 智能体 → Part VI, 智能体网络 → Part VII).

100.3 Agent Network

Frame as bonded opt-in collaboration — never "marketplace". Attach the 智能体网络 overview chapter link (§44) and the Join flow (§45). Call out that strangers cannot recruit the local 智能体.

100.4 外部智能体

List OpenClaw (bundled), HomeClaw, Hermes, OpenHuman with Compatibility-preset labels where applicable; link each to its guide chapter (§38–§42). State that only one 外部智能体 is active per bridge.

100.5 Use cases

Curate 6–8 scenarios (personal AI across 设备, family mesh, trusted research, small-team 智能体网络, Claude Desktop via MCP, A2A delegation, self-hosted 中继). Each links to the matching tutorial in §14 or use case in §5.

100.6 How it works

Plain-language architecture diagram (所有者 → 绑定 → signed 消息 → optional 中继). Link to §4 and the 安全 model page; keep Ed25519/libp2p in an expandable technical note.

100.7 Security and privacy

Summarize the Diplomat/绑定引擎/语义防火墙/保险箱 boundaries without claiming "unbreakable" 安全. Link to §84 and Appendix H checklists; surface the vulnerability-reporting 联系人.

100.8 Downloads

Per-platform cards (macOS, Windows, iOS, Android, source) with Verified badges and last-verified dates. Link to §8 install steps; surface release notes and Appendix J status boundaries.

100.9 Guide

Entry point to this guidebook: Getting Started, Everyday Use, 外部智能体, 智能体网络, Troubleshooting. Mirror the "Proposed Guide Navigation" tail of this document.

100.10 Community and support

GitHub, discussions, roadmap, release notes, support 联系人. Keep it actionable — where to 文件 bugs, where to ask questions, where to read the roadmap.

101. 产品页面

101.1 私信

Pitch signed 点对点 messaging with 绑定-gated delivery. Attach Available + Desktop + Mobile labels; link to §16 and §17. Note 群聊 and audio 消息 as related.

101.2 个人 AI

Pitch EnvoyAI/OpenClaw as the bundled assistant under 所有者 策略. Attach Available + Desktop label; link to §21–§28. Cross-reference external 智能体 for users who prefer a different runtime.

101.3 Knowledge Base

Pitch 本地优先 notes, 保险箱 文件, RAG, and Obsidian integration. Attach Available + Desktop label; link to §29–§35. Note 敏感度 labels and federated RAG as differentiators.

101.4 Agent Network and 协作任务

Pitch bonded multi-智能体 collaboration with attributed reports. Attach Available + Desktop label (EnvoyGo is read-only mirror); link to §44–§63. Emphasize "not a marketplace".

101.5 外部智能体

Pitch the safe bridge for OpenClaw/HomeClaw/Hermes/OpenHuman. Attach Compatibility-preset labels; link to §36–§43. State the one-active-bridge rule.

101.6 桌面与 EnvoyGo

Pitch two surfaces, one 身份: desktop home 节点 + EnvoyGo thin client. Attach Available + Desktop + Mobile labels; link to §8 and §13. Surface the macOS/Windows bundle difference (§9.4/§9.5).

101.7 语音与文件共享

Pitch 语音通话 (Phase 42I on iOS) and content-addressed 文件 sharing. Attach Available + Desktop + Mobile labels; link to §18 and §19. Mark 视频通话 as Planned.

101.8 终端与浏览器

Pitch remote 终端 and `envoy://` browsing. Attach Available + Desktop label (EnvoyGo mirrors); link to §78 and §79. Surface herdr/TmuxAI as external integrations.

101.9 MCP 与 A2A

Pitch MCP 工具 bridging (consumer + server) and A2A 智能体 cards/任务. Attach Experimental/Beta labels as appropriate; link to §68—§73. Note OAuth/resources as future scope.

101.10 中继与自托管

Pitch optional 中继 for connectivity and self-hosted fleet operations. Attach Operator label; link to §74—§77. Surface the community 中继 and the operator fleet guide.

102. 外部智能体网站页面

102.1 外部智能体 overview

Explain the bridge model (no raw P2P for 智能体). Attach Compatibility-preset guidance; link to §36 and Appendix C matrix. Audience: integrators choosing an 智能体运行时.

102.2 OpenClaw / EnvoyAI

Detail the bundled runtime, gateway port 18789, canonical extension, macOS/Windows bundle differences. Attach Available + Desktop label; link to §38.

102.3 HomeClaw

Detail the default preset at 8010/消息, externally maintained channel. Attach Compatibility-preset label; link to §39. State verification responsibility.

102.4 Hermes

Detail the preset at 8020/消息, 知识-oriented runtime, migration path. Attach Compatibility-preset label; link to §40.

102.5 OpenHuman

Detail the preset at 8021/消息, disabled by default, externally maintained. Attach Compatibility-preset + Planned-for-production labels; link to §41.

102.6 自定义智能体集成

Document the `envoymesh-message` adapter contract. Attach Experimental label; link to §42 and the bridge wire contract in §37. Audience: developers.

102.7 集成状态矩阵

Render Appendix C as a sortable table (智能体 × mode × port × status × last-verified). Keep it the single source of truth; every other page links here.

102.8 安全边界

Explain why 智能体 never hold Ed25519 密钥 and how Bearer auth gates `/bridge/*`. Link to §37 and §84.10; do not overstate — say "策略-checked", not "secure".

102.9 开发者交接链接

Cross-link to `docs/agent_bridge_guide.md`, `docs/openclaw-agent-bridge-adr.md`, `OpenClawExtension/`, and the MCP/A2A design docs. Audience: engineers implementing an 智能体.

103. 智能体网络网站页面

103.1 智能体网络概述

Define bonded opt-in collaboration; attach Available + Desktop label; link to §44. Emphasize "not a marketplace" and "中继 stay lean".

103.2 加入智能体网络

Step-by-step enable Join + publish 资料; link to §45 and §46. Attach screenshots of the Settings → 智能体网络 tab.

103.3 智能体身份与名片

Explain 所有者-authorized 智能体 credentials and Agent Card; link to §47. Surface the A2A Agent Card bridge as the external face.

103.4 已绑定工作节点发现

Explain card auto-fetch on 绑定 + 能力 index; link to §48 and §49. Note that broad anonymous discovery is Planned, not current.

103.5 协作任务

Define 协作任务 (product name) vs chains (code name); link to §50–§58. Attach screenshots of the Chains/协作任务 UI.

103.6 规划与分配

Explain 协调代理 plan + direct-assign vs competitive bidding; link to §51–§53. Keep LLM planner details in an expandable.

103.7 竞标与预算

Explain 授权, cost ceilings, rebalance 策略; link to §53 and §54. Surface CSV export and cost visibility controls.

103.8 多轮协作

Explain iteration (draft → judge → replan); link to §56. State default `iterationMaxRounds=1`.

103.9 结果与来源

Explain composite 交付物 and 工作节点 attribution; link to §58 and Appendix G. Emphasize that flattened anonymous answers lose provenance.

103.10 Trust and safety

Summarize 绑定 gates, 授权 limits, 敏感度 ceilings, 审批; link to §61 and Appendix H.5/H.6 checklists.

103.11 Network connectivity

Explain LAN, direct, 中继-assisted paths; link to §62 and Part X. Surface NAT/TURN guidance.

103.12 功能状态与路线图

Render Appendix J.4—J.11 as the authoritative boundary list; link each item to its design doc. Mark Planned/Parked/Deferred explicitly.

104. 可复用内容模板

104.1 页面标题

简洁、面向动作，≤ 60 字符。镜像用户搜索的名词（如“私信”、“加入智能体网络”），而非内部术语。

104.2 一句话摘要

先说读者能做什么，再说功能是什么。“向已绑定联系人发送已签名的点对点消息”胜过“使用 Ed25519 信封的消息子系统”。

104.3 可用性标签

Render the standard labels exactly: Available, Beta, Experimental, Compatibility preset, Planned, Parked, Desktop, Mobile, and Operator. A page may carry more than one label, such as Available + Desktop.

104.4 功能做什么

最多两到三句。点明用户动作、边界（涉及谁/什么）与结果。除非页面面向开发者，否则避免协议名称。

104.5 为何使用

围绕真实目标组织（隐私、控制、协作、成本）。若有多种受众，每类受众一句话——分别用“面向个人”、“面向团队”、“面向运维”开头。

104.6 开始之前

以项目符号列出硬性前提：运行中的家庭节点、已绑定的联系人、已启用的开关、已配置的模型。每个前提链接到其设置章节。

104.7 分步说明

编号步骤，每步一个动作，附精确 UI 路径（Settings → …）或命令。路径不明显处加截图或简短代码块。每步可独立校验。

104.8 幕后发生什么

可选的可折叠小节，用于协议/加密细节。用它满足技术读者，而不强迫所有人先读 Ed25519/libp2p 术语。链接到设计文档，不要重复。

104.9 隐私与安全提示

说明该功能强制执行的边界（已签名、策略受限、敏感度封顶、需审批）以及它不能防护的内容。引用安全章节，而非重述。

104.10 故障排查

三到五条“症状 → 原因 → 修复”。深入诊断请链接到对应的 §91–§98 章节。除非重启确实是修复，否则避免泛泛的“重启应用”建议。

104.11 相关主题

三到五个交叉链接，指向相邻章节与下一个合理动作。帮助读者从“设置”走到“使用”再到“排查”，无需退回目录。

104.12 最后校验版本与日期

Every page should record the last EnvoyMesh version and date against which its steps and status were checked. Re-verify after UI, protocol, packaging, or 安全 changes.

105. 编辑与术语指南

105.1 优先为终端用户写作

Address the reader directly ("you"), lead with the 任务, defer protocol internals to expandable notes. Mirror the tone of §1–§14.

105.2 技术细节渐进披露

Surface the user-facing concept first; link to the deeper guide chapter; reserve code identifiers, schemas, and config 密钥 for the technical layer. Never force a reader to learn Ed25519 to send a 消息.

105.3 产品术语与代码名

Prefer current product terms such as 协作任务 and EnvoyGo. Mention code names such as chains only when they help developers find logs, settings, or protocol references.

105.4 功能状态用语

严格使用 §“功能状态标签”中的九个规范标签（可用、Beta、实验、兼容预设、计划中、暂缓、桌面、移动、运维）。绝不创造新的状态词；若某能力不匹配，用散文说明而非新增标签。

105.5 平台标签

每个功能页面配一个平台标签（桌面、移动、运维）。若功能当前仅桌面可用但移动镜像已计划，写“桌面（移动镜像计划中）”，而非让平台含糊。

105.6 安全声明与证据

Security statements must identify their boundary and evidence. Say “signed by the sender 密钥 and checked by the inbound guard,” not “completely secure.”

105.7 集成成熟度声明

Describe HomeClaw, Hermes, and OpenHuman as compatibility presets and state that their 智能体-side runtimes are externally maintained. Do not imply equal maturity with the bundled OpenClaw integration.

105.8 无障碍与包容性语言

Use plain language, alt text for diagrams, sufficient color contrast, and avoid assumed-ability phrasing. Mirror WCAG-AA contrast in website pages.

105.9 截图、图示与替代文本

Every screenshot needs alt text describing the action, not the chrome. Diagrams should be SVG with text labels; keep ASCII diagrams as a fallback in code blocks.

105.10 翻译与本地化

Translate prose; keep brand names (EnvoyMesh, OpenClaw, etc.), code identifiers, and UI paths in English. Follow the Chinese edition glossary; coordinate locale updates with UI i18n.

105.11 版本与审查节奏

Bump the guidebook version with each release; re-verify status labels against `docs/implementation-plan.md` and Appendix J. Record the last-verified date on every website page.

附录

附录 A —— 术语表

A.1 Agent

智能体（agent）是由所有者授权、可通信或执行受限任务的 AI 身份。

A.2 Agent Card

Agent Card（智能体名片）是一份描述能力的文档（可选由中继签名），用于发现某个智能体能做什么、是否已加入智能体网络。

A.3 Agent Network

智能体网络（Agent Network）指已绑定的所有者之间，让其自愿加入的本地智能体协同工作的协作关系。

A.4 Artifact

交付物（artifact）是带类型的任务结果：文本、文件、结构化数据或复合包。

A.5 Bond

绑定（bond）是本地记录的对另一位所有者的信任关系及信任层级。

A.6 Capability

能力（capability）是已广播或已授权的操作，例如任务执行或知识查询。

A.7 Contact

联系人（contact）是出现在本地目录与关系界面中的已知所有者或智能体。

A.8 Device

设备（device）是经所有者授权、拥有独立密钥与证书的安装实例。

A.9 DID

DID（去中心化标识符）是由加密身份派生的标识呈现。EnvoyMesh 的所有者/设备/智能体 DID 分别使用 `envoy:owner:` / `envoy:device:` / `envoy:agent:` 前缀（见 §10.6）。

A.10 EnvoyAI

EnvoyAI 是 EnvoyMesh 内置的、由 OpenClaw 驱动的个人智能体体验。

A.11 EnvoyGo

EnvoyGo 是当前 iOS/Android 上与家庭节点配对的瘦客户端。

A.12 External agent

外部智能体（external agent）是通过本地 HTTP 桥接连接的、独立维护的运行时。

A.13 Library

资料库（Library）组织知识条目，供本地搜索、共享、发布与浏览使用。

A.14 Mandate

授权（mandate）是一份由所有者签名的授权文档，用于限定某个智能体任务的范围。

A.15 Owner

所有者（owner）是长期存在的人类身份与根授权密钥。

A.16 Peer

对等节点（peer）是签名并传输信封的运行时网络身份。

A.17 Relay

中继（relay）协助可达性、查询与转发，但本身不成为应用权威。

A.18 Task

任务（task）是委派工作与带类型结果的已签名生命周期。

A.19 Team job

协作任务（Team job，代码中称 chain）协调多个智能体子任务，并合并其带归属的结果。

A.20 Vault

The **保险箱** is 带路径安全的本地存储与索引 for private 文件 and 知识.

附录 B —— 功能与平台矩阵

B.1 macOS

macOS — Tauri desktop bundle with embedded 节点; fuller OpenClaw extensions; DMG/notarized install. Home 节点 runs all mesh features; EnvoyGo pairs as mirror. Profile under Tauri app data area (Appendix K.1).

B.2 Windows

Windows — Installer with slimmer OpenClaw essential set; 资料 in `%AppData%` / `%USERPROFILE%\.envoymesh\`. Allow firewall for inbound 对等节点 when prompted.

B.3 iOS 上的 EnvoyGo

EnvoyGo iOS — Flutter thin client; QR pair to home; chat, 通话, 终端, 浏览器 mirror; no standalone mesh 节点.

B.4 Android 上的 EnvoyGo

EnvoyGo Android — Same mirror scope as iOS; home 节点 must stay reachable via WebSocket/中继 when off-LAN.

B.5 仅家庭节点功能

Home-节点-only — Identity, 保险箱, 智能体, Team orchestration, MCP/A2A bridges, full Settings. Required for authoritative signing and 策略.

B.6 EnvoyGo 移动只读镜像

EnvoyGo mirrors — Read-heavy remote UI; AI engine and bridge config read-only on phone; change on desktop.

B.7 运维功能

运维 — Relay deployment, bootstrap lists, `--advertise-addr`, fleet manifest CLI; not end-user Social features.

B.8 可用、Beta、实验、计划中与暂缓功能

Status labels — Available, Beta, Experimental, Planned, Parked, Deferred per front matter; Appendix J is canonical over marketing copy.

附录 C —— 外部智能体矩阵

C.1 EnvoyAI / OpenClaw

EnvoyAI / OpenClaw — Bundled personal 智能体; Gateway default 18789; bridge 3031; EnvoyMesh-maintained extensions on desktop.

C.2 HomeClaw

HomeClaw — Compatibility preset; external runtime; 消息 port 8010; bearer auth via bridge-config.

C.3 Hermes

Hermes — Compatibility preset; external runtime; port 8020; verify adapter logs separately.

C.4 OpenHuman

OpenHuman — Compatibility preset; external runtime; port 8021; human-in-loop workflows external to mesh.

C.5 自定义 `envoymesh-message` 智能体

Custom envoymesh-消息 — HTTP 消息 adapter; you maintain 智能体 process; match bridge token and JSON schema.

C.6 兼容 MCP 的应用

MCP applications — Clients attach to home MCP adapter; 工具 map to registry; Bearer auth; no OAuth resources yet (J.11).

C.7 兼容 A2A 的智能体

A2A 智能体 — Public card at `/.well-known/agent-card.json`; JSON-RPC 任务; 中继 home-tunnel forwarding when enabled.

C.8 运行时归属与校验状态

Verification — Record last tested version/date per integration; compatibility preset ≠ equal maturity to EnvoyAI.

附录 D —— 智能体网络速查

D.1 成员资格清单

Membership checklist: 所有者 授权 valid → Join toggle on → signed Agent Card published → 能力 tags match plan → direct 绑定 to 协调代理.

D.2 工作节点资格清单

Worker eligibility: bonded direct 联系人 → remote Join enabled → card lists required 能力 → probe succeeds → not blocked tier.

D.3 协作任务状态参考

协作任务 states: track 协调代理 state machine (Chapter 64); 终端: completed/failed/cancelled; stall 触发器 rebalance 策略.

D.4 授予模式

Award modes: competitive vs single-assign per job settings; competitive waits for bids before award.

D.5 预算与再平衡策略

Budget/rebalance: 授权 `maxCost` and job budget caps; rebalance when 工作节点 offline or stall timeout fires.

D.6 迭代模式

Iteration modes: single-round vs multi-round collaboration; 所有者 审批 may pause between rounds.

D.7 交付物类型

Artifact types: text, 文件, structured, composite—validate hash and 敏感度 before merge (Appendix G).

D.8 故障排查 decision tree

Decision tree: 绑定? → card fresh? → 能力 match? → 授权 OK? → 审计 correlation → then 网络/probe.

附录 E —— 信任层级参考

E.1 自身

Self is the 绑定 tier for your own 所有者, 设备, and locally authorized 智能体. `evaluatePolicy` returns `{ action: "allow", maxSensitivity: "private" }`—the highest ceiling. Mandates and 能力 checks still apply; self tier does not bypass inbound guard or 语义防火墙.

E.2 直接

Direct (friends tier) is a mutual 绑定 with explicit trust. Policy allows intents subject to `limitSensitivity(requested, "friends")`—friends-tier 知识 and 资料库 reads proceed; trusted/private items require 所有者 审批 when requested 敏感度 exceeds friends. Chat, 任务, and 智能体网络 工作节点 discovery among direct 绑定 are the default collaboration path.

E.3 介绍

Referred is introduction-backed trust—stronger than public, weaker than direct. `knowledge.query` caps at **public** 敏感度; `library.read` caps at **friends** visibility. `feed.notify`, intro intents, `system.ping`, and `bond.request` are allowed at public 敏感度; most other intents return **`approval_required`** (`referred peer requires approval`).

E.4 公开

Public is stranger/unbonded tier. Allowed: `system.ping`, `social.intro.sync`, public `knowledge.query`, public `library.read`, with rate limits on public 知识 (default 5 queries/minute). `bond.request` and `social.intro.propose` return **`challenge`** (referral or manual 审批). All other intents are **`deny`** (`public peers cannot use this intent`).

E.5 阻止

Blocked 对等节点 are hard-denied: `evaluatePolicy` returns `{ action: "deny", reason: "peer is blocked" }` for every intent. Use block for abuse or revoked relationships; unblock requires explicit trust restoration. Blocked status is local—your 节点 will not send or accept application traffic regardless of remote reachability.

E.6 典型权限

Typical 权限 by tier (before 授权 and 能力 gates):

Tier	Chat / 任务	Knowledge query max	资料库 read max	Agent card fetch
Self	Yes (local)	private	private	N/A
Direct	Yes	friends	friends	Yes (bonded)
Referred	Approval usually	public	friends	After 审批

Tier	Chat / 任务	Knowledge query max	资料库 read max	Agent card fetch
Public	Deny	public (rate-limited)	public	Challenge/deny
Blocked	Deny	Deny	Deny	Deny

Raw 文件 sharing (`allowRawFiles`) always returns **`approval_required`** regardless of tier.

E.7 知识敏感度限制

Knowledge-敏感度 limits use ordered ranks: public < friends < trusted < private. Bond tier sets the ceiling; requesting higher 敏感度 yields **`approval_required`** (`requested sensitivity exceeds <tier>`). Item visibility in handlers is checked against `maxSensitivity` from 策略—not 绑定 tier alone.

E.8 智能体网络资格

智能体网络 eligibility requires direct (or higher) 绑定 for 工作节点 discovery and card fetch; public 绑定 do not auto-fetch 智能体 cards. Workers must opt in (`capabilityProviderEnabled`) and advertise matching 能力 tags on signed Agent Card. 协作任务 still enforce 授权 bounds, budgets, and per-action 审批 independently of 绑定 tier.

附录 F —— 任务状态参考

F.1 EnvoyMesh 状态

EnvoyMesh states: `created` → `planned` → `discovering` → `negotiating` → `waiting_for_peer` | `waiting_for_owner` → `running` → `partial` → `completed` | `failed` | `cancelled` (Chapter 65).

Typical transitions: `created` → `planned` (协调代理 accepts the objective); `planned` → `discovering`|`negotiating` (工作节点 search or bid exchange); `negotiating` → `waiting_for_owner` (审批 needed); `running` → `partial` (interim result, more work pending); `partial` → `completed` (final merge); any non–终端 state → `cancelled` (所有者/对等节点/策略 cancel). `completed`, `failed`, and `cancelled` are 终端.

F.2 有效状态转换

Valid transitions: forward along lifecycle; `partial` may precede 终端 success; reject/cancel intents from negotiating or running per 授权.

F.3 终态

终端 states: `completed`, `failed`, `cancelled`—no further 任务 intents except 审计; collect–N 授权 may close early on first completion.

F.4 A2A 状态对应

A2A equivalents: twelve internal states map to nine A2A states via `a2a-state-map.ts` (Chapter 73)—document mapping for client UX.

F.5 取消行为

Cancellation: 所有者 or 授权 holder sends `task.cancel`; in–flight work should heartbeat until ack; A2A clients use `tasks/cancel` for tracked ids.

附录 G —— 交付物与内容映射

G.1 文本交付物

Text 交付物 — UTF-8 summaries and chat extracts; map to A2A Text Parts; apply 语义防火墙 before model ingestion.

G.2 文件交付物

File 交付物 — 保险箱-backed paths with optional `?hash=` verification; size and path safety enforced at serve time.

G.3 结构化交付物

Structured 交付物 — JSON with schema hints; validate before automation; map to MCP/A2A Data Parts.

G.4 复合交付物

Composite 交付物 — Bundles of child 交付物 with attribution weights; expand to multiple Parts when bridging.

G.5 MCP 内容映射

MCP mapping — Tool results become typed content blocks; preserve correlation IDs for 审计 stitch.

G.6 A2A Part 映射

A2A Part mapping — Text/Data/File Parts ↔ native 交付物 types (Chapter 73); hash-check File Part URIs before fetch.

附录 H —— 隐私与安全清单

H.1 首次设置

First-time setup: create 所有者 密钥 → backup `owner-key.pem` → first 设备 cert → set display 资料 → test ping with low 敏感度.

H.2 添加联系人

Add 联系人: verify out-of-band 身份 → scan full QR → complete 绑定/challenge → start at referred unless mutual direct trust intended.

H.3 添加设备

Add 设备: 所有者-signed 设备证书 → record 设备 ID → pair EnvoyGo or secondary desktop → revoke lost 设备 promptly.

H.4 连接外部智能体

External 智能体: generate bridge bearer → compatibility preset → test localhost port → minimal 工具 通话 → review 审计 before broad 授权.

H.5 加入智能体网络

Join 智能体网络: direct 绑定 → enable Join → publish card → refresh 工作节点 → trial single-工作节点 任务 before 协作任务.

H.6 发起协作任务

Start 协作任务: 授权 bounds set → eligible 工作节点 visible → plan approved → budget/deadline realistic → monitor heartbeats.

H.7 运营中继

Operate 中继: `--advertise-addr` for WAN → bootstrap multiaddr documented → monitor 中继-manager 审计 → no LLM/保险箱 on 中继 host.

H.8 应对丢失设备

Lost 设备: revoke 设备 cert immediately → rotate bridge tokens if exposed → review 审计 for post-loss traffic → re-pair from backup 所有者 密钥 only on trusted hardware.

附录 I —— 速查卡

I.1 配对联系人

Pair 联系人: Contacts → Invite → show QR → other scans → complete 绑定 flow → confirm direct/referred tier in trust UI.

I.2 配对 EnvoyGo

Pair EnvoyGo: home Settings → Devices → show pair QR → scan in EnvoyGo → verify WebSocket connected on phone.

I.3 更改信任

Change trust: open 联系人 → 信任层级 → confirm 策略 implications (Appendix E) → approve if lowering requires re-绑定.

I.4 添加知识

Add 知识: 资料库 → Add → pick 保险箱-safe path → set 敏感度 label → re-index if search misses.

I.5 批准动作

Approve action: Desktop Approvals queue → read 授权 context → Allow/Deny → 任务 resumes or cancels per 策略.

I.6 连接外部智能体

Connect 外部智能体: Settings → External 智能体 → preset → paste bearer to 智能体 config → test 消息 round-trip.

I.7 加入智能体网络

Join 智能体网络: Settings → 智能体网络 → Join → verify card published → Refresh 工作节点 on 对等节点.

I.8 发起协作任务

Start 协作任务: 智能体网络 → New job → select 工作节点 → set 授权 → launch → watch state in job panel.

I.9 取消任务

Cancel 任务: open 任务 → Cancel → confirm 授权 allows cancel → 审计 records 终端 cancelled state.

I.10 吊销设备

Revoke 设备: Settings → Devices → Revoke → confirm cert revoked → remove pairing on 设备 app.

I.11 收集诊断

Collect diagnostics: Appendix K bundle checklist → redact secrets → attach correlation IDs → CLI connectivity-status.

附录 J —— 状态与路线图边界

J.1 可用功能

Available (0.1.0) — intended for current use on supported platforms:

- Signed messaging, 群组, audio 消息, 语音通话, 文件/资料 sharing (Chapters 11–14)
- Personal AI via EnvoyAI/OpenClaw and external-智能体 bridges (Part VI)
- 保险箱, 资料库, 知识 query, 浏览器/`envoy://` publishing (Part V)
- 智能体网络, 协作任务, 授权, 审批 (Part VII)
- 终端, 中继, MCP 工具 bridge, A2A 智能体 card + JSON-RPC 任务 (Parts VIII–IX, X)
- Desktop Social (macOS/Windows) and EnvoyGo thin client (iOS/Android) per Chapter 9

Confirm exact packaging in release notes before production rollout.

J.2 Beta 与实验功能

Beta / Experimental — implemented but still receiving validation; interfaces may change:

- Experimental toggles in Settings (§80.11)—enable only on non-production 资料
- MCP stdio live servers and extended interop smoke paths (Phase 48 docs)
- A2A home-tunnel forwarding and 交付物 mapping edge cases (Part IX)
- IPFS/Helia sidecars when bundled—optional content experiments, not core chat
- Multi-中继 coordination under load—works but operator tuning may be required

Report issues with the **Beta** or **实验** label and redacted 审计 excerpts.

J.3 平台专属功能

Platform-specific boundaries:

- **macOS desktop** — fuller OpenClaw extension bundle; Tauri notarization path (Chapter 9.2–9.4)
- **Windows desktop** — slimmer extension set; user AppData 资料 paths (9.3, 9.5)
- **EnvoyGo iOS/Android** — thin client only: chat, 通话, 终端, 浏览器 mirror, read-only Team status; no local 保险箱, 智能体运行时, or MCP/A2A server (9.1, 9.9)
- **Home-节点-only** — mesh 身份, 保险箱 indexing, Team orchestration, bridge 端点, full Settings (9.8)
- **运维** — 中继 binary, fleet manifests, bootstrap tuning (Part X, Appendix K)

Do not infer desktop availability from mobile mirrors or vice versa.

J.4 计划中的视频通话

Planned. Voice calling is available; video calling remains architecturally anticipated but is not a current user feature.

J.5 计划中的广泛或匿名发现

Planned boundary. Contact- and 能力-scoped discovery exists, but open anonymous 工作节点 recruitment and marketplace behavior are not current 智能体网络 features.

J.6 暂缓：EnvoyGo 作为完整 mesh 节点（EnvoyGo 仍为瘦客户端）

Parked. EnvoyGo remains a home-paired thin client. Running it as an independent full mesh 节点 has no committed release.

J.7 暂缓的全球信誉

Parked. Local feedback and reputation signals exist, but a federated global reputation ledger is intentionally deferred.

J.8 暂缓的多跳商业

Parked. Multi-hop commerce, payment, and receipt workflows are outside the current collaboration product.

J.9 Deferred Filecoin 持久化

Deferred. Helia and Kubo IPFS paths are available, but Filecoin-based long-term persistence is not part of the current release.

J.10 延期的分层中继图

Deferred. Multi-中继 sibling coordination exists; a full hierarchical 中继 graph is not complete.

J.11 未来 MCP 资源与 OAuth

Future. MCP currently focuses on 工具 and Bearer-authenticated bridges. Resources, prompts, and OAuth 2.1 remain future interoperability work.

J.12 其他路线图参考

Other roadmap references (documented direction, not current general features):

- Video calling (J.4)—voice only today
- Broad/anonymous 工作节点 recruitment (J.5)
- EnvoyGo as full mesh 节点 (J.6—parked; thin client remains product path)
- Global reputation ledger (J.7), multi-hop commerce (J.8)
- Filecoin persistence (J.9), full hierarchical 中继 graph (J.10)
- MCP resources/prompts/OAuth 2.1 (J.11)

See `docs/implementation-plan.md` for phase numbers; design docs alone do not imply shipment.

附录 K —— 支持参考

K.1 应用数据位置

EnvoyMesh keeps state outside the application install directory. **Source / developer runs** default to `./data/default` for the 资料 (身份, trust, 任务, 审批, bridge config) and `./shared_vault/` for 资料库 content. **Packaged desktop builds** use OS-specific user data paths (for example `~/.local/share/envoymesh/` on Linux, `%AppData%` or `%USERPROFILE%\.envoymesh\` on Windows, and the Tauri app data area on macOS—confirm the exact path in release notes for your installer). The 保险箱 may appear as `shared_vault/` beside the 资料 or under a `vault/` subdirectory depending on platform packaging. Always back up the whole 资料 directory **and** the 保险箱 together before migration. Include only the relevant subtree in support bundles; remove `owner-key*`, 设备 密钥, `bridge-config.json` secrets, model API 密钥, and unrelated personal 文件.

K.2 默认端口

Common defaults include 外部智能体桥接 `3031`, OpenClaw Gateway `18789`, 中继 HTTP `15432`, and HomeClaw/Hermes/OpenHuman 消息 ports `8010` / `8020` / `8021`. Confirm configuration because operators may override every value.

K.3 公开 endpoints

Public A2A routes include `/.well-known/agent-card.json` and `/.well-known/a2a/jsonrpc` when the 中继 bridge is enabled. Keep home-only bridge and administrative 端点 private.

K.4 日志位置

Primary operational history is append-only **JSONL** in the 资料 directory, not a separate syslog tree. Key 文件 include `audit-events.jsonl` (allow/deny outcomes and connectivity traces), `task-journal.jsonl`, `approval-queue.jsonl`, `discovery-events.jsonl`, and `share-events.jsonl`, plus JSON state such as `trust-records.json` and `peer-directory.json`. Relay operators also generate 中继-manager snapshot rows inside 中继 资料 审计 logs. Console output from `npm run node:dev` or the desktop wrapper is supplementary—prefer redacted 审计 excerpts with correlation IDs when opening support tickets. Strip bearer tokens, envelope payloads, and 密钥 material before sharing any log 文件.

K.5 诊断命令

From the repository root (adjust `--profile` to your absolute 资料 path):

```
npm run typecheck                    # TypeScript build check
npm test                             # Unit tests
npm run test:orchestrator -- dev     # Fast dev loop (~35s, no E2E)
npm run test:orchestrator -- full    # Full gate incl. libp2p E2E + smoke (~10 min)
npm run node:dev -- --profile ./data/default
npm run cli -w @envoymesh/node -- --help
npm run cli -w @envoymesh/node -- connectivity-status --profile ./data/default --rich
npm run cli -w @envoymesh/node -- relay-status --profile ./data/default
npm run cli -w @envoymesh/node -- audit --profile ./data/default --limit 40 --include-p2p-trace
```

See `QuickStart.md` for setup scripts, cross-网络 中继 walkthroughs, and the end-to-end verification checklist. Global `envoymesh doctor` is available after `npm i -g .` from the repo root.

K.6 常见错误主题

EnvoyMesh surfaces failures by **theme** in 审计 summaries, CLI output, and bridge responses rather than a single printed error-code handbook. Common patterns:

- **`auth-required`** — bearer or 会话 authorization failed (missing/invalid token, or 信任层级 too weak for the requested A2A/MCP/任务 action). Fix pairing tokens, bridge secrets, or 绑定 level before retrying.
- **Bond deny** — `evaluatePolicy` returned deny (for example `peer is blocked`, `public peers cannot use this intent`, expired 授权, disallowed action, or 敏感度 above 授权). Inspect 信任层级 and 授权 bounds; raising trust requires explicit human 审批, not a connectivity tweak.
- **Schema / guard reject** — inbound guard rejected malformed, oversized, replayed, or unsigned envelopes (`malformed or unsigned envelope`, `envelope exceeds maximum size`, `replayed message`). Usually indicates version skew, corrupt payloads, or attack traffic—not a 中继 routing issue.

A2A JSON-RPC may also return `-32001` with an `auth-required:` 消息 when Authorization headers are missing on 中继-proxied 任务 端点. Capture the 审计 row's `summary` and `correlationId` instead of inventing numeric codes when filing issues.

K.7 支持与社区链接

In-repo documentation: start with `QuickStart.md`, `README.md`, `docs/implementation-plan.md`, and the scenario/design docs referenced from QuickStart (for example `docs/UserStory.md`, `docs/scenarios.md`).

Source repository: https://github.com/allenpeng0705/EnvoyMesh — use GitHub Issues for bug reports and feature discussion when that repository is your distribution channel. There is no separate commercial support portal documented in this release; enterprise operators should maintain internal runbooks.

Before opening an issue: reproduce on a current build, note platform (macOS/Windows/EnvoyGo), 资料 path, feature status label (**Beta** / **实验**), and redacted `audit-events.jsonl` excerpts with correlation IDs. Placeholder community chat/forum links are not bundled with 0.1.0—watch release notes for official channels as they are announced.

> **面向网站编辑的信息架构建议。** 下方两个列表不是终端用户章节。它们是根据本指南结构推导的公开网站导航骨架建议。编辑应将其作为起点，并根据实际网站信息架构调整。

建议的网站主导航

- 产品
- 智能体网络
- 外部智能体
- 用例
- 工作原理
- 安全
- 下载
- 指南
- 社区

建议的指南导航

- 入门
- 对话与共享
- 个人 AI
- 知识与资料库
- 外部智能体
- 智能体网络与协作任务
- 任务与交付物
- MCP 与 A2A
- 网络与中继
- 隐私与安全
- 设置与数据
- 故障排查
- 常见问题